Native astrodynamics in Rust

Release 0.1.5 adds a trajectory gym and matched PyKEP benchmarks.

`pykep-rust` is an independent native Rust port of the numerical C++ library in pykep version 3 (`kep3`). The reusable `pykep-core` crate contains the astrodynamics implementation without a C or C++ runtime dependency. The optional `pykep-rust` Python distribution exposes the same implementation through PyO3.

New to orbital mechanics? Start with Astrodynamics for programmers, which builds from gravity and orbits to gravity assists, low thrust and trajectory optimization with fcmaes, AI agents and multi-machine campaigns.

This book connects the narrative material for both interfaces:

- examples and quick starts provide runnable Rust and Python entry points;
- numerical conventions define units, epochs, frames, array layouts, tolerances, and error behavior;
- the dynamics, ephemeris, propagation, and low-thrust guides explain algorithm-specific contracts;
- validation records current parity and independent checks; the stabilization evidence preserves the historical pre-0.1.0 coverage, performance, and release-candidate snapshot.

Exact Rust types, methods, and compiled examples are in the generated `pykep-core` API reference. Python users should start with the Python API contract and migration matrix.

This project is not an official ESA release. Its pinned upstream source and MPL-2.0 adaptation policy are recorded in the GitHub repository.

Astrodynamics for programmers

This chapter is for curious programmers who have never studied orbital mechanics. It explains the ideas behind interplanetary trajectory design, from gravity and orbits to gravity assists and low-thrust propulsion, and shows each one in a few lines of `pykep-rust`. The second half turns those ideas into optimization problems and solves them with fcmaes-rust. The last two sections show how an AI agent can run the optimization through optimization-jobs, and how experiment-craft organizes repeated runs on one or more machines.

No physics background is assumed beyond "things fall down". Every number quoted below comes from a script in this repository, so you can rerun and change it:

```
python -m pip install pykep-rust            # 0.1.5 or later includes the gym
python python/examples/astrodynamics_tour.py
```

The tour prints one line per section of this chapter. Its source, `python/examples/astrodynamics_tour.py`, is also a handy starting point for your own experiments.

1. Gravity and orbits

Every mass attracts every other mass. For a small spacecraft near a much heavier body, such as the Sun, only the heavy body matters, and the spacecraft's acceleration points towards it:

```
acceleration = −μ · r / |r|³        with  μ = G · M
```

`r` is the position relative to the body's center. The single number `mu`, the *gravitational parameter*, is the gravitational constant times the body's mass. It is known far more precisely than either factor alone, so `pykep-rust` uses it directly: `pk.MU_SUN` is about `1.327e20 m³/s²`, and `pk.MU_EARTH` is about `3.986e14 m³/s²`.

This *two-body problem* has an exact solution. The path is always a conic section: a circle or ellipse for a bound **orbit**, or a hyperbola for an object that arrives with enough speed to escape again. Six numbers, the *classical orbital elements*, describe an orbit and the position on it:

Element	Symbol	Meaning
Semi-major axis	`a`	Size: half the longest diameter of the ellipse

Element	Symbol	Meaning
Eccentricity	`e`	Shape: 0 is a circle, close to 1 a long thin ellipse, above 1 a hyperbola
Inclination	`i`	Tilt of the orbital plane against the reference plane
Longitude of the ascending node	`Ω`	Where the orbit crosses the reference plane going “up” (for Earth orbits, the right ascension of the ascending node)
Argument of periapsis	`ω`	Where in the plane the closest approach (*periapsis*) lies
True anomaly	`ν`	Where the object is right now, measured from periapsis

An equivalent description is the *Cartesian state*: position and velocity, six numbers `[x, y, z, vx, vy, vz]`. Most algorithms work with states, and `pykep-rust` converts between the two. Kepler’s third law gives the period of an ellipse, `T = 2π √(a³/μ)`:

```
import math
import pykep_rust as pk

AU = pk.ASTRONOMICAL_UNIT                       # mean Earth-Sun distance, m
elements = [AU, 0.0167, 0.0, 0.0, math.radians(102.9), 0.0]   # Earth-like
state = pk.classical_to_cartesian(elements, pk.MU_SUN)
period = 2 * math.pi * math.sqrt(AU**3 / pk.MU_SUN)
later = pk.propagate_lagrangian(state, period / 2, pk.MU_SUN)
```

The period is 365.26 days and the speed at periapsis 30.29 km/s. Half a period later the planet is at its farthest point, the *apoapsis*, at `a(1 + e) = 1.0167 AU`. `propagate_lagrangian` computes that in closed form by solving Kepler’s equation; no step-by-step simulation is needed while only one body pulls.

Units are SI throughout: meters, seconds, kilograms, m/s. Dates are *epochs*, counted in days since 2000-01-01 00:00 (MJD2000). Positions around the Sun are heliocentric in the ecliptic J2000 frame, whose reference plane is Earth’s orbit. The numerical conventions list every convention.

2. Planets move: ephemerides

A trajectory has to meet planets that are themselves moving. An **ephemeris** is a function from an epoch to a planet’s state. `pykep-rust` ships three: a Keplerian ellipse, JPL’s low-precision analytical model for Mercury to Neptune (1800–2050), and the VSOP2013 theory for higher accuracy.

```
epoch = pk.Epoch.from_iso("2033-01-01")
earth = pk.Planet.jpl_low_precision("earth")
mars = pk.Planet.jpl_low_precision("mars")
r_earth = earth.state(epoch.mjd2000)        # [x, y, z, vx, vy, vz]
```

On 2033-01-01 Earth and Mars are 1.852 AU apart. Because the two planets circle the Sun at different speeds, their relative geometry repeats only every *synodic period*, about 26 months for Earth and Mars. That is why Mars missions launch in short **launch windows** roughly two years apart. See ephemerides for the providers and their accuracy.

3. Changing orbits: impulses and the rocket equation

A rocket changes its velocity by throwing propellant backwards. A short burn is modeled as an instantaneous velocity change, an **impulse** of size `Δv` ("delta-v"). Delta-v is the currency of mission design, because the propellant it costs grows exponentially, as given by Tsiolkovsky's **rocket equation**:

```
m_final / m_initial = exp(-Δv / (Isp · g0))        g0 = 9.80665 m/s²
```

The *specific impulse* `Isp` measures engine efficiency, and `Isp·g0` is the exhaust velocity. For `Δv = 5 km/s`:

Engine	Isp	Mass left after 5 km/s
Chemical	320 s	20.3%
Electric (ion)	3000 s	84.4%

Electric engines are ten times more efficient but deliver only a fraction of a newton of thrust. Section 7 shows why that changes the mathematics.

The textbook two-impulse transfer between two circular orbits is the **Hohmann transfer**: half an ellipse touching the inner orbit at periapsis and the outer orbit at apoapsis.

```
total, duration, impulses = pk.hohmann(AU, 1.524 * AU, pk.MU_SUN)
```

Between orbits of Earth's and Mars's radius it costs 5.60 km/s in two burns of 2946 m/s and 2650 m/s, and takes 259 days. Real planets are not on circular coplanar orbits and are rarely in the right place, so practical design needs something more general.

4. Lambert's problem and the porkchop plot

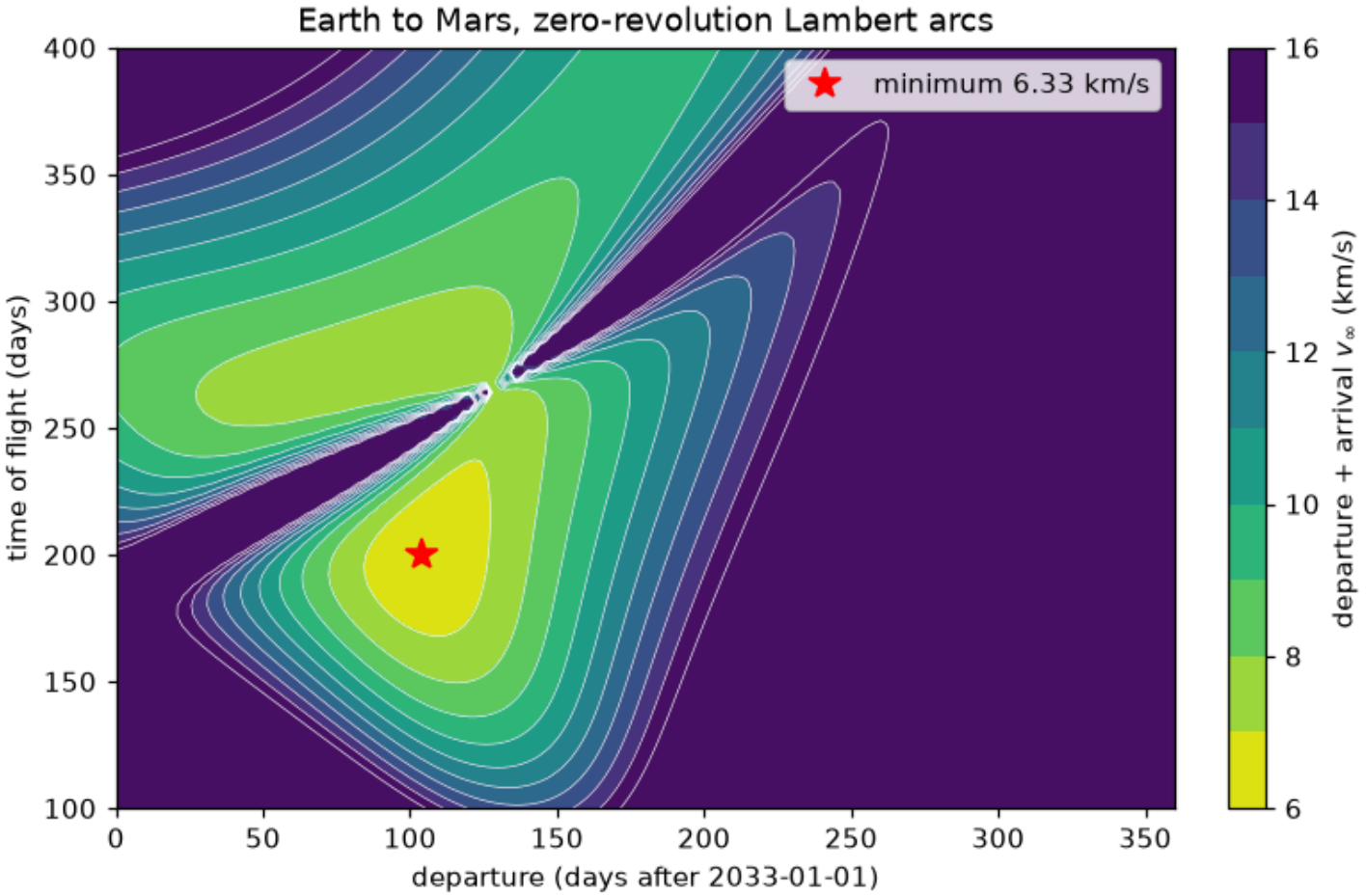
Lambert's problem is the workhorse of trajectory design. Given two positions and a time of flight, find the conic arc that connects them. For a launch on day `t0` and arrival `T` days later, take Earth's position at `t0` and Mars's at `t0 + T`, solve Lambert's problem, and compare the spacecraft's velocities with the planets' velocities. The differences are the **hyperbolic excess velocities** `v∞`: the speed relative to the planet, after escaping it or before being captured. The launcher must provide the departure `v∞`, often quoted as `C3 = v∞²`. The arrival `v∞` must be removed by a braking burn for a **rendezvous**, when the spacecraft has to stay with the target, or kept for a **flyby**.

Scanning a grid of launch dates and flight times gives the famous **porkchop plot**. `pykep-rust` solves independent Lambert problems in parallel batches:

```
import numpy as np

DAY = pk.DAY_TO_SECONDS
departures = epoch.mjd2000 + np.arange(0.0, 361.0, 6.0)
flight_days = np.arange(120.0, 361.0, 10.0)
grid = np.array([(t0, tof) for t0 in departures for tof in flight_days])
start = earth.states(grid[:, 0])                    # N x 6, parallel
end = mars.states(grid[:, 0] + grid[:, 1])
problems = pk.lambert_problem_batch(
    np.ascontiguousarray(start[:, :3]), np.ascontiguousarray(end[:, :3]),
    grid[:, 1] * DAY, pk.MU_SUN, maximum_revolutions=0,
)
departure_vinf = [np.linalg.norm(np.subtract(p.solutions[0].departure_velocity,
s[3:]))
                  for p, s in zip(problems, start)]
```

The best of the 1,525 arcs leaves on 2033-04-13 and arrives 200 days later, with `v∞ = 3.00 km/s` at departure and 3.34 km/s at arrival. A finer grid gives the picture below. The two lobes are the two families of transfers, shorter and longer than half a revolution. The ridge between them is the degenerate case of exactly half a revolution, where the transfer plane is undefined.



`maximum_revolutions` above 0 also returns arcs that circle the Sun several times before arriving; examples shows them.

5. Flybys and gravity assists

Passing close to a planet bends the spacecraft's path. Seen from the planet, the spacecraft flies a hyperbola: it arrives and leaves with the same speed $|v_\infty|$, but in a different direction. The closer the pass and the heavier the planet, the larger the turn. A minimum altitude, the *safe radius*, keeps the spacecraft out of the atmosphere.

Seen from the Sun, this rotation of the relative velocity changes the spacecraft's heliocentric velocity, because the planet's own orbital velocity is added back. This is a **gravity assist**: free delta-v, paid for by an immeasurably small change in the planet's orbit.

```
venus = pk.Planet.jpl_low_precision("venus")
v_planet = venus.state(epoch.mjd2000)[3:]
v_in = [v + d for v, d in zip(v_planet, [4_000.0, -3_000.0, 500.0])]  # v_inf 5
km/s
v_out = pk.flyby_outgoing_velocity(
    v_in, v_planet, 1.1 * venus.radius, 0.5, venus.body_mu,  # periapsis, plane
angle
)
```

Passing at 1.1 Venus radii turns `v∞` by 82.4°. With this orientation of the turn (`beta = 0.5`), the heliocentric speed rises by 4.29 km/s without burning propellant; other orientations raise or lower it by different amounts. When the incoming and outgoing `v∞` of a planned trajectory differ in magnitude, a burn at closest approach makes up the difference: a **powered flyby**, whose cost `pk.flyby_delta_v` returns.

6. Multi-leg missions

Cassini reached Saturn by flying past Venus, Venus again, Earth and Jupiter. Such **multiple gravity assist (MGA)** trajectories chain Lambert arcs, one *leg* per pair of consecutive planets:

- The decision is the launch epoch plus one flight time per leg.
- Each leg is a Lambert arc between the planets' positions at those epochs.
- At each intermediate planet, the incoming and outgoing `v∞` must match a feasible flyby. Any mismatch is paid as a powered-flyby delta-v.
- The objective is the total delta-v: launch excess beyond what the launcher provides, all flyby corrections, and the final braking.

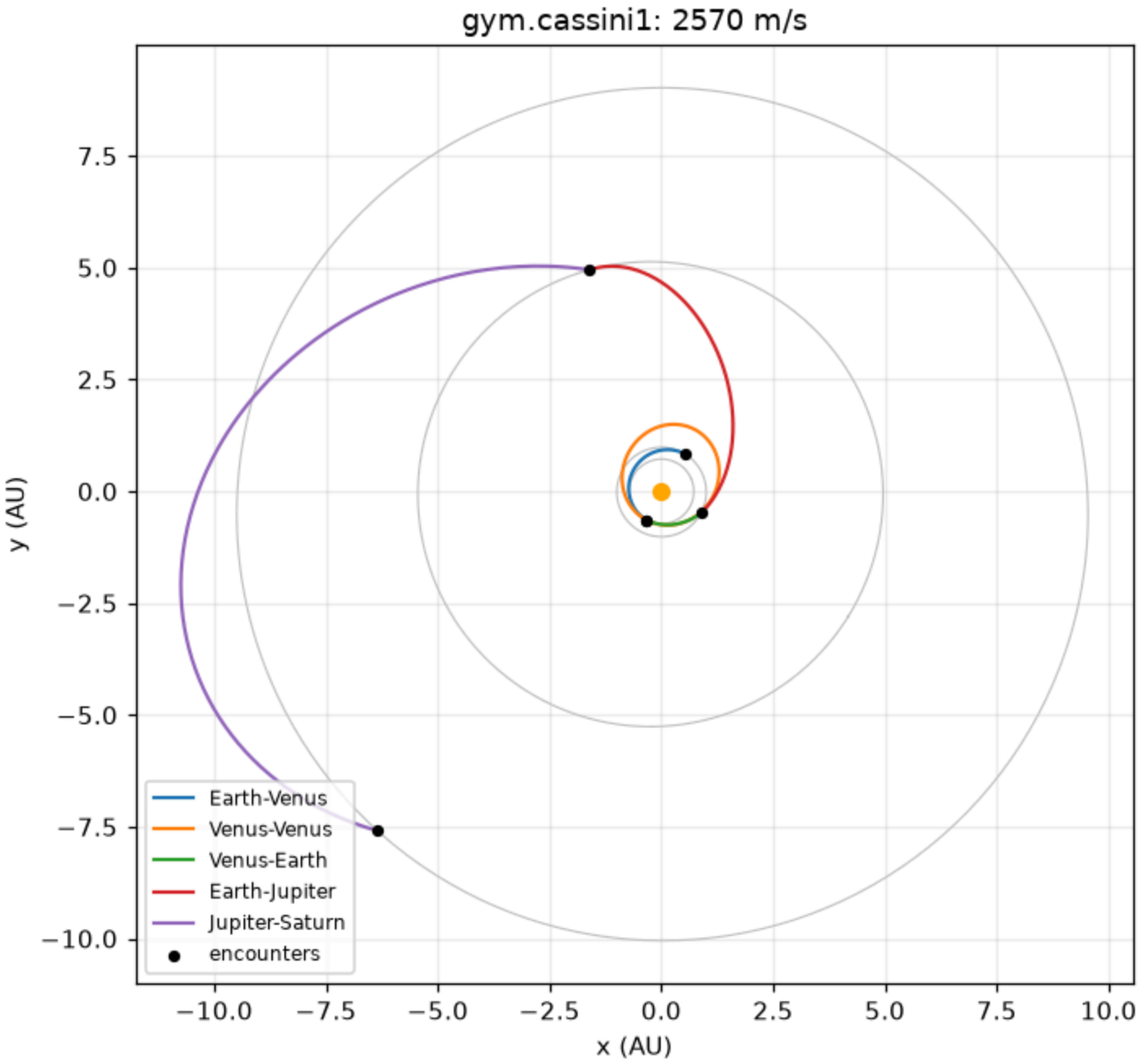
MGA-1DSM adds one *deep-space maneuver* per leg, a burn somewhere along the arc. It gives the trajectory more freedom at the price of four more decision variables per leg. Flight times can also be encoded as fractions of a total (*alpha* encoding) or of a maximum (*eta* encoding), which makes some searches easier; see impulsive trajectory formulations.

The trajectory gym contains 14 such missions from PyKEP 3.1.0: Cassini in several variants, Earth–Venus–Earth, Rosetta, Messenger, Earth–Mars with 3, 5 and 7 impulses, and JUICE with a single-objective and a two-objective version. Every problem has the same interface:

```
from pykep_rust.trajopt import gym

cassini = gym.Problem("gym.cassini1")
lower, upper = cassini.get_bounds()         # six decisions, in days
x = [-772.4793517902046, 136.48521715578022, 449.40073064324264,
     57.37453338756138, 948.5084493971979, 4434.572372085129]
cassini.fitness(x)                          # [2569.97...]   total delta-v, m/s
cassini.decode(x)["trajectory"]             # epochs, encounter states, per-
encounter delta-v
```

This decision, found by the optimizers of section 9, launches with a 2.1 km/s excess beyond the free launch speed, needs essentially no correction at the four flybys, and brakes by 461 m/s at Saturn. The flight takes 16.5 years, because the objective only counts propellant; the JUICE problems show the trade-off between delta-v and time.



This landscape is rugged. The Venus–Venus leg lasts almost exactly two Venus years, so its Lambert arc is nearly a full revolution and extremely sensitive: making it 9 seconds longer raises the total from 2570 to 3433 m/s. Optimizers must cope with many such narrow valleys.

7. Low thrust

An electric engine thrusts for months. Its path is no longer a sequence of conics joined by impulses, so the equations of motion with thrust must be integrated numerically. Mission

design then has to choose the thrust direction and magnitude over time: an infinite-dimensional choice that *transcriptions* reduce to finitely many numbers.

The **Sims–Flanagan** transcription divides a leg into segments. Each segment has a small impulse in its middle, bounded by what the engine could deliver in that segment, and coasts on conics in between. The leg is propagated forward from the departure and backward from the arrival, and the two halves must meet: the *mismatch* in position, velocity and mass at the cut must be zero.

```
leg = pk.SimsFlanaganLeg(
    departure_state, 1_500.0,              # state and mass at departure (kg)
    [[0.0, 0.0, 0.0]] * 10,                # throttle vector per segment, |u| <= 1
    mars_state, 1_500.0,                   # state and mass at arrival
    200 * DAY, 0.5, 3_000.0 * pk.STANDARD_GRAVITY, pk.MU_SUN,
)                                          # flight time, max thrust (N), exhaust
velocity
leg.mismatch_constraints()                 # 7 values that must reach zero
leg.throttle_constraints()                 # |u|^2 - 1 <= 0 per segment
leg.mismatch_jacobian()                    # analytic derivatives for gradient
solvers
```

With the launcher providing only 2.5 km/s of the 3.0 km/s the Lambert arc of section 4 needs, and the engine switched off, the leg misses Mars by 35.7 million km and 4.22 km/s. Finding throttles that close this gap, while using as little propellant as possible, is a constrained optimization problem.

A **zero-order-hold (ZOH)** transcription keeps the thrust constant on each segment and integrates the motion with a high-accuracy Taylor integrator. The gym contains the 68 **TOPS** problems built this way, for four dynamics: Sun-centered two-body motion in Cartesian or equinoctial coordinates, the rotating Earth–Moon **circular restricted three-body problem (CR3BP)** with its halo orbits, and solar sails, which use sunlight instead of propellant. Each comes with fixed endpoints and with *moving* endpoints on the departure and target orbits. See TOPS low-thrust problems.

```
import json
from pathlib import Path

record = json.loads(Path("crates/pykep-
trajopt/data/solutions/twobody.P4.json").read_text())
tops = gym.Problem(record["id"], nseg=16)
result = tops.evaluate(record["x"])
```

The bundled solution of `tops.twobody.P4` keeps 50.8% of the initial mass, and all 23 constraints hold, the largest mismatch being `1.7e-14`.

8. Trajectory optimization

All the problems above share one shape:

```
minimize f(x)   subject to   lower ≤ x ≤ upper,   h(x) = 0,   g(x) ≤ 0
```

`x` is the decision vector (epochs, flight times, throttles), `f` the objective (delta-v, or final mass with a minus sign), `h` the equality constraints (mismatches), and `g` the inequality constraints (throttle limits). The gym follows the PyGMO convention: `fitness(x)` returns the objectives, then the equalities, then the inequalities, and `get_nec()` and `get_nic()` count the constraints.

	Single leg	Multi-leg
Impulsive	One Lambert arc, two or three variables. The porkchop above is an exhaustive search.	5–30 variables, many local optima and narrow resonant valleys. No constraints, but a rugged landscape.
Low thrust	30–300 variables, a few equality constraints, smooth.	Long chains of segments, or a global sequence search over low-thrust legs.

Two families of solvers complement each other:

- **Gradient-free global optimizers**, such as differential evolution (DE), CMA-ES and BiteOpt, sample the decision space cleverly and need only `fitness`. With many independent restarts they are the method of choice for impulsive multi-leg problems.
- **Gradient-based local optimizers**, such as SLSQP or least squares, follow derivatives. They converge in a few iterations on smooth, heavily constrained low-thrust problems, but only from a good starting point. The gym's TOPS problems provide analytic gradients (`problem.gradient(x)`).

Which planets to visit, and in which order, is a discrete choice on top. In practice it is searched by an outer loop that tries sequences and optimizes each one, which is exactly the kind of work an agent can organize; see section 10.

9. Solving with fcmaes-rust

fcmaes-rust is a pure-Rust library of gradient-free optimizers (BiteOpt, DE, CMA-ES, CR-FM-NES, PGPE, MODE and more) with parallel retry strategies and Python bindings. Install it next to the pykep-rust wheel:

```
python -m pip install fcmaes-rust
python python/examples/gym/optimize_cassini1.py
```

The script runs eight BiteOpt searches with different seeds on Cassini1:

```
import numpy as np
import fcmaes_rust as fcmaes
from pykep_rust.trajopt import gym

problem = gym.Problem("gym.cassini1")
lower, upper = map(np.asarray, problem.get_bounds())

def delta_v(x):
    try:
        return problem.fitness(x)[0]
    except (ArithmeticError, RuntimeError, ValueError):
        return 1e10        # an impossible geometry is a bad point, not a crash

for seed in range(8):
    x, value, evaluations, _, _ = fcmaes.optimize_bite(
        delta_v, [], lower, upper, seed=seed, max_evaluations=50_000, M=6)
```

Seven of the eight searches stop at 2718.5 m/s, a local optimum; one finds 2570.0 m/s. That is typical: a single run of a global optimizer is a lottery ticket, so you buy many. The whole loop took about three seconds. The `try` matters: some decisions inside the bounds describe geometries the evaluator rejects, such as a flyby that would overflow, and the gym raises an exception for them.

In Rust, `fcmaes-core` runs the restarts in parallel on all cores. The example `tools/gym-benchmark/examples/cassini1_retry.rs` is complete:

```
use fcmaes_core::{BiteParams, RetryBounds, RetryConfig, RetryRunResult,
optimize_bite, retry};
use pykep_trajopt::{GymProblem, Problem, TopsOptions};

let problem = GymProblem::new("gym.cassini1", TopsOptions::default())?;
let lower: Vec<f64> = problem.spec().bounds.iter().map(|b| b[0]).collect();
let upper: Vec<f64> = problem.spec().bounds.iter().map(|b| b[1]).collect();
let objective = |x: &[f64]| problem.evaluate(x).map_or(1e10, |e| e.objectives[0]);

let config = RetryConfig { num_retries: 32, workers: 0, max_evaluations: 50_000,
seed: 42,
                           ..RetryConfig::default() };
let best = retry(&objective, &RetryBounds::new(lower, upper)?, &config,
|objective, context| {
    let result = optimize_bite(objective, context.bounds.lower(),
context.bounds.upper(), None,
        &BiteParams { max_evaluations: context.max_evaluations, seed:
context.run_seed,
                      ..BiteParams::default() }, 6);
    RetryRunResult { x: result.x, y: result.y, evaluations: result.evaluations }
});
```

```
cargo run --release -p pykep-gym-benchmark --example cassini1_retry
```

It reports 2570.0 m/s after 32 runs and 1.6 million evaluations, in under five seconds on two cores. fcmaes also offers a *coordinated retry* (`advanced_retry`): later runs get growing budgets and start from crossovers of the best earlier results, inside local bounds around them. Its retry guide explains it, and its GTOP benchmarks show results on the classic ESA trajectory problems.

Constrained problems need one more step. The gym's command-line driver turns constraint violations into a penalty, removes coordinates that the bounds fix, and runs DE, DE followed by CMA-ES, or the multi-objective MODE:

```
cargo run --release -p pykep-gym-benchmark -- optimize --problem gym.juice_mo \
  --optimizer mode --evaluations 4096 --seed 42 --output artifacts/gym/juice
cargo run --release -p pykep-gym-benchmark -- validate --run
artifacts/gym/juice/run.json
```

`validate` replays the recorded best point and fails if it no longer matches. For low-thrust problems with many equality constraints, penalties make gradient-free search slow: in the bounded study of the benchmark report, no short DE-to-CMA run reached feasibility. Use the analytic Jacobian with a gradient solver instead, as the TOPS walkthrough does with SciPy's bounded least squares, and keep global optimizers for the outer search over epochs and flight times.

10. Letting an AI agent run the optimization

An AI agent can write and run these scripts directly. For longer studies it helps to give it a narrower, auditable interface. **optimization-jobs** is such an interface:

- A **node** is a directory on a machine with a CPU budget.
- **Workers** are registered solver programs, such as `fcmaes-job-worker`.
- A **job** is a JSON document that names a registered worker, a registered problem model, an optimizer, a seed and a strict budget. It carries no code.
- The result records what was found, how much work it took, and a *claim* with its evidence. A gradient-free search claims `best_found`, and the worker re-evaluates the reported point.

Through these tools the agent cannot run arbitrary programs or exceed a budget, and every value in a result was recorded by the worker that computed it. Setup, following the job API guide (Linux):

```
# in a checkout of fcmaes-rust
cargo build --release --locked -p optimization-runner -p fcmaes-job-worker
NODE=$HOME/opt-node
target/release/optimization-run --root $NODE node init --capacity 8
target/release/optimization-run --root $NODE workers add --id fcmaes \
  --executable $PWD/target/release/fcmaes-job-worker
target/release/optimization-run --root $NODE problems describe fcmaes
# give an MCP-capable agent the 15 job tools
claude mcp add optimization-jobs -- $PWD/target/release/optimization-mcp --root
$NODE
```

A complete job, from the fcmaes job tutorial:

```
{
  "schema_version": "optimization.job/0.1",
  "request_key": "tutorial.fcmaes.cassini1.v1",
  "worker": "fcmaes",
  "payload_schema": "fcmaes.retry/1",
  "resources": {"cpu_slots": 8, "outer_workers": 8, "inner_threads": 1},
  "limits": {"wall_time_seconds": 30.0, "grace_seconds": 1.0},
  "inputs": [],
  "payload": {
    "problem": {"id": "gtop.cassini1", "version": "1", "config": {"dimension": 6},
      "bounds": {"lower": [-1000.0, 30.0, 100.0, 30.0, 400.0, 1000.0],
                 "upper": [0.0, 400.0, 470.0, 400.0, 2000.0, 6000.0], "broadcast":
false}},
    "optimizer": {"kind": "biteopt", "parameters": {"population": null}},
    "seed": 42, "retries": 16, "evaluations_per_retry": 1000,
    "evaluation_limit_mode": "hard"
  }
}
```

`gtop.cassini1` is the classic ESA GTOP benchmark model built into the fcmaes worker. It is related to, but not identical with, the gym's `gym.cassini1`, which follows the current PyKEP definition, and it reports delta-v in km/s. `optimization-run --root $NODE run --spec cassini1.json` completes all 16 retries in well under a second; `result JOB_DIRECTORY` shows each retry's best point with claim `best_found` and evidence level `rechecked`, the best one at 5.89 km/s.

With the MCP tools connected, a prompt can stay at the level of the problem, and the agent does the bookkeeping:

> Use the optimization-jobs tools. Describe the fcmaes worker, then run the `gtop.cassini1` model with BiteOpt and with CMA-ES (BIPOP restarts), seeds 1 to 4, at most 4 CPU slots and 60 seconds per job. Wait for the jobs, re-read every result, and report a table of best delta-v, actual evaluations and the claim for each run. Do not change budgets to get a better number; report failed or cancelled jobs as such.

The optimization-jobs repository has prompt-driven tutorials, including a GTOC1 route search that screens planet sequences, the discrete layer of section 8.

The gym problems of this book are not yet registered models of a job worker. Until a pykep worker exists, there are two honest routes:

- Let the agent call `pykep-gym-benchmark optimize` or the Python scripts above directly. The `run.json` they write records budget, seed and the replayable best point, and `validate` rechecks it.

- Wrap the same command in an `execution.command/0.1` job, which the node owner must enable with `optimization-run --root $NODE node commands --enable`. It is run from the CLI rather than the MCP server, and its result records exit status and output files, without an optimization claim.

A native worker would register gym IDs as fcmaes models, the way the GTOC1 application worker registers its route model.

11. Many runs, many machines: experiment-craft

A serious study repeats each optimization with many seeds, optimizers and budgets. **experiment-craft** composes such *campaigns* from optimization-jobs envelopes. A campaign declares a total budget and steps with an optional parameter *matrix*; each matrix entry becomes one job:

```
{
  "schema_version": "experiment.campaign/0.1",
  "name": "Cassini1 seed sweep",
  "budget": {"max_jobs": 8, "max_wall_seconds": 600, "max_reserved_slot_seconds":
2000},
  "steps": [{
    "id": "biteopt",
    "job": {
      "schema_version": "optimization.job/0.1",
      "worker": "fcmaes",
      "payload_schema": "fcmaes.retry/1",
      "resources": {"cpu_slots": 4, "outer_workers": 4, "inner_threads": 1},
      "limits": {"wall_time_seconds": 60.0, "grace_seconds": 1.0},
      "inputs": [],
      "payload": {
        "problem": {"id": "gtop.cassini1", "version": "1", "config": {"dimension":
6},
          "bounds": {"lower": [-1000.0, 30.0, 100.0, 30.0, 400.0, 1000.0],
                     "upper": [0.0, 400.0, 470.0, 400.0, 2000.0, 6000.0],
"broadcast": false}},
        "optimizer": {"kind": "biteopt", "parameters": {"population": null}},
        "seed": 42, "retries": 16, "evaluations_per_retry": 1000,
        "evaluation_limit_mode": "hard"
      }
    },
    "matrix": {"/payload/seed": [1, 2, 3, 4, 5, 6, 7, 8]},
    "accept": {"claims": ["best_found"]}
  }]
}
```

The campaign compiler assigns the request keys, so the job omits `request_key`. A slot-second is one declared CPU slot for one second of wall time and grace, so eight jobs of four slots and 61 seconds reserve 1,952 slot-seconds. Run and inspect it with the JSON command-line tool:

```
experiment-craft dry-run campaign.json
experiment-craft run campaign.json --workspace $HOME/campaigns/cassini1 \
  --runner $PWD/target/release/optimization-run --node-root $NODE --mode
foreground
experiment-craft status --workspace $HOME/campaigns/cassini1
experiment-craft report --workspace $HOME/campaigns/cassini1 --destination
$HOME/reports
experiment-craft dashboard --workspace $HOME/campaigns/cassini1 --state-dir
$HOME/dashboard
```

`report` writes `report.json`, `results.csv` and `report.md` from the recorded results, including failed and cancelled steps. In a test run on a two-core machine all eight jobs succeeded. `results.csv` had 128 rows, one per retry; the best value per seed ranged from 5.60 to 11.2 km/s. The best known Cassini1 solution needs 4.93 km/s (see the fcmaes GTOP benchmarks), so 1,000 evaluations per retry are far too few: the campaign makes that visible before anyone draws a conclusion. The dashboard serves a browser page on `127.0.0.1:8765` with start, pause and stop controls and progress charts for up to 32 campaign workspaces. On a remote machine, forward it with `ssh -L 8765:127.0.0.1:8765 host` and open the printed URL locally.

Several machines are supported at two levels today:

- **One campaign per machine.** Each machine has its own node and campaign workspace; split the matrix between them, monitor each machine's dashboard, and merge the reports.
- **Explicit placement.** optimization-jobs can submit a job to another registered node over SSH with `optimization-run fleet submit --node NAME --spec JOB.json`, and follow it with `fleet get`, `fleet reconcile` and `fleet execution`. It requires an SSH key, a pinned host key and a systemd user manager with lingering on the remote host. `experiment-craft deployment-export` turns a machine inventory into the exact setup, registration and probe commands for each node, without running them.

Automatic routing of one campaign across machines is not implemented yet: experiment-craft refuses a remote campaign target. The deployment guide and the dashboard reference describe the current state. The dashboard charts declared slots and step counts, not measured CPU use.

For an agent, the same commands form a complete loop: write the campaign, run `dry-run`, start it, poll `status` and `events`, and summarize `report.json`. Each step's claims and

evidence come from optimization-jobs, and the report keeps them separate from any conclusion the agent draws.

Glossary

Term	Meaning
Apoapsis / periapsis	Farthest / closest point of an orbit from the central body
C3	Square of the departure hyperbolic excess speed, the launcher's figure of merit
CR3BP	Circular restricted three-body problem: a spacecraft moving under two bodies on circular orbits, such as Earth and Moon
Delta-v (Δv)	Velocity change produced by propulsion; its propellant cost follows the rocket equation
Deep-space maneuver (DSM)	An impulse between two planetary encounters
Ephemeris	Function giving a body's position and velocity at an epoch
Epoch	A point in time; here days since 2000-01-01 (MJD2000)
Flyby / gravity assist	A close pass that rotates the relative velocity and changes the heliocentric velocity
Gravitational parameter (μ)	Gravitational constant times mass
Halo orbit	A periodic three-dimensional orbit near a Lagrange point of the CR3BP
Hyperbolic excess velocity (v∞)	Velocity relative to a planet far from it, before capture or after escape
Impulse	Instantaneous model of a short burn
Lambert's problem	Find the orbit connecting two positions in a given time
Launch window	Period in which the planetary geometry allows a transfer at acceptable cost
MGA, MGA-1DSM	Multiple gravity assist trajectory, without or with one DSM per leg
Orbital elements	Six numbers describing an orbit's size, shape, orientation and position
Porkchop plot	Map of launch cost over departure date and flight time
Rendezvous	Arriving at a target with matching velocity

Term	Meaning
Sims–Flanagan	Low-thrust transcription with one bounded impulse per segment
Specific impulse (Isp)	Engine efficiency in seconds; exhaust velocity divided by g0
TOPS	Benchmark set of low-thrust problems with piecewise-constant thrust
Zero-order hold (ZOH)	Constant control on each segment, integrated numerically

Where to go next

- Examples and quick starts has the Rust versions of most snippets.
- Trajectory optimization gym, TOPS and impulsive formulations document every problem.
- Sims–Flanagan legs and ZOH legs explain the low-thrust transcriptions and their derivatives.
- fcmaes-rust examples and its GTOC1 tutorial go further with real competition problems.
- The optimization-jobs AI context is the entry point an agent should read before using the job tools.

Documentation

The documentation describes the implemented native Rust core and its thin Python interface:

- AI problem-solving context: selecting physical models, APIs, units, frames, solver settings, and validation for a user problem;
- astrodynamics-introduction.md: a programmer's introduction to orbits, gravity assists, low thrust and trajectory optimization with fcmaes, AI agents and experiment campaigns;
- examples.md: Rust/Python quick starts and runnable examples;
- conventions.md: units, epochs, frames, array layouts, and numerical behavior;
- dynamics.md, taylor-integration.md, pontryagin.md, zero-order-hold.md, ephemerides.md, low-thrust-legs.md, and zoh-leg.md: module guides;
- trajectory-gym.md, tops.md and trajectory-formulations.md cover the trajectory catalog, optimizer examples and trajectory replay.
- matched-benchmarks.md separates current native, Python and optimizer evidence.
- python-api.md: Python units, arrays, defaults, errors, ownership, GIL, and typing contract;
- python-migration.md: upstream C++/Python names, deliberate differences, deferrals, and unsupported ecosystem modules;
- `decisions/`: architecture and dependency decisions;
- performance.md: benchmark methodology and results;
- stabilization.md: historical pre-0.1.0 matched distributions, profiling, regression limits, Miri/fuzz/Valgrind evidence, and resolved release blockers;
- status.md, source-map.md, and validation.md: limitations, provenance, and validation evidence;
- development.md: contributor commands and repository policy;
- add-ode-system.md: complete implementation, Taylor, test, benchmark, Python, and documentation checklist for a new dynamics model.

Documentation is updated only for implemented behavior. Explicitly deferred or unavailable functionality is labelled as such in the migration and status documents.

Examples and quick starts

The examples are deliberately small and deterministic. Runtime statements are orientation for a release build, not performance guarantees; use the maintained benchmark harnesses for measurements.

Rust quick start

Add the native crate and propagate a normalized circular orbit:

```rust
use pykep_core::astro::propagation::propagate_lagrangian;

fn main() -> pykep_core::Result<()> {
    let initial = [1.0, 0.0, 0.0, 0.0, 1.0, 0.0];
    let final_state =
        propagate_lagrangian(&initial, core::f64::consts::FRAC_PI_2, 1.0)?;
    assert!((final_state[1] - 1.0).abs() < 1e-12);
    Ok(())
}
```

From this workspace, run `cargo run --release -p pykep-examples --bin propagation`. The default feature set includes the embedded VSOP2013 coefficients; use `pykep-core` with `default-features = false` when they are not needed.

Python quick start

Install a wheel in a virtual environment and use the same Rust core:

```
python -m venv .venv
.venv/bin/python -m pip install target/wheels/pykep_rust-*.whl
.venv/bin/python python/examples/elements_propagation.py
```

```python
import numpy as np
import pykep_rust as pk

states = np.tile([1.0, 0.0, 0.0, 0.0, 1.0, 0.0], (4096, 1))
times = np.linspace(0.0, 1.0, len(states))
result = pk.propagate_lagrangian_batch(states, times, 1.0, workers=8)
assert result.shape == (4096, 6)
```

The Python package ships type information and accepts NumPy batch arrays only at `float64`; see python-api.md and batch-processing.md.

Runnable matrix

Capability	Rust binary	Python script	Units and expected output
Epoch/anomaly	`epoch-anomalies`	`epoch_anomalies.py`	MJD2000 days/radians; 180-day offset and exact round trips
Elements	`elements`	`elements_propagation.py`	SI/radians; finite state and stable element round trip
Propagation/STM	`propagation`	`elements_propagation.py`	Normalized or consistent SI; quarter orbit / 60-second state
Lambert	`lambert`	`lambert.py`	Normalized; seven ordered zero/multi-revolution branches
Ephemerides	`ephemeris-comparison`	`ephemeris_comparison.py`	MJD2000, metres, m/s; two finite frame-labelled states
Gravity assist	`gravity-assist`	`gravity_assist.py`	SI/radians; constraints, positive delta-v,

Capability	Rust binary	Python script	Units and expected output
			outgoing velocity
Sims–Flanagan	`low-thrust-legs`	`low_thrust_legs.py`	Consistent units; 7 mismatch values and 7 × 13 Jacobian
CR3BP/ZOH	`dynamics`	`dynamics.py`	Normalized; finite six-/seven-state propagation
Batch throughput	core Criterion benches	`batch.py`	Normalized; newly owned 4096 × 6 output

The workspace test and Clippy gates compile the Rust binaries. `python/tests/test_examples.py` runs the basic Python examples from the installed wheel. The gym has separate installed-wheel tests and replay commands.

Complete trajectory-optimization tutorial

The local trajectory gym includes native CLI examples, four feasible TOPS replays and impulsive mission and JUICE examples.

For an end-to-end application, see the GTOC1 “Save the Earth” Rust tutorial. It combines `pykep-core` ephemerides, propagation, multi-revolution Lambert solutions, gravity assists, and Sims–Flanagan legs with `fcmaes-core` optimization. The tutorial covers a reproducible fixed planet sequence, multi-fidelity sequence discovery, split-brain planet-order search, strict validation, and the distinction between a VSOP2013 model score and an official GTOC1 result.

Documentation map

- conventions.md defines units, epochs, frames, layout, and accuracy interpretation.
- Dynamics, Pontryagin, zero-order-hold, ephemeris, Sims–Flanagan, and generic ZOH-leg behavior have dedicated module guides in this directory.
- python-migration.md maps the C++/Python upstream surface and records deliberate differences and unavailable ecosystem areas.
- The `decisions/` records explain fixed-size types, errors, VSOP data, and adaptive-integration dependencies.
- performance.md gives benchmark methodology and results.
- status.md, source-map.md, and validation.md state implemented scope, limitations, and evidence without implying support for deferred modules.

Numerical conventions

Unless an API states otherwise:

- scalar computations use IEEE-754 binary64 (`f64`);
- position is in metres and velocity is in metres per second;
- gravitational parameters are in cubic metres per square second;
- durations are in seconds, while Julian-date conversions operate in days;
- angles are in radians;
- scalar ephemeris epochs use MJD2000;
- Cartesian state ordering is `[x, y, z, vx, vy, vz]` ;
- classical element ordering is `[a, e, i, Ω, ω, ν]` ;
- modified equinoctial ordering is `[p, f, g, h, k, L]` ;
- matrices use row-major nested arrays in Rust and C-contiguous row-major arrays at the Python boundary;
- public functions reject NaN and positive or negative infinity;
- deterministic batch APIs preserve input order; `workers=0` uses Rayon's shared pool, `workers=1` is serial, and `workers=N` uses a cached pool with exactly `N` workers.

Some upstream functions propagate non-finite values or use them as invalid domain sentinels. The Rust API reports explicit errors instead. Algorithm guides document any narrower valid domain and all intentional deviations.

Classical conversion reports circular and equatorial states as singular because their node/periapsis angles are undefined. Modified equinoctial elements cover those states: choose the prograde convention except at inclination `π` , and the retrograde convention except at inclination zero. Element Jacobians are `6 × 6` , with output components as rows and input components as columns.

Two-body propagation accepts any caller-consistent position, velocity, time, and gravitational-parameter units. Negative durations propagate backward. Time grids are relative to their first entry. State-transition matrices use `∂state_final/∂state_initial` , with output components as rows.

Lambert solutions are ordered deterministically: the zero-revolution solution first, then left and right solutions for each increasing revolution count. `clockwise = false` selects prograde motion as viewed from positive `z` ; collinear endpoints are rejected because that automatic direction convention is undefined.

Ephemeris scalar epochs use MJD2000 days. Returned states use SI units for built-in Solar System providers and caller-consistent units for constructed Keplerian providers. Provider

metadata uses `Option` rather than upstream negative sentinels, and hyperbolic periods are `None`.

Ephemerides

The matched report measures Keplerian, JPL low-precision and VSOP2013 wrapper dispatch and batch calls separately.

JPL low-precision planets

`JplLowPrecision` and `Planet.jpl_low_precision()` evaluate the eight heliocentric approximate planetary models carried by pykep/kep3 3.0.1: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, and Neptune. Body lookup is ASCII case-insensitive. The returned name is the lowercase body followed by `(jpl_lp)`.

Inputs are MJD2000 day counts and must satisfy the open interval `-73048 < epoch < 18263`, approximately 1800–2050. Cartesian output is `[x, y, z, vx, vy, vz]` in metres and metres per second, heliocentric and referred to the mean ecliptic and equinox of J2000. The underlying JPL table uses Julian ephemeris date/JDTDB; this library does not perform time-scale conversion. The `earth` coefficients describe the Earth–Moon barycentre, as in the source table.

The coefficients and rates come from the JPL Solar System Dynamics approximate-position table. JPL describes these as lower-accuracy fitted formulae and warns against using them outside their fitted interval. Its nominal 1800–2050 errors are:

Body	Longitude (arcsec)	Latitude (arcsec)	Distance (1000 km)
Mercury	15	1	1
Venus	20	1	4
Earth–Moon barycentre	20	8	6
Mars	40	2	25
Jupiter	400	10	600
Saturn	600	25	1500
Uranus	50	2	1000
Neptune	10	1	200

These models are suitable for approximate mission design, not precision navigation. Use a high-precision integrated ephemeris when those error bounds are too large.

The Rust provider exposes true-anomaly, mean-anomaly, and both modified equinoctial element forms. Python scalar and NumPy state, element, period, and optional-acceleration batches call the same provider. Batch order is preserved; `workers=0` uses the shared pool, one is serial, and larger values select an exact cached worker count.

VSOP2013

`Vsop2013` and `Planet.vsop2013()` implement the analytical theory for Mercury, Venus, the Earth–Moon barycentre (`earth_moon`), Mars, Jupiter, Saturn, Uranus, Neptune, and Pluto. The coefficient source is the IMCCE VSOP2013 solution; the exact adaptation and license chain are recorded beside the embedded data and in ADR 0003.

Input is an MJD2000 day count interpreted as TDB. The evaluator applies `T = (mjd2000 - 0.5) / 365250` , because the theory is measured in thousands of Julian years from J2000 at JD 2451545.0, twelve hours after the MJD2000 origin. Output is heliocentric ICRF `[x, y, z, vx, vy, vz]` in metres and metres per second. The provider does not convert UTC, TAI, TT, or TDB.

The theory was fitted to INPOP10a over 1890–2000. IMCCE also publishes comparison errors over −4000 to +8000, but accuracy degrades with distance from the fit interval and especially for Pluto. There is no artificial hard date cutoff; callers must choose a time span appropriate to their accuracy requirements. Over the fit interval, the published largest heliocentric longitude/latitude/distance differences are:

Body	Longitude (mas)	Latitude (mas)	Distance (km)
Mercury	0.06	0.01	0.008
Venus	0.02	0.05	0.002
Earth–Moon barycentre	0.02	0.08	0.011
Mars	0.93	0.06	0.162
Jupiter	0.20	0.02	0.277
Saturn	0.24	0.05	0.592
Uranus	2.19	0.13	5.962
Neptune	0.38	0.05	2.764
Pluto	10.83	3.19	118.419

The `vsop2013` Cargo feature is enabled by default. It embeds 4.3 MiB of coefficients and supports thresholds greater than or equal to `1e-9` ; the upstream default is `1e-5` . Smaller thresholds are rejected because the remaining 2.5 million terms are intentionally not embedded. Disable default features to omit the data and retain the Keplerian and JPL low-

precision providers. `Vsop2013::available()` and the corresponding Python static method make the build configuration queryable.

Evaluated dynamics

Phase 11 provides three stateless six-state models:

- `KeplerDynamics` for inertial two-body Cartesian motion;
- `Cr3bpDynamics` for the circular restricted three-body problem;
- `BcpDynamics` for the time-dependent bicircular problem.

All states are ordered `[x, y, z, vx, vy, vz]`. Kepler inputs use one consistent dimensional unit system: if position and time are SI, `mu` is in `m³/s²`. CR3BP and BCP use the upstream nondimensional rotating-frame convention. Their primaries are fixed at `(-mu, 0, 0)` and `(1 - mu, 0, 0)`. The BCP Sun is at `rho_sun [cos(omega_sun t), sin(omega_sun t), 0]`.

`mu` is the secondary-to-total mass ratio and accepts the mathematical range `[0, 1]`; the customary ordering has `mu <= 0.5`. BCP parameters are `[mu, mu_sun, rho_sun, omega_sun]`. `mu_sun` must be non-negative and `rho_sun` positive. The constants module supplies the upstream Earth–Moon–Sun defaults.

Evaluation and propagation

Each model exposes `evaluate`, `propagate`, and `propagate_with_stm`. Nominal `propagate` uses the fixed-system Taylor backend, matching upstream pykep's `ta` families; `propagate_with_method` can select DOP853 explicitly. State-transition matrices are row-major, with output state components in rows and initial state components in columns. `propagate_with_stm` retains the generic DOP853 direct-variational sensitivity contract. `propagate_with_stm_method` selects the Taylor tangent recurrence explicitly. See Taylor integration and matched native benchmarks.

For comparisons with the pinned C++ Taylor-adaptive implementation, the test profile uses relative and absolute tolerances of `2e-13` and a maximum step of `0.01` nondimensional time units. Across the committed sample grid this supports state tolerances of `3e-11` for Kepler/BCP and `2e-9` for the longer, close-approach CR3BP trajectory. These are validation tolerances, not a guarantee for arbitrary trajectories; callers must choose tolerances based on their scale, duration, and invariant drift requirements.

CR3BP also exposes the positive effective potential

```
U = (x² + y²)/2 + (1-mu)/r1 + mu/r2
```

and the Jacobi constant `C = 2 U − |v|²`.

Python boundary

Python exposes `kepler_rhs`, `cr3bp_rhs`, `bcp_rhs`, the CR3BP potential and Jacobi functions, and model-specific propagation functions. The propagation calls release the GIL and accept `initial_time`, scalar tolerances, and an optional maximum step. Variational variants return `(state, stm)` without exposing heyoka or any internal expression graph.

Exact collisions with the active model bodies are `SingularGeometryError`. Invalid mass or distance parameters are `ValueError`. A failure to finish an otherwise valid propagation is `IntegrationError`; tests keep those model and integrator failure categories separate.

Unlike upstream, no compiled-expression cache is needed: the evaluated model types are zero-sized and integration configuration is local to each call.

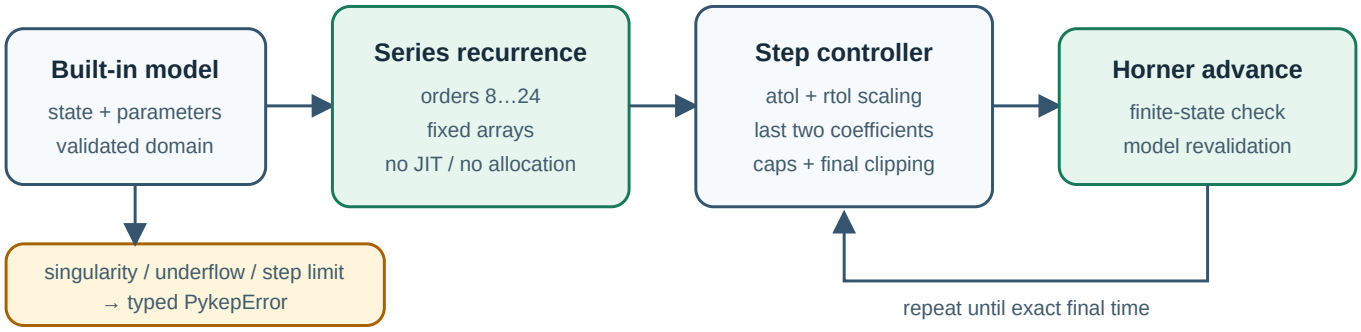
High-accuracy Taylor integration

The current benchmark report compares all eleven built-in nominal fixtures with warmed native heyoka under a shared endpoint-error gate. It gives separate results for full ZOH-leg Jacobians and the Python trajectory gym.

`pykep-core` has two pure-Rust adaptive integration backends:

- `Taylor` is the default for the eleven built-in dynamics types, matching upstream pykep's `ta` model families. It is most useful near `1e-14` and below, or when long-term invariant drift matters more than minimum low-accuracy wall time.
- `Dop853` accepts arbitrary `DynamicsModel` implementations, supports event location, and remains the general-purpose choice.

The Taylor backend specializes its coefficient recurrences for the built-in astrodynamics equations. Ordinary builds compile the checked-in Rust kernels.



What the backend computes

For a state expanded around the current time,

\[x(t+h)=\sum_{k=0}^{p}x_kh^k, \]

the model recurrence constructs coefficients through order `p`. Products use Cauchy convolution; real powers, square roots, exponentials, sine, and cosine use coefficient recurrences. The controller:

1. selects order 8–24 from the requested tolerance;
2. scales the final two coefficients by `atol + rtol * max(abs(state), 1)`;

3. chooses a conservative step from both coefficient estimates;
4. applies initial and maximum step caps;
5. clips the final step exactly to the requested time.

Backward propagation uses the same coefficients with a negative step. Every accepted state is checked for finite values and revalidated against the model domain. A zero or non-advancing floating-point step is reported as underflow.

The implementation follows the recurrence and step-selection ideas described by Jorba and Zou, while being written independently for fixed Rust arrays: A software package for the numerical integration of ODEs by means of high-order Taylor methods.

Selecting the backend

`KeplerDynamics::propagate()`, the corresponding CR3BP and BCP convenience methods, and `AdaptiveIntegrator::default()` select Taylor. The method can also be stated explicitly:

```
use pykep_core::dynamics::KeplerDynamics;
use pykep_core::integration::{IntegrationMethod, IntegratorOptions};

let result = KeplerDynamics.propagate_with_method(
    0.0,
    [0.8, -0.2, 0.1, 0.03, 1.0, 0.02],
    0.5,
    1.0,
    IntegratorOptions {
        relative_tolerance: 1e-14,
        absolute_tolerance: 1e-14,
        ..IntegratorOptions::default()
    },
    IntegrationMethod::Taylor,
)?;
assert_eq!(result.time, 0.5);
```

The lower-level `Taylor` and `AdaptiveIntegrator` facades also provide final, dense-grid, and seeded-sensitivity propagation. ZOH schedules have parallel `*_with_method` functions.

Taylor dense output samples the accepted-step polynomial and uses the same adaptive steps as final-state propagation. A ZOH control boundary still ends the segment and its polynomial. On baseline x86 builds, interior samples use separate multiply and add in Horner evaluation to avoid repeated FMA helper calls. Nonfinite results are reevaluated with fused arithmetic.

Use `Taylor::propagate_until_event` for a terminal plane, sphere, or radial stationary point on a built-in Cartesian model. `Taylor::propagate_with_events` records crossings without

modifying the trajectory and takes an explicit event limit. Crossing direction follows propagation order, including for backward integration. Root isolation works on the truncated event polynomial and reports ambiguous near-stationary roots as integration errors. Use `Dop853::propagate_until_event` for a general callback or external model.

`Taylor::propagate_batch` preserves input order and the existing worker policy. The optional `simd` feature evaluates four Kepler, CR3BP, BCP, or ZOH Kepler coefficient jets together. Each lane has its own time and step size. The vector kernel extracts the coefficient tails for step selection and evaluates the new states directly from the vector jet. Other models and partial groups use scalar propagation.

Supported models

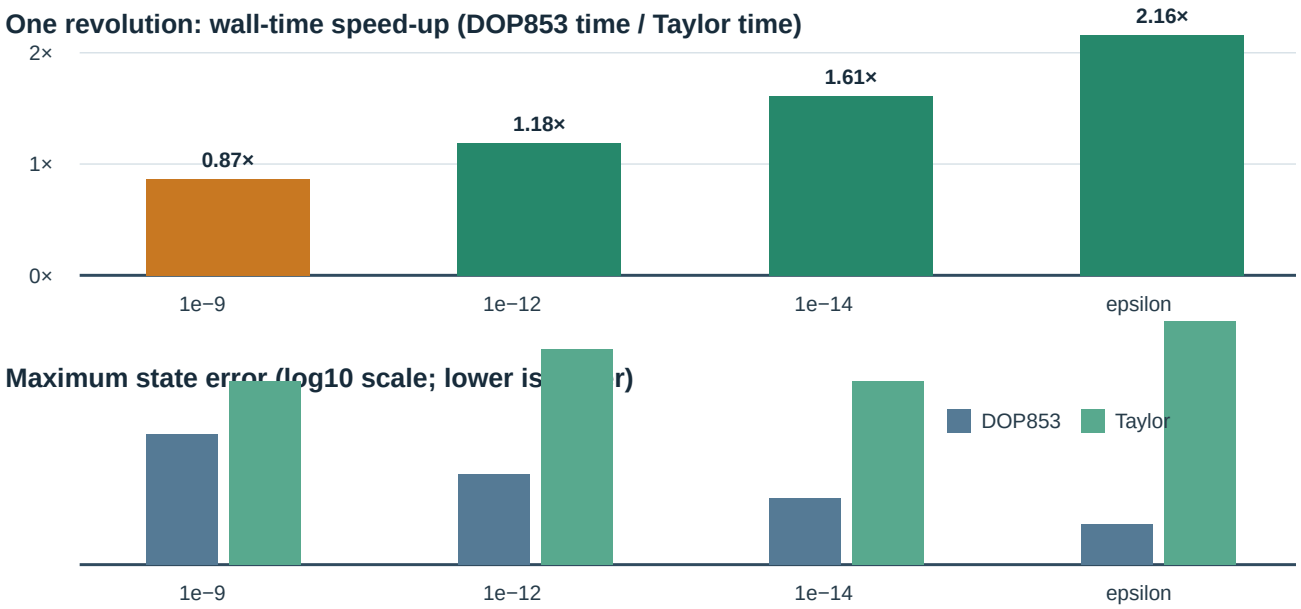
The nine upstream equation families produce eleven concrete Rust types:

Family	Rust type
Cartesian Kepler	`KeplerDynamics`
CR3BP	`Cr3bpDynamics`
Bicircular problem	`BcpDynamics`
ZOH Cartesian Kepler	`ZohKeplerDynamics`
ZOH CR3BP	`ZohCr3bpDynamics`
ZOH equinoctial	`ZohEquinoctialDynamics`
ZOH solar sail	`ZohSolarSailDynamics`
Cartesian Pontryagin	`CartesianMassOptimal`, `CartesianTimeOptimal`
Equinoctial Pontryagin	`EquinoctialMassOptimal`, `EquinoctialTimeOptimal`

Singular points stay errors. Examples include collisions, zero Pontryagin primer norm, non-positive mass, and a zero equinoctial radial denominator. The series backend does not smooth or silently cross these non-analytic model boundaries.

Measured crossover

The fixed benchmark uses the eccentric nondimensional state `[0.5, 0, 0, 0, sqrt(3), 0]`, one revolution, release mode, and analytical Lagrange propagation as the error reference.



tolerance	DOP853	Taylor	Taylor speed-up	max error, DOP853	max error, Taylor
`1e-9`	14.96 µs	17.22 µs	0.87×	`3.80e-8`	`1.08e-10`
`1e-12`	28.62 µs	24.17 µs	1.18×	`9.13e-11`	`1.65e-13`
`1e-14`	44.77 µs	27.80 µs	1.61×	`1.60e-12`	`6.84e-14`
machine epsilon	71.31 µs	33.03 µs	2.16×	`8.39e-14`	`1.25e-15`

These are measurements on one development host, not portable promises. The 100- and 1,000-revolution rows and all raw counters are in `data/taylor-kepler-benchmark.csv`. Taylor coefficient sweeps and DOP853 RHS calls are different work units, so wall time at matched accuracy—not the work-counter ratio—is the meaningful comparison.

Reproduce the data with:

```
cargo run --release -p pykep-taylor-benchmark \
  > docs/data/taylor-kepler-benchmark.csv
```

Incremental coefficient evaluators

All eleven built-in models advance only the next required Taylor coefficient. Kepler and ZOH Kepler use handwritten scalar recurrences. Solar sail uses its incremental tape for nominal

propagation. CR3BP uses generated Rust through order 18 and its tape at higher orders. The other seven models use checked-in generated Rust. Generated kernels also supply the tangent and SIMD recurrences.

The generator reuses the model graphs, prunes unreachable nodes while retaining trigonometric companions, and specializes products with factors constant in time. Such factors can still carry parameter derivatives. State histories are read from the solution jet, and local values hold intermediate coefficients that are used only at the current degree. Recurrence operands retain their coefficient histories. Run `python tools/taylor-codegen/regenerate.py --report layout.json` from the repository root to inspect the emitted row counts.

The nominal step safety factor is 0.85 for CR3BP, solar sail, and the two equinoctial Pontryagin models. The other nominal paths and the augmented sensitivity controller use 0.75. Accuracy and performance for the expanded fixture corpus are documented in the Taylor performance follow-up.

The following timings describe the earlier migration to incremental recurrences.

The table records the eight final migrations against the previous repeated full-series evaluator at commit `c9ecd17`. Each row is an end-to-end propagation at `1e-12`, pinned to one core, with 1,000 iterations per sample and the median of seven warmed samples:

model	previous	incremental	speed-up
CR3BP	158.102 µs	21.515 µs	7.35×
bicircular problem	104.921 µs	16.257 µs	6.45×
ZOH CR3BP	51.266 µs	6.791 µs	7.55×
ZOH equinoctial	49.683 µs	6.979 µs	7.12×
ZOH solar sail	91.894 µs	12.876 µs	7.14×
Cartesian time-optimal	124.124 µs	4.285 µs	28.97×
equinoctial mass-optimal	7.696 ms	377.991 µs	20.36×
equinoctial time-optimal	7.797 ms	333.640 µs	23.37×

These development-host measurements establish that every migration cleared the significance gate; they are not portable latency guarantees. Matching final-state checksums and the validation layers below guard the optimization. Criterion entries under `taylor/*` keep all eight paths visible in routine performance runs.

Sensitivities and validation boundaries

`Taylor::propagate_with_sensitivities()` computes the continuous first variations from tangent coefficients. Up to 16 seed directions share one nominal coefficient evaluation in a dual jet. Wider seed matrices use four-direction blocks to bound kernel scratch space. Step selection includes the nominal state and every sensitivity component. Baseline x86 builds use separate multiply and add to evaluate derivative polynomials, with fused reevaluation for nonfinite results. `ZohLeg::mismatch_jacobian_with_method(IntegrationMethod::Taylor)` composes the segment sensitivities into endpoint, control, and time-grid derivatives.

For Pontryagin systems, stop-gradient boundaries are applied when forming the Hamiltonian costate equations. Tangent propagation differentiates the resulting closed-loop equations, including the control dependence. Taylor sensitivities use fixed end times. Event-time and reset derivatives are outside this API.

Validation has three independent layers:

- closed-form tests for series products, real powers, trigonometric functions, exponentials, and square roots;
- fixed DOP853 comparisons for all eleven concrete dynamics types, including all four Pontryagin variants;
- a development-only official-heyoka harness for Kepler, CR3BP, BCP, and a Kepler STM.

The heyoka fixture records version `7.10.1` and is regenerated with `tools/heyoka-cross-validation/generate.py`. It is intentionally not a normal CI dependency. The larger ZOH and Pontryagin families use the existing pinned upstream pykep fixtures plus DOP853 comparisons; they are not presented as direct heyoka cross-checks.

Regenerating the kernels

Run the generator from a workspace with the supported Rust toolchain:

```
python tools/taylor-codegen/regenerate.py
python tools/taylor-codegen/regenerate.py --check
```

The ignored Rust exporter serializes private model graphs, and the developer tool emits and formats the Rust module. An ordinary crate or wheel build uses the checked-in source. The tape and full-series tests provide reference evaluators. See the comparison instructions for reproducible native and Python timings.

Population, history, and shooting workflows

For a BCP population, create one `InitialValueProblem<6, 4>` per candidate. Keep the rotating-frame Cartesian state order `[x, y, z, vx, vy, vz]` and the BCP parameter order `[mu, mu_sun, rho_sun, omega_sun]`. Each candidate can have its own time interval. With a pool already supplied by the optimizer, use one inner worker.

```
use pykep_core::dynamics::BcpDynamics;
use pykep_core::integration::{InitialValueProblem, IntegratorOptions, Taylor};

let jobs = [0.12, 0.20, 0.25, 0.30].map(|time| {
    InitialValueProblem::new(
        0.0, [0.8, -0.2, 0.1, 0.03, -0.04, 0.02], time,
        [0.01215058560962404, 328900.56, 389.172, -0.925195985520347],
    )
});
let states = Taylor.propagate_batch(&BcpDynamics, &jobs,
IntegratorOptions::default(), 1)?;
```

Use the same rotating-frame scales for every position, velocity, and time in a batch. Validate several population endpoints with a tighter scalar solve. The `model_comparison` binary includes balanced and mixed-duration batches using this fixture.

For a normalized low-thrust Cartesian leg, request `leg.state_history_with_method(101, IntegrationMethod::Taylor)`. The returned forward and backward histories each contain one list per propagated segment. Both endpoints belong to each segment's sampled list. Positions, velocities, mass, and time use the scales declared when constructing the leg. Compare interior points against independent endpoint solves with the same control over the corresponding subinterval.

For single shooting, construct a `SensitivityProblem` with one column per decision variable. An initial-velocity seed puts a one in that component of `initial_sensitivities`. A gravity-parameter seed goes into `parameter_seeds`. Use the propagated state to form the endpoint residual and select the same state rows from the returned sensitivity matrix for the Jacobian. For a ZOH leg, the complete mismatch Jacobian can be requested directly with `mismatch_jacobian_with_method(IntegrationMethod::Taylor)`. Check the assembled Jacobian against a small perturbation of the complete problem, particularly when a variable moves a shared control boundary.

Taylor performance follow-up

This report records the preceding Taylor implementation rounds. For the subsequent native PyKEP 3.1.0 and trajectory-gym measurements, see the matched benchmark report.

The follow-up reduces generated coefficient storage, evaluates four-lane Taylor polynomials without copying their histories into scalar workspaces, and shares nominal coefficients across seeded sensitivity directions.

Measurements use Rust 1.88.0 on an Intel Xeon Platinum 8573C, affinity to CPUs 2–5, and the repository's release profile. B0 is GitHub main at `542158b575ad9d235841d501772c179177d69660`. B1 adds the first Taylor patch. The final source adds the accepted follow-up changes. Paired results use three ABBA rounds and seven warmed samples per pass. The ratios are medians of paired round ratios. Host scheduling affects the spread of short timings.

Scalar propagation at requested tolerance 1e−12

The endpoint metric is `max(abs(actual - reference) / max(1, abs(reference)))`. All rows meet the `2e-11` fixed-tolerance gate against the tight DOP853 reference. B0 comparisons use the default crate features for both sides. B1 comparisons enable `simd` in both builds and call the scalar API here.

Portable build

Model	B1 µs	Final µs	B1 speedup	B0 speedup	Scaled endpoint error
kepler	3.740	4.285	0.91×	0.99×	6.66e-16
cr3bp	18.285	11.514	1.62×	1.70×	8.60e-15
bcp	7.698	5.904	1.34×	1.69×	6.94e-16
zoh_kepler	4.637	4.805	0.95×	1.10×	2.83e-15
zoh_cr3bp	10.424	9.750	1.07×	1.68×	3.03e-15
zoh_equinoctial	13.530	14.026	0.96×	1.07×	3.03e-15
solar_sail	25.385	19.759	1.36×	1.33×	7.11e-15
cartesian_mass	9.500	8.441	1.13×	1.87×	5.55e-16

Model	B1 µs	Final µs	B1 speedup	B0 speedup	Scaled endpoint error
cartesian_time	8.120	6.277	1.29×	1.64×	2.31e-15
equinoctial_mass	583.367	446.650	1.22×	2.04×	9.21e-14
equinoctial_time	516.446	437.909	1.15×	1.94×	9.15e-14

Comparison	Geometric mean	Median	Slowest ratio
B1 to final	1.16×	1.15×	0.91× (kepler)
B0 to final	1.51×	1.68×	0.99× (kepler)

Native build

Model	B1 µs	Final µs	B1 speedup	B0 speedup	Scaled endpoint error
kepler	3.086	3.227	0.95×	1.14×	6.66e-16
cr3bp	19.069	10.545	2.14×	2.21×	8.60e-15
bcp	8.642	5.816	1.49×	1.41×	6.94e-16
zoh_kepler	3.735	3.705	1.01×	0.94×	2.83e-15
zoh_cr3bp	10.062	8.400	1.16×	1.63×	3.03e-15
zoh_equinoctial	14.660	14.217	1.03×	0.99×	3.03e-15
solar_sail	25.567	19.716	1.23×	1.37×	7.11e-15
cartesian_mass	7.883	8.407	0.91×	1.82×	5.55e-16
cartesian_time	7.691	6.437	1.24×	1.33×	2.31e-15
equinoctial_mass	582.622	498.064	1.20×	1.98×	9.21e-14
equinoctial_time	555.152	448.806	1.18×	1.91×	9.15e-14

Comparison	Geometric mean	Median	Slowest ratio
B1 to final	1.20×	1.18×	0.91× (cartesian_mass)
B0 to final	1.47×	1.41×	0.94× (zoh_kepler)

Complete workload comparisons

Grid errors use analytical Kepler values. BCP batch deltas compare against scalar calls. Jacobian errors cover endpoint, control, and time-grid columns. Both builds enable SIMD.

Portable application workloads

Workload	B1 µs	Final µs	Speedup	Maximum error or delta
kepler_grid	366.18	69.75	5.04×	1.32e-15
bcp_population_balanced_1	3448.23	1741.56	2.06×	1.11e-16
bcp_population_mixed_1	3835.83	1914.77	2.08×	1.25e-16
bcp_population_balanced_4	1508.66	849.20	1.89×	1.11e-16
bcp_population_mixed_4	1651.59	825.40	1.96×	1.25e-16
kepler_stm	49.59	20.41	2.42×	2.55e-15
zoh_jacobian_20	414.14	219.98	1.84×	4.04e-11
zoh_jacobian_32	705.77	342.06	2.12×	3.34e-11

Native application workloads

Workload	B1 µs	Final µs	Speedup	Maximum error or delta
kepler_grid	49.91	42.31	1.06×	1.32e-15
bcp_population_balanced_1	2781.32	1364.36	1.91×	0.00e+00
bcp_population_mixed_1	2863.75	1615.07	1.74×	0.00e+00
bcp_population_balanced_4	1055.61	634.36	1.63×	0.00e+00
bcp_population_mixed_4	1692.33	700.40	2.25×	0.00e+00
kepler_stm	52.18	17.94	2.86×	2.55e-15
zoh_jacobian_20	434.88	229.00	1.99×	4.04e-11
zoh_jacobian_32	657.29	342.88	1.86×	3.34e-11

Small scalar confirmations

The focused runs use the same sources with four times as many calls per sample. The native Kepler case remains slower than B1. The default-feature comparison against B0 still shows a gain.

Build	Model	B1 µs	Final µs	B1 speedup
portable	kepler	3.913	4.020	0.98×

Build	Model	B1 µs	Final µs	B1 speedup
portable	zoh_kepler	5.216	5.064	1.02×
portable	zoh_equinoctial	13.872	12.780	1.09×
native	kepler	3.041	3.219	0.92×
native	cartesian_mass	8.704	8.664	1.00×

Short-arc matched-error sweep

Each configuration meets scaled endpoint error `1e-11` on its fixture. These timings use the fastest tested tolerance from one seven-sample pass per configuration. The paired fixed-tolerance comparisons above give a better view of timing variation.

Model	B1 tolerance	Final tolerance	B1 µs	Final µs	Final error
kepler	1e-09	1e-09	3.896	3.491	3.07e-12
cr3bp	1e-09	1e-12	14.633	11.602	8.60e-15
bcp	1e-09	1e-09	7.367	5.305	1.88e-12
zoh_kepler	1e-09	1e-09	3.485	3.563	3.28e-12
zoh_cr3bp	1e-09	1e-12	8.302	10.561	3.03e-15
zoh_equinoctial	1e-09	1e-09	11.103	10.872	2.09e-13
solar_sail	1e-09	1e-12	13.742	19.673	7.11e-15
cartesian_mass	1e-09	1e-09	5.977	8.449	5.55e-16
cartesian_time	1e-09	1e-09	6.108	5.177	2.31e-15
equinoctial_mass	1e-09	1e-12	374.187	590.888	9.21e-14
equinoctial_time	1e-09	1e-12	314.934	420.812	9.15e-14

CPU build comparison

Both columns use the final source and the `simd` feature. The native column uses `-C target-cpu=native`.

Workload	Portable µs	Native µs	Portable / native
kepler_grid	80.95	45.04	1.94×
bcp_population_balanced_1	1814.81	1380.00	1.32×
bcp_population_mixed_1	1953.42	1385.15	1.42×
kepler_stm	21.51	17.13	1.29×
zoh_jacobian_20	241.96	206.80	1.08×

Portable mixed sensitivity seed widths

Workload	B1 µs	Final µs	Speedup	Final error
kepler_stm_w1	22.05	20.43	1.42×	1.67e-15
kepler_stm_w14	109.68	41.52	2.65×	2.39e-15
kepler_stm_w21	173.19	111.57	1.58×	2.39e-15
kepler_stm_w4	33.15	17.57	1.78×	2.16e-15
kepler_stm_w6	70.84	22.53	3.14×	2.16e-15
kepler_stm_w7	55.22	30.28	1.89×	2.16e-15

Cumulative portable grid and STM

Workload	B0 µs	Final µs	Speedup	Final error
kepler_grid	1061.59	74.24	14.62×	1.32e-15
kepler_stm	58.89	21.25	2.72×	2.55e-15

Cumulative native grid and STM

Workload	B0 µs	Final µs	Speedup	Final error
kepler_grid	787.78	42.44	18.89×	1.32e-15

Workload	B0 µs	Final µs	Speedup	Final error
kepler_stm	44.22	17.79	2.49×	2.55e-15

Complete Python ZOH history calls

Workload	B1 µs	Final µs	Speedup	Final error
zoh_history_10	16.24	8.97	1.81×	6.11e-16
zoh_history_1001	1490.78	761.22	1.96×	7.22e-16
zoh_history_101	130.69	64.76	2.22×	6.66e-16
zoh_history_2	5.33	4.55	1.17×	6.11e-16

fcmaes objective replay

The same 1,024 candidates were evaluated in the same order. Times include construction, propagation, and scoring.

Objective	Workers	B1 µs	Final µs	Speedup	Maximum score/residual delta
bcp	1	4255.43	2573.56	1.65×	1.11e-16
zoh_kepler	1	33630.62	29975.51	1.12×	0.00e+00
bcp	4	1784.45	1038.34	1.72×	1.11e-16
zoh_kepler	4	9870.46	9796.29	1.01×	0.00e+00

Each optimizer cell uses seeds 11, 19, and 42 and 4,096 evaluations per run. These short campaigns compare wall time and final objectives. Time-to-target records are in the raw data.

Objective	Workers	Median B1 ms	Median final ms	Largest final objective delta	Target hits B1 / final
bcp	1	21.71	13.37	3.12e-19	3 / 3
zoh_kepler	1	110.62	114.27	0.00e+00	0 / 0
bcp	4	42.26	19.45	3.12e-19	3 / 3
zoh_kepler	4	68.35	68.34	0.00e+00	0 / 0

Python calls at matched endpoint accuracy

PyKEP 3.0.1 with heyoka.py 7.10.1 reuses the compiled integrator. Both sides include Python calls. Rows select the fastest tested tolerance at scaled endpoint error `1e-11`.

Model	PyKEP µs	Rust Taylor µs	Rust / PyKEP
kepler	4.023	3.473	0.86×
cr3bp	4.866	9.682	1.99×
bcp	4.564	5.469	1.20×
zoh_kepler	5.430	5.446	1.00×
zoh_cr3bp	5.681	7.913	1.39×
zoh_equinoctial	8.694	12.387	1.42×
solar_sail	7.403	22.481	3.04×
cartesian_mass	4.653	8.618	1.85×
cartesian_time	4.135	6.129	1.48×
equinoctial_mass	622.882	508.383	0.82×
equinoctial_time	554.198	413.126	0.75×

See the comparison guide for reproduction commands. The raw measurements are in the measurement archive attached to the 0.1.5 release. Comparison guide

Zero-order-hold dynamics

Phase 12 provides a common `ControlSchedule<C>` and four evaluated models:

- `ZohKeplerDynamics`: `[x,y,z,vx,vy,vz,mass]`, normalized `mu = 1`;
- `ZohCr3bpDynamics`: the same seven-state layout in the CR3BP synodic frame;
- `ZohEquinoctialDynamics`: `[p,f,g,h,k,L,mass]`, normalized `mu = 1`;
- `ZohSolarSailDynamics`: six Cartesian states with cone/clock attitude.

The first three control rows are `[thrust, i1, i2, i3]`. The direction is Cartesian for Kepler/CR3BP and radial-transverse-normal for equinoctial dynamics. Kepler/equinoctial constants contain mass-flow coefficient `c`; CR3BP constants are `[c, mu]`. The preserved upstream mass equation is `dm/dt = -c thrust exp(-1 / (mass 1e16))`.

Solar-sail rows are `[alpha, beta]` in radians. Its constant `c` scales ideal sail acceleration as `c cos(alpha)^2 / r²`.

Schedule and switch contract

A schedule has `S + 1` strictly increasing finite boundaries and exactly `S` finite control rows. Construction performs all dimension, finiteness, and monotonicity checks.

For lookup, segment `i` owns `[t_i, t_(i+1))`; the final boundary belongs to the final segment. Forward propagation ends each integration exactly at a switch and starts the next segment with its new control. Backward propagation uses the interval to the left of each encountered switch. This directional rule avoids evaluating a segment across a discontinuity.

Each segment is integrated exactly once. The active control is copied into a fixed-size parameter array before the solve, so no allocation or schedule scan occurs in an RHS call. `control_at` uses binary search for occasional external lookup.

The established `propagate_schedule*` functions continue to use DOP853. The corresponding `*_with_method` functions accept `IntegrationMethod::Taylor` for all four built-in models. Each segment still ends exactly at its switching boundary; backend selection does not change switch ownership.

Sensitivities

`ZohSensitivitySeeds` carries an arbitrary compile-time seed width through the schedule in one augmented solve per segment. It contains:

- initial state seeds;
- one control seed matrix per segment;
- constant-parameter seeds.

State sensitivities are continuous at switches. A control column for a future segment remains exactly zero until that segment becomes active. Runtime is linear in the segment count for a fixed seed width; the DOP853 implementation does not repropagate every prior segment for every control.

The Taylor backend propagates the tangent coefficients directly and shares nominal coefficients across practical seed widths. Complete Jacobians and mismatch timings are reported separately in the matched benchmarks.

For DOP853, the Kepler and CR3BP models supply analytic state and parameter Jacobians. Since 0.1.5 they replace the earlier fixed-step central differences, which lost accuracy near a primary: on one backward segment of a moving CR3BP TOPS vector with a close lunar pass, the old DOP853 transition matrix differed from Taylor and from differences of the nominal flow by 43%. The equinoctial and solar-sail models still use fixed-size, allocation-free central differentiation of the evaluated source equations. All four are checked against C++ Taylor variations and end-to-end schedule finite differences. The longest CR3BP reference uses a `2e-5` relative/absolute variation comparison; nominal state parity remains `3e-10`.

Python API

Python exposes all four RHS functions and `propagate_zoh_*` functions. Boundaries and controls are ordinary nested sequences. Propagation releases the GIL and accepts `backward`, scalar tolerances, and `maximum_step`. Malformed row widths and grids are rejected before native integration.

Pontryagin low-thrust dynamics

The matched benchmark report includes native mass- and time-optimal scalar fixtures in both coordinate systems.

Phase 13 provides evaluated Cartesian and modified-equinoctial canonical dynamics for indirect low-thrust optimization. The implementation is native Rust and uses the common integration and sensitivity contracts; it does not expose or depend on a symbolic-expression runtime. The four built-in types can be passed to the default Taylor `AdaptiveIntegrator`, to `Taylor` directly, or to `Dop853` when events or direct variational propagation are needed.

Orders and units

The Cartesian augmented state is

```
[x, y, z, vx, vy, vz, m, lx, ly, lz, lvx, lvy, lvz, lm]
```

The modified-equinoctial augmented state is

```
[p, f, g, h, k, L, m, lp, lf, lg, lh, lk, lL, lm]
```

The equinoctial convention is prograde and thrust direction is expressed in the radial, transverse, normal (RTN) frame. Cartesian thrust direction is inertial. `p`, position, velocity, mass, time, `mu`, thrust, and exhaust velocity must form one consistent unit system. Angles are radians. Costate units follow from the caller's chosen normalization.

Mass-optimal models use the parameter order

```
[mu, maximum_thrust, exhaust_velocity, barrier, lambda0]
```

`barrier` is the positive logarithmic regularization coefficient. Time-optimal models use

```
[mu, maximum_thrust, exhaust_velocity]
```

and have full throttle with the upstream implicit `lambda0 = 1`. Maximum thrust may be zero, which is useful for canonical coordinate checks; all other physical scale parameters and `lambda0` must be positive.

Control and Hamiltonian

The public control evaluators return `OptimalControl`, containing throttle, direction, and the switching function `rho`. Mass optimality uses the regularized upstream control

```
u = 2 eps / (rho + 2 eps + sqrt(rho² + 4 eps²))
```

evaluated with an algebraically equivalent branch that avoids cancellation. Time optimality returns `u = 1`. The Hamiltonian functions evaluate the minimized autonomous Hamiltonian using that control.

The minimizing direction is undefined when the relevant primer norm is exactly zero. Rust reports `PykepError::SingularGeometry`, and Python reports `SingularGeometryError`; no arbitrary direction or NaN vector is returned. Cartesian position at the origin, non-positive mass or equinoctial semilatus rectum, and a zero equinoctial radial denominator are also explicit errors.

Models and sensitivities

The four zero-sized model types are:

- `CartesianMassOptimal`
- `CartesianTimeOptimal`
- `EquinoctialMassOptimal`
- `EquinoctialTimeOptimal`

Each implements `DynamicsModel<14, P>` and `DifferentiableDynamicsModel<14, P>`. Canonical costate rates use forward-mode derivatives of the minimized Hamiltonian. Full state and parameter Jacobians use centered fixed-size differences and are stored as `jacobian[output][input]`. This path performs no heap allocation in the model right-hand side.

In the Taylor recurrence the minimizing control evolves with the complete state/costate series. When forming canonical costate rates, the control is held fixed with respect to the physical-state derivative, as required by the envelope theorem. Zero primer norm remains an explicit non-analytic error.

Python requires the native `Optimality.Mass` or `Optimality.Time` enum. Unvalidated strings are not accepted. The `pontryagin_*_rhs`, control, Hamiltonian, and propagation functions use the same `pykep-core` implementation.

Validation

The committed C++ oracle covers both coordinates and both objectives, their eight upstream variational arguments (seven initial costates and `lambda0` in mass mode), and the upstream dimensional 100-day trajectory. Independent checks cover Hamiltonian conservation, propagated central differences, Jacobian orientation, zero-primer errors, normalized controls, and canonical Hamiltonian agreement after transforming costates between coordinates with the analytic element Jacobian.

At `2e-12` DOP853 scalar tolerances and a `0.01` maximum step for normalized cases, nominal trajectories agree with the pinned C++/heyoka oracle within a scaled `3e-10`. The dimensional case uses a 12-hour maximum step and a scaled `3e-9`. First-order variations use a scaled `2e-4` tolerance because the generic Jacobian contract is numerical.

Sims–Flanagan low-thrust legs

The matched benchmarks cover direct leg calls, complete optimizer-facing gradients and PyGMO SLSQP convergence.

`pykep-core::leg` provides the fixed-duration `SimsFlanaganLeg` and the variable-duration `SimsFlanaganAlphaLeg`. Both are immutable after validated construction, use native Rust two-body propagation, and have no C or C++ runtime dependency.

Units and ordering

All inputs must use one self-consistent unit system:

- endpoint state is `[x, y, z, vx, vy, vz]`;
- endpoint mass and maximum thrust use compatible mass and force units;
- time of flight, segment durations, and exhaust velocity use compatible time units;
- `mu` has length cubed per time squared;
- throttle vectors are dimensionless and appear in chronological order.

An endpoint is represented by `SpacecraftEndpoint`, which rejects non-finite state components, zero radius, and non-positive mass. `SimsFlanaganSettings` rejects negative time of flight or maximum thrust, non-positive exhaust velocity or `mu`, non-finite values, and cuts outside `[0, 1]`. At least one three-component throttle is required.

Transcription and cut

For a leg with `N` segments, the number propagated forward from departure is

```
floor(N * cut)
```

and the remaining segments are propagated backward from arrival. A cut of zero therefore starts at the departure endpoint and propagates every segment backward; a cut of one propagates every segment forward.

Each segment applies its finite impulse at the segment midpoint. The fixed-duration leg uses duration `time_of_flight / N` for every segment. The velocity increment has magnitude `maximum_thrust * duration * |throttle| / mass`; the mass update follows the Tsiolkovsky

exponential using `exhaust_velocity`. Backward propagation reverses both the impulse and mass update.

The alpha leg accepts direct non-negative segment durations. As in the upstream class, their sum is not required to equal `time_of_flight`. `SimsFlanaganAlphaLeg::from_time_weights` is the explicit alternative that requires positive weights and normalizes them to the configured time of flight.

Constraints and derivatives

`mismatch_constraints()` returns seven values in this order:

```
[forward_rx - backward_rx,
 forward_ry - backward_ry,
 forward_rz - backward_rz,
 forward_vx - backward_vx,
 forward_vy - backward_vy,
 forward_vz - backward_vz,
 forward_mass - backward_mass]
```

`throttle_constraints()` returns `N` values, `dot(throttle_i, throttle_i) - 1`. A zero value is the unit-throttle limit; negative values are inside it.

The fixed leg exposes an analytic `mismatch_jacobian()` with output-by-input rows:

Group	Shape	Column order
departure	`7 × 7`	`[x,y,z,vx,vy,vz,mass]`
arrival	`7 × 7`	`[x,y,z,vx,vy,vz,mass]`
controls and time	`7 × (3N + 1)`	`[u0x,u0y,u0z,...,u(N-1)z,time_of_flight]`

`throttle_jacobian()` has shape `N × 3N` in the same flattened control order. At zero throttle the mass-direction derivative uses the defined zero subgradient, so the returned matrix remains finite.

The upstream alpha class does not expose an analytic mismatch gradient, and neither does this port. Its exact throttle Jacobian remains available.

Python

`pykep_rust.SimsFlanaganLeg` and `pykep_rust.SimsFlanaganAlphaLeg` expose the same immutable evaluations. Python states and controls are converted once at construction; all astrodynamics calculations call `pykep-core`. The fixed-leg `mismatch_jacobian()` returns a tuple of the three row matrices described above.

Construction and evaluation raise `ValueError` for invalid values or shapes and `RuntimeError` for propagation failures.

Generic zero-order-hold leg

`pykep_core::leg::ZohLeg` transcribes a transfer between two fixed endpoint states as non-uniform segments with continuous, piecewise-constant controls. It is generic over any Rust model implementing `ZeroOrderHoldModel`; aliases cover every built-in ZOH dynamics family:

Alias	State	Control	Constants
`ZohKeplerLeg`	`[x,y,z,vx,vy,vz,m]`	`[thrust,ix,iy,iz]`	`[c]`
`ZohCr3bpLeg`	`[x,y,z,vx,vy,vz,m]`	`[thrust,ix,iy,iz]`	`[c,mu]`
`ZohEquinoctialLeg`	`[p,f,g,h,k,L,m]`	`[thrust,ir,it,in]`	`[c]`
`ZohSolarSailLeg`	`[x,y,z,vx,vy,vz]`	`[cone,clock]`	`[c]`

The units and frame conventions of each row are those of its model in zero-order-hold.md. `c` is the normalized mass-flow coefficient for the three low-thrust models and the normalized lightness coefficient for the solar sail.

Grid, controls, and cut

For `S` segments, construction requires exactly `S + 1` finite, strictly-increasing time-grid nodes and `S` finite control vectors. Controls are stored in chronological order and own the half-open interval `[time_grid[i], time_grid[i+1])`; the final endpoint belongs to the final segment.

The cut uses the same convention as the upstream leg:

```
forward_segments = floor(S * cut)
backward_segments = S - forward_segments
```

The forward state starts at the initial endpoint and the backward state starts at the final endpoint. The mismatch is their component-wise difference at the cut. Cuts zero and one are supported without special placeholder states.

The leg is immutable after construction. It rejects invalid endpoint states, model constants, dimensions, grids, cuts, tolerances, maximum steps, and maximum-step magnitudes before evaluation.

`mismatch_constraints()` preserves the established DOP853 propagation. Built-in models additionally provide `mismatch_constraints_with_method(IntegrationMethod)`, so repeated derivative-free objective evaluations can select the accelerated Taylor backend explicitly.

Sensitivity layout

`mismatch_jacobian()` returns four output-by-input matrices:

Group	Shape	Column order
initial state	`N × N`	model state order
final state	`N × N`	model state order
controls	`N × (C*S)`	segment 0 controls, then segment 1, and so on
time grid	`N × (S+1)`	every grid node, including both endpoints

Each segment propagates a local state-transition and active-control matrix. The leg composes those matrices from each side of the cut and includes the dynamics jump at every switching time. Constant model parameters are fixed leg configuration and therefore are not a returned derivative group.

Use `mismatch_jacobian_with_method(IntegrationMethod::Taylor)` for direct Taylor tangent propagation on the built-in models. The DOP853 path uses analytic local RHS Jacobians for the Kepler and CR3BP models. For the equinoctial and solar-sail models it uses fixed-size centered differences with a relative step of `3e-6`; there, integrator tolerances such as `1e-12` do not imply `1e-12` derivative accuracy, and the pinned end-to-end validation uses scaled tolerances up to `3e-5`. See zero-order-hold.md for the model-level contract and validation evidence. The matched report covers the measured full Jacobians and nominal mismatch calls at 20, 40 and 80 segments.

Integration and failures

`IntegratorOptions` applies independently to every segment, including relative and absolute tolerances, optional initial and maximum step sizes, maximum steps, and maximum rejections. A propagation failure is reported as an `IntegrationFailure` containing the direction, chronological segment index, and attempted time interval.

`state_history(samples_per_segment)` uses one DOP853 dense solve per segment for uniformly spaced states including both endpoints.
`state_history_with_method(samples_per_segment, IntegrationMethod)` selects DOP853 or Taylor for built-in models. DOP853 backward segments are evaluated through the equivalent increasing coordinate `tau = segment_start - time` to avoid a decreasing-time dense-output boundary issue in the numerical backend; Taylor supports the decreasing grid directly. At least two samples are required. Backward histories list the final segment first, matching propagation order. `evaluate_zoh_mismatch_batch()` evaluates validated Rust legs in input order.

Python

`pykep_rust.ZohLeg` accepts a `ZohModel` value: `Kepler`, `Cr3bp`, `Equinoctial`, or `SolarSail`. The constructor validates dynamic state, control, and constant dimensions for that selection. Mismatch, Jacobian, and history evaluation release the Python GIL. Pass `method="taylor"` to any of them, or to their batch forms, to use the Taylor path.

The ZOH-leg batch methods clone the small immutable leg descriptors, release the GIL once, and return results in input order. `workers=0` uses the shared pool, `workers=1` is serial, and a larger value selects a cached pool of that exact size.

Trajectory optimization gym

Since 0.1.5 the `pykep-rust` wheels include the current PyKEP mission definitions and TOPS nonlinear programs from the Rust crate `pykep-trajopt`. Constructing a problem reads embedded data and creates native Rust state. The benchmark tools supply optional optimizers.

`pykep-trajopt` is not published on crates.io; Rust users depend on it through the repository workspace or a path. Maturin uses the feature list in `pyproject.toml` by default. With an explicit Maturin feature override, include `trajopt` alongside any other requested features.

Catalog	Entries
Classical trajectories	Cassini1 in three encodings, Cassini2, three EVE variants, Rosetta, Messenger, Earth–Mars with 3/5/7 impulses, JUICE and multiobjective JUICE
TOPS fixed boundaries	5 two-body Cartesian, 9 MEE, 14 CR3BP, 6 solar sail
TOPS moving boundaries	The same family-qualified case keys with a moving endpoint formulation
Launcher tables	Ariane 5, Ariane 64, Atlas 501, Atlas 551, Soyuz-Fregat, Falcon 9

The stable catalog has 82 instance IDs. Each TOPS ID also accepts segment, cut, time encoding, constraint and integration options, including `maximum_steps`. These settings belong to the problem specification and are recorded with experiment results. Classical `gym.*` IDs have fixed definitions and ignore the TOPS options.

Fixed-boundary TOPS problems default to the upstream limit of 1,000 integration steps per propagation. The upstream moving-boundary factories impose no limit, so moving problems default to 1,000,000.

```
cargo run --release --locked -p pykep-gym-benchmark -- list
cargo run --release --locked -p pykep-gym-benchmark -- evaluate --problem
gym.eve_mga1dsm
cargo run --release --locked -p pykep-gym-benchmark -- replay --problem
tops.twobody.P4 --nseg 16 --count 64
```

```
from pykep_rust.trajopt import gym

problem = gym.tops_twobody("P4", nseg=16, time_encoding="softmax")
lower, upper = problem.get_bounds()
x = [(a + b) * 0.5 for a, b in zip(lower, upper)]
result = problem.evaluate(x)
jacobian = problem.gradient(x)
```

`fitness(x)` returns minimized objectives, equalities, then inequalities. Raw equalities target zero. Raw inequalities are feasible at nonpositive values. The full gradient is flattened in row-major order for the PyGMO protocol. `fitness_batch` takes a two-dimensional float64 NumPy array and returns the same rows in input order. Classical mission functions expose `has_gradient() == False`.

A decision inside the bounds can still be numerically unusable, for example a TOPS schedule that exhausts the propellant before the cut. Such calls raise the mapped integration or numerical exception rather than returning a penalty, and `fitness_batch` raises for the first failing row in input order. Wrap the call when an optimizer needs a penalty value; the native `tools/gym-benchmark` adapters map failures to a large merit.

The Rust API also exposes configurable `ImpulsiveProblem` constructors, `TopsProblem`, `ProblemSpec`, `Launcher` and the `Problem` trait. The optimizer adapters live in `tools/gym-benchmark`. Native execution can use DE, DE followed by CMA-ES, or MODE. Frozen coordinates are removed inside the solver and restored exactly for evaluation. Replayed results retain raw physical objectives and residuals.

```
cargo run --release --locked -p pykep-gym-benchmark -- optimize \
  --problem gym.eve_mga1dsm --optimizer de-cma --evaluations 4096 \
  --seed 42 --workers 1 --output artifacts/gym/eve
cargo run --release --locked -p pykep-gym-benchmark -- validate \
  --run artifacts/gym/eve/run.json
```

`validate` restores the recorded settings and backend, replays the best decision with a tighter nominal tolerance and exits with an error if the objectives differ from the record by more than `1e-6`, a recorded feasible point becomes infeasible, or a TOPS forward residual exceeds `1e-6`.

The run directory contains a schema-v1 manifest, the complete problem specification, convergence data and a decoded trajectory when available. The optional wall deadline is checked between populations. Actual budget and elapsed time are recorded. PyGMO can wrap the Python problem directly when installed. The native library has no PyGMO dependency.

See the TOPS walkthrough, impulsive formulations and benchmark results.

TOPS low-thrust problems

TOPS transcribes a trajectory into a chronological sequence of held controls. The leg integrates from both ends and returns the difference at the selected cut. The source JSON stores physical scales and reference states. The Rust catalog retains the source units, then applies the normalization of each pinned PyKEP constructor. The problem metadata records both scale systems.

Family	Control per segment	State	Default fixed constraints and objective
Two-body Cartesian	Throttle fraction and three direction components	Position, velocity, mass	Seven mismatch equalities, direction inequalities, minimize negative final mass
MEE	Throttle fraction and three direction components	Prograde modified equinoctial elements, mass	Seven mismatch equalities with unwrapped longitude, direction inequalities, minimize negative final mass
CR3BP	Throttle fraction and three direction components	Rotating position, velocity, mass	Seven mismatch equalities and direction equalities, minimize negative final mass
Solar sail	Sail cone and clock angles	Position and velocity	Six mismatch equalities, minimize flight time

Direction constraints use the squared norm minus one. The evaluator keeps the decision components as supplied. The solar-sail catalog's six fixed-boundary entries all fix flight time, so their default optimization task is feasibility.

Uniform grids divide total duration equally. Softmax grids use one logit per segment and positive weights that sum to one. The Jacobian contracts the leg's state, control and grid derivatives with the decoding maps. The softmax contraction uses a reverse cumulative sum for each residual row.

Moving problems add the departure offset, endpoint relative-velocity magnitudes and their direction components. The endpoint states follow Keplerian or CR3BP flow. The normalized epoch chain rule includes both the departure shift and the arrival shift due to flight time. MEE additionally applies the coordinate Jacobian and preserves the source revolution offset. The moving CR3BP source factory uses uniform time even when it receives `softmax`. The Rust softmax formulation carries the version `tops-v1-cr3bp-moving-softmax`.

The moving MEE Jacobian follows the selected longitude winding branch. At the P5 and P8 fixed-time anchors, varying the frozen epoch or TOF can change that branch. Optimizer adapters eliminate those frozen coordinates. The benchmark report records free-column checks at the anchors and full derivative checks away from the transition.

Known limitations

- **Moving CR3BP close lunar passes.** Two moving-CR3BP stress vectors, in P1 and P13, differ from the PyKEP gradient by a scaled `1.07e-6` and `4.23e-6`, above the `1e-7` gate used for the other rows. Both trajectories pass close to the Moon, where the transition matrices are ill-conditioned. Against a `1e-16` reference run, Rust Taylor gradients at tolerances `1e-12` to `1e-15` differ by up to a scaled `2.4e-5` on the same vectors, so the remaining source difference lies inside the double-precision conditioning spread rather than indicating a formulation error. The DOP853 backend agrees with the Taylor gradient within a scaled `5.1e-5`, and central differences of the fitness agree to their truncation error. Both vectors are regression tests in `crates/pykep-trajopt/tests/cr3bp_conditioning.rs`.
- **Moving MEE source discrepancies.** The pinned source applies a different MEE chain rule at the endpoints and returns nonfinite values for four configurations per encoding. The Rust formulation is checked against the source symbolic MEE map instead; see the benchmark report.
- **Moving CR3BP softmax.** The source factory always uses uniform time. The Rust softmax variant is a separate formulation, `tops-v1-cr3bp-moving-softmax`, without an upstream reference.
- **Fixed sail P5 scaling.** The source normalizes the sail characteristic acceleration by the acceleration unit. Because the sail acceleration scales with the inverse square of the distance, this equals the unnormalized constant only when the initial radius is one. P5 starts at radius `0.7`, so `tops.sail.P5` has `0.49` times the sail acceleration of `tops.sail.P5.moving`. The Rust catalog keeps the source behaviour.
- **DOP853 derivatives.** With `backend="dop853"`, the Cartesian two-body and CR3BP Jacobians use analytic model Jacobians. MEE and solar-sail Jacobians use centered-difference model Jacobians, so their accuracy is not set by the integrator tolerance. The default Taylor backend differentiates the coefficient recurrences for all four families.

Feasible replay examples

The bundled decisions were generated using a bounded least-squares search with the Rust Jacobian. They satisfy the raw bounds and constraints, and a tighter independent forward

DOP853 integration checks the terminal state.

Case	Segments	Maximum absolute forward residual
Cartesian P4	16	4.88e-14
MEE P8	24	1.59e-13
CR3BP P0	30	1.59e-12
Solar sail P4	24	8.15e-14

Cartesian two-body P4

The first coordinate is final mass. Each segment contributes a throttle fraction and three direction components. The last coordinate is time of flight. The integrator uses the initial radius and mass to set its units. Solve for the seven state and mass matching rows before comparing the final-mass objective.

```
python tools/gym-benchmark/solve_tops.py tops.twobody.P4 --nseg 16 \
  --seed 42 --evaluations 320 --output artifacts/gym/twobody-solved.json
```

Modified equinoctial P8

The integrator normalizes the initial semi-latus rectum and uses prograde MEE. The longitude residual includes its winding, so leave the longitude unwrapped in the terminal check. The saved control directions are expressed in the local radial, transverse and normal frame.

```
python tools/gym-benchmark/solve_tops.py tops.mee.P8 --nseg 24 \
  --seed 42 --evaluations 320 --output artifacts/gym/mee-solved.json
```

Rotating CR3BP P0

The dynamics use the dataset's rotating coordinates and normalized mass ratio. The default fixed-boundary directions have unit-norm equality constraints. Inspect the terminal residual in the rotating frame after replaying the chronological controls.

```
python tools/gym-benchmark/solve_tops.py tops.cr3bp.P0 --nseg 30 \
  --seed 11 --evaluations 900 --output artifacts/gym/cr3bp-solved.json
```

Solar sail P4

Each segment holds cone and clock angles in radians. There is no spacecraft mass coordinate in the six-state integration. The source fixes the TOF, so the six terminal-state rows determine success.

```
python tools/gym-benchmark/solve_tops.py tops.sail.P4 --nseg 24 \
  --seed 11 --evaluations 900 --output artifacts/gym/sail-solved.json
```

The commands use SciPy's bounded least-squares solver with the analytic Rust Jacobian and a `1e-8` feasibility gate. Each uses one thread when the environment sets `OPENBLAS_NUM_THREADS=1` and `OMP_NUM_THREADS=1`.

After installing the wheel built from this checkout, replay a case from the repository root. The other three fixture names are `mee.P8.json`, `cr3bp.P0.json` and `sail.P4.json`.

```
python python/examples/gym/replay.py \
  --solution crates/pykep-trajopt/data/solutions/twobody.P4.json \
  --output artifacts/gym/twobody-trajectory.json
```

Use `--solution` with either the checked-in vector or a newly solved record. The replay includes the original decision, scaled controls, chronological time grid, endpoint states and separate forward/backward sample paths. The backward paths are ordered from the arrival end toward the cut. Use `tools/gym-benchmark/solve_tops.py` with SciPy to regenerate a feasible vector. Its output records the requested budget and actual fitness and gradient calls.

```
python python/examples/gym/plot.py artifacts/gym/twobody-trajectory.json \
  artifacts/gym/twobody-trajectory.png
python python/examples/gym/replay.py --problem tops.twobody.P4.moving --nseg 16
```

Plotting uses Matplotlib in a separate process. The moving-boundary command illustrates a finite midpoint evaluation.

Set the TOPS backend explicitly to `taylor` or `dop853`. Nominal and derivative tolerances are separate options. See measured objective and derivative performance.

Impulsive trajectory formulations

The impulsive problems use the core ephemeris, Lambert, Lagrange propagation, and flyby routines. Encounter epochs are MJD2000 days, coast durations are seconds, states use metres and metres per second, and maneuver magnitudes are metres per second.

- MGA joins successive encounter positions with zero-revolution Lambert arcs, applies powered flyby costs, then adds the configured launch and arrival costs. Direct, alpha and eta encodings specify different bound boxes. They evaluate the same physical trajectory when decoded into the same leg durations.
- MGA-1DSM propagates a coast before each maneuver and closes the leg using a Lambert arc. Later legs begin with an unpowered flyby. The decoded trace includes the DSM vectors, flyby radii and angles, and each coast/Lambert arc. The launch uvV vector uses two angles encoded in the unit interval and a magnitude in metres per second.
- N-impulse problems use alpha weights for the coast times. The final two burns come from the last Lambert arc and the arrival velocity match. The Earth–Mars gym instances use three, five or seven impulses.
- JUICE evaluates the Ariane 5 table at the decoded launch speed and declination. It applies the pinned insertion orbit, the added 275 m/s cost and rocket-equation mass loss. The first minimized objective is negative log final mass. The multiobjective variant also minimizes total flight time in days.

The launcher interpolation matches the pinned source conventions. The two-dimensional splines clamp to their tabulated domain. The one-dimensional Atlas 551 and Falcon 9 tables return 0.1 kg outside their speed range. `Launcher::mass` and `launcher_mass` accept speed in kilometres per second and declination in degrees.

Run the generic replay example with any classical ID and inspect the returned arc endpoint closure. Use MODE to produce a JUICE trade-off study.

```
python python/examples/gym/replay.py --problem gym.cassini1
python python/examples/gym/replay.py --problem gym.rosetta
python python/examples/gym/replay.py --problem gym.em7imp
cargo run --release --locked -p pykep-gym-benchmark -- optimize \
  --problem gym.juice_mo --optimizer mode --evaluations 4096 \
  --seed 42 --workers 1 --output artifacts/gym/juice
```

The `gym.*` namespace identifies the current PyKEP definitions. The legacy fcmaes GTOP objectives use separate result identifiers and kernels.

Current gym	Related legacy GTOP	Definition details
`gym.cassini1`, `gym.cassini1_a`, `gym.cassini1_n`	`cassini1`	Fixed encounter sequence with distinct encoding, ephemeris and launch conventions. The legacy mixed-integer variant includes sequence decisions.
`gym.cassini2`	`cassini2`	Current gym charges Saturn insertion. Legacy GTOP uses rendezvous in the DSM formulation.
`gym.messenger`	`messenger`, `messenger_full`	The reduced legacy case has a different sequence. The full case shares the insertion target. Ephemeris, constants and bounds follow each implementation's definition.
`gym.rosetta`	`rosetta`	Both use a Keplerian comet endpoint. Their ephemeris and objective conventions are versioned separately.
EVE, JUICE, Earth–Mars impulse, TOPS	Native gym IDs	These use the definitions in the pinned PyKEP gym.

See the legacy implementations. See the current upstream definitions.

Python API contract

Install the native wheel and import the collision-safe package:

```
import numpy as np
import pykep_rust as pk

epoch = pk.Epoch.from_iso("2030-01")
state = pk.classical_to_cartesian(
    [7.0e6, 0.01, 0.4, 1.0, 0.5, 0.2], pk.MU_EARTH
)
future = pk.propagate_lagrangian(state, 60.0, pk.MU_EARTH)

batch = np.asarray([state, future], dtype=np.float64)
times = np.asarray([60.0, -60.0], dtype=np.float64)
propagated = pk.propagate_lagrangian_batch(batch, times, pk.MU_EARTH)
assert propagated.shape == (2, 6)

parallel = pk.propagate_lagrangian_batch(
    batch, times, pk.MU_EARTH, workers=4
)
assert np.array_equal(parallel, propagated)
```

Trajectory gym

Release wheels from 0.1.5, built with the `trajopt` feature, include `pykep_rust.trajopt.gym`. The problem protocol exposes raw fitness, flattened TOPS gradients, metadata, decoding and ordered float64 batches. See gym constructors and examples and Python timing boundaries.

Units, shapes, and defaults

- Angles are radians. Epoch and ephemeris arguments are MJD2000 days. Propagation durations are seconds when the supplied gravitational parameter uses SI units; normalized inputs produce normalized output.
- Cartesian states are `[x, y, z, vx, vy, vz]`. Classical elements are `[a, e, i, Ω, ω, v]`. Modified equinoctial elements are `[p, f, g, h, k, L]`.
- Scalar vector inputs accept finite Python sequences. Element and propagation batches require two-dimensional `float64` arrays with shape `N × 6`; one-dimensional epoch/time batches require `float64` arrays. Strided and read-only arrays are accepted and outputs are newly owned.

- Every parallel batch accepts `workers=0` for the shared pool, `workers=1` for a serial native loop, or an exact positive worker count. Results and the first reported error retain input order.
- Jacobians are row-major, output-by-input. State-transition matrices are `6 × 6`. Sims–Flanagan and ZOH leg matrix shapes are documented with their transcription in the low-thrust guides.
- Adaptive propagators default to relative and absolute tolerance `1e-12`, no maximum step, and an implementation limit of 100,000 accepted/rejected steps where exposed. ZOH legs default to `cut=0.5`.

Errors and ownership

NaN, infinity, invalid dimensions, non-positive physical parameters, and invalid time grids are rejected before numerical work. Invalid input uses `ValueError` (or `TypeError` when a NumPy dtype/rank cannot satisfy the typed buffer). Numerical failures derive from `PykepError`: `ConvergenceError`, `SingularGeometryError`, `IntegrationError`, and `UnsupportedCapabilityError`.

Objects copy constructor inputs and can outlive those Python sequences. `Planet` and immutable leg objects can be reused from multiple Python threads. Batch conversion, anomaly, Stumpff, ephemeris, mission, propagation, dynamics, and leg workloads release the GIL while executing the native loop. Python wrappers validate and convert data for the Rust libraries.

The nominal Kepler, CR3BP, BCP, Pontryagin, and ZOH propagators accept the keyword-only `method="taylor"` or `method="dop853"` in their scalar and batch forms. Kepler, CR3BP, and BCP default to Taylor. Pontryagin and ZOH schedule calls default to DOP853. `ZohLeg` mismatch, history and Jacobian calls, including their batch forms, accept the selector and default to DOP853.

See batch-processing.md for the complete batch matrix, array shapes, worker-pool contract, SpOC 4 motivation, and guidance for avoiding nested parallelism.

The complete callable surface and defaults are statically described by the shipped `py.typed` marker, `_pykep_rust.pyi` and `trajopt/gym.pyi`. See python-migration.md for the upstream name and behavior mapping, conventions.md for numerical conventions, and the model-specific dynamics and leg guides for parameter layouts.

Ordered parallel batch processing

The trajectory gym also provides ordered fitness batches. The matched report separates those measurements from single-call timings.

`pykep-core` and `pykep_rust` deliberately extend the scalar-oriented pykep surface with ordered parallel batches. This is not an upstream compatibility claim. Experience optimizing large Lambert-transfer populations during the SpOC 4 competition, captured in the `dietmarwo/pykep-lambert` comparison project, motivated moving the reusable pattern into this library.

The extension addresses two independent costs:

1. one Python-to-native transition replaces thousands of scalar calls; and
2. independent native evaluations can use several CPU cores.

Every batch returns results in input order and uses the declared physical model, units, frame, tolerances, and error criteria. SIMD Taylor evaluations can differ from scalar results at the level of floating-point rounding.

Worker contract

APIs with a `workers` argument use one shared contract:

`workers`	Execution
`0`	Rayon's process-wide shared pool; this is the Python default
`1`	Serial native loop
`N > 1`	A cached pool containing exactly `N` worker threads

Explicit pools are cached by worker count, so repeated optimizer generations do not pay thread-startup cost. Explicit counts above 1024 are rejected. Python releases the GIL around native work.

Parallel evaluation collects row results before returning an error. This keeps both successful output and the reported first error deterministic in input order, even if a later failing row finishes first on another worker. An empty batch returns an empty result with the documented output shape.

Do not add another thread pool around a parallel batch. In an already parallel optimizer or task scheduler, pass `workers=1` at the inner level to avoid oversubscription. Cheap arithmetic often

benefits from one native batch because it removes Python-call overhead but may not benefit from multiple threads. Measure representative batch sizes.

Enable `pykep-core/simd` for the built-in Taylor batches that support four lanes. Groups of four use a shared vector coefficient jet with separate lane steps. Smaller batches and the remaining jobs in a partial group use scalar propagation. The benchmark's `--batch-sweep` mode measures crossover sizes and reports lane utilization with the same worker policy.

Rust interfaces

The following astrodynamics operations have named Rust batch APIs:

- `propagate_lagrangian_batch`, `propagate_universal_batch`, and `propagate_keplerian_batch`;
- `propagate_lagrangian_with_stm_batch` and `propagate_lagrangian_grid_parallel`;
- `solve_lambert_batch`, using one `LambertRequest` per problem;
- `Ephemeris::states_parallel`, `accelerations_parallel`, `periods_parallel`, and `elements_parallel`;
- all anomaly-conversion functions with a `_batch` suffix;
- `dot_batch`, `norm_batch`, `normalize_batch`, `cross_batch`, and `skew_batch`;
- `evaluate_zoh_mismatch_batch_parallel`.

Other immutable scalar Rust operations can use the same public `pykep_core::batch::try_map` executor. This avoids duplicating dozens of near-identical request structures while preserving typed scalar functions as the only numerical implementation:

```
use pykep_core::astro::transfers::hohmann;
use pykep_core::batch::try_map;

let radii = [(1.0, 2.0), (1.2, 2.4)];
let transfers = try_map(&radii, 2, |&(r1, r2)| hohmann(r1, r2, 1.0))?;
assert_eq!(transfers.len(), radii.len());
```

`try_map` is appropriate only for independent rows. Algorithms that carry state from one time to the next remain scalar/sequential unless their public contract explicitly defines independent initial-value problems.

Python interfaces

Fixed-size numeric batches use `float64` NumPy arrays. Ragged schedules, encoding vectors, and batches of immutable leg objects use Python sequences. The shipped `_pykep_rust.pyi` is the authoritative signature reference.

Foundations and representations

Family	Batch functions
Stumpff	`stumpff_c_batch`, `stumpff_s_batch`
Anomalies	`_batch` counterpart for every elliptic and hyperbolic anomaly conversion
Three-vectors	`dot_batch`, `norm_batch`, `normalize_batch`, `cross_batch`, `skew_batch`
Elements	`_batch` counterpart for all six conversions
Element derivatives	`cartesian_to_modified_equinoctial_jacobian_batch`, `modified_equinoctial_to_cartesian_jacobian_batch`

Vector input has shape `N × 3`; element/state input has shape `N × 6`. Vector outputs have shape `N` or `N × 3`, skew matrices `N × 3 × 3`, element outputs `N × 6`, and element Jacobians `N × 6 × 6`.

Propagation, Lambert, and ephemerides

Family	Batch functions
Two-body	`propagate_lagrangian_batch`, `propagate_universal_batch`, `propagate_keplerian_batch`
Two-body STM/grid	`propagate_lagrangian_with_stm_batch`, `propagate_lagrangian_grid(..., workers=...)`
Adaptive evaluated dynamics	`propagate_kepler_dynamics_batch`, `propagate_cr3bp_batch`, `propagate_bcp_batch` and all three `_with_stm_batch` variants
Lambert	`lambert_problem_batch`
Ephemerides	`Planet.states`, `Planet.acceleration_batch`, `Planet.period_batch`, `Planet.elements_batch`

Propagation batches pair row `states[i]` with `times[i]` or `final_times[i]`. STM batches return `(states[N,6], stms[N,6,6])`. `lambert_problem_batch` pairs `initial_positions[N,3]`, `final_positions[N,3]`, and `times[N]`, while sharing `mu`, direction, and maximum-revolution configuration. It returns `N` normal `LambertProblem` objects; each object retains the scalar zero/left/right branch ordering.

```
states = np.tile(
    np.asarray([1.0, 0.0, 0.0, 0.0, 1.0, 0.0]), (32_768, 1)
)
times = np.linspace(0.0, 1.0, len(states))
propagated = pk.propagate_lagrangian_batch(
    states, times, 1.0, workers=8
)
assert propagated.shape == states.shape
```

Mission utilities

Family	Batch functions
Circular transfers	`hohmann_batch`, `bielliptic_batch`
Time encodings	`alpha_to_direct_batch`, `direct_to_alpha_batch`, `eta_to_direct_batch`, `direct_to_eta_batch`
Flybys	`flyby_constraints_batch`, `flyby_constraints_jacobian_batch`, `flyby_delta_v_batch`, `flyby_outgoing_velocity_batch`
Mass estimates	`mima_batch`, `mima2_batch`

Hohmann and bi-elliptic batches return `(delta_v[N], time[N], impulses[N,K])`. Flyby constraint output is `N × 2`, its Jacobian is `N × 2 × 6`, and outgoing velocity is `N × 3`. MIMA variants return `(mass[N], acceleration[N])`.

Controlled dynamics and legs

Family	Batch functions
Evaluated RHS/invariants	`kepler_rhs_batch`, `cr3bp_rhs_batch`, `bcp_rhs_batch`, `cr3bp_effective_potential_batch`, `cr3bp_jacobi_constant_batch`
ZOH RHS	all four `zoh_*_rhs_batch` functions
ZOH schedules	all four `propagate_zoh_*_batch` functions

Family	Batch functions
Pontryagin	Cartesian/equinoctial RHS, control, Hamiltonian, and propagation `_batch` functions
Sims–Flanagan objects	class batch methods for mismatch/throttle constraints and available Jacobians
Generic ZOH-leg objects	`mismatch_constraints_batch`, `mismatch_jacobian_batch`, `state_history_batch`

ZOH schedule batches accept one state row, boundary vector, and control matrix per independent schedule. Shared physical constants and integrator settings remain scalar arguments. Pontryagin batches share one optimality mode and parameter vector.

Where the primitives are useful

These batches support existing computations without adding new physical models:

- Lambert departure/arrival/time-of-flight sweeps;
- pork-chop or transfer-table calculations assembled from ephemeris and Lambert batches;
- optimizer populations for CMA-ES, differential evolution, PGPE, MODE, or similar methods;
- Monte Carlo evaluation of independent uncertain initial conditions;
- state/STM ensembles for covariance or sensitivity workflows;
- independent flyby candidates, MIMA estimates, and low-thrust legs.

Closest-approach, eclipse, access, conjunction, and other event physics are not implemented merely because the underlying state batches make such external calculations easier. Use an appropriate validated model for those quantities.

Validation

Rust tests compare named batches with scalar functions for serial, global, and explicit worker pools and verify deterministic error order. Python tests exercise every batch family, compare values with scalar calls, validate shapes, and verify the runtime exports against the typed stub. The companion `pykep-lambert` project remains the end-to-end performance comparison for the SpOC 4–motivated propagation/Lambert workload.

Python migration from kep3

The native package is named `pykep_rust`, so it can be installed beside `pykep` / `kep3` without import collisions. Phase 16 deliberately does not add a compatibility facade: a partial set of legacy spellings would hide important differences in ownership, dynamics construction, and error behavior. A separate facade can be considered after the deferred ecosystem modules have clear support decisions.

The comparison below is against the public Python surface of the pinned upstream version recorded in `UPSTREAM_NOTICE.md`.

Equivalent capabilities with descriptive names

Upstream	`pykep_rust`	Notes
`AU`, `CAVENDISH`, `EARTH_VELOCITY`, `G0`	`ASTRONOMICAL_UNIT`, `CAVENDISH_CONSTANT`, `EARTH_ORBITAL_VELOCITY`, `STANDARD_GRAVITY`	Same pinned SI values
`RAD2DEG`, `DEG2RAD`, `DAY2SEC`, `SEC2DAY`	`RADIANS_TO_DEGREES`, `DEGREES_TO_RADIANS`, `DAY_TO_SECONDS`, `SECONDS_TO_DAY`	Same conversion factors
`m2e`, `e2m`, `m2f`, `f2m`, `e2f`, `f2e`	`mean_to_eccentric_anomaly`, `eccentric_to_mean_anomaly`, `mean_to_true_anomaly`, `true_to_mean_anomaly`, `eccentric_to_true_anomaly`, `true_to_eccentric_anomaly`	Scalar elliptic conversions
`n2h`, `h2n`, `n2f`, `f2n`, `h2f`, `f2h`	Gudermannian/hyperbolic functions with full names	Scalar hyperbolic conversions
`m2e_v`, `e2m_v`, `m2f_v`, `f2m_v`, `e2f_v`, `f2e_v`	Corresponding descriptive elliptic `_batch` functions	Ordered native batches that release the GIL
`n2h_v`, `h2n_v`, `n2f_v`, `f2n_v`, `h2f_v`, `f2h_v`, `zeta2f_v`, `f2zeta_v`	Corresponding descriptive hyperbolic/Gudermannian `_batch` functions	Ordered native batches that release the GIL

Upstream	`pykep_rust`	Notes
`ic2par`, `par2ic`, `ic2mee`, `mee2ic`, `par2mee`, `mee2par`	Cartesian/classical/modified-equinoctial functions with full names	`N × 6` NumPy batches are explicit `_batch` APIs
Lagrangian and Taylor propagation families	`propagate_lagrangian`, `propagate_lagrangian_grid`, evaluated model propagators	Native Rust numerical implementation
`lambert_problem`	`LambertProblem`, `LambertSolution`	Deterministic branch objects
`hohmann`, `bielliptic`, `mima`, `mima2`	Same names	Return values are typed in the stub
`alpha2direct`, `direct2alpha`, `eta2direct`, `direct2eta`	`alpha_to_direct`, `direct_to_alpha`, `eta_to_direct`, `direct_to_eta`	Descriptive direction
`fb_con`, `fb_dv`, `fb_vout`	`flyby_constraints`, `flyby_delta_v`, `flyby_outgoing_velocity`	Jacobian has a separate named function
`leg.sims_flanagan`, `leg.sims_flanagan_alpha`, `leg.zoh`	`SimsFlanaganLeg`, `SimsFlanaganAlphaLeg`, `ZohLeg`	Same numerical cores, immutable validated construction
Selected `trajopt.gym` and `trajopt.launchers` from PyKEP 3.1.0	`pykep_rust.trajopt.gym` in wheels from 0.1.5	Stable IDs, raw PyGMO-compatible fitness and analytic TOPS gradients

Intentionally different

Area	`pykep_rust` contract
Epochs	`Epoch` is immutable, has a microsecond-granular internal representation, and requires an explicit `mjd2000`, `mjd`, or `jd` numeric scale. Binary64 JD input has about 40 µs spacing near J2000; use MJD2000 or calendar construction for single-microsecond input resolution. Arithmetic day counts do not imply UTC, TT, or TDB.
Planets	`Planet` uses explicit static constructors and owns a thread-safe native provider instead of accepting arbitrary Python UDPLA objects.
Taylor dynamics	Python receives evaluated RHS and propagation functions. It does not expose or require heyoka expression graphs or integrator objects.
Time-optimal Pontryagin	The upstream unused barrier parameter is omitted and `lambda0` is fixed to `1`; callers that vary time-optimal `lambda0` must rescale their formulation explicitly.
ZOH legs	`ZohModel` selects one of four built-in native dynamics. Constructor input is copied; validated legs do not expose mutation setters.
Errors	Invalid values/shapes raise `ValueError`; singular geometry, convergence, integration, and missing capabilities have typed `PykepError` subclasses.
Batches	Throughput-sensitive entry points explicitly accept `float64` NumPy arrays, preserve order, return owned arrays, and release the GIL during native work.

Deferred or unsupported

The following upstream areas are not part of the completed native numerical core and are not emulated:

- SPICE kernels, TLE parsing, and Python-defined UDPLA providers (`PY-EXT-001`);
- general-purpose Python trajectory callbacks and the upstream plotting API (`PY-ECOSYSTEM-001`);
- symbolic heyoka dynamics/integrator construction and arbitrary user-supplied expression graphs (`TA-SYMBOLIC-001`);
- `mima_from_hop` and `mima2_from_hop` (`MIMA-HOP-001`);
- deprecated compatibility aliases and nested namespace layouts.

Use `has_acceleration()` and the VSOP2013 availability/threshold queries when code depends on an optional provider capability. Unsupported provider operations raise

`UnsupportedCapabilityError`; unsupported ecosystem modules are absent rather than silently approximated.

The trajectory gym guide lists the implemented formulations. Matched benchmarks records source-specific differences and separates Python from native speed.

Implementation status

Release 0.1.5 adds the trajectory gym and PyKEP 3.1.0 comparisons. Those reports give the current scope, numerical discrepancies and timing boundaries. The core inventory below records the module status as of 2026-08-03.

Module	Rust core	Python API	Golden parity
Foundations	implemented	implemented	3.0.1
Epoch/anomalies	implemented	implemented	3.0.1
Elements	implemented	implemented	3.0.1
Propagation/STM	implemented	implemented	3.0.1
Lambert/transfers/flyby/MIMA	implemented	implemented	3.0.1
Planet/Keplerian ephemeris	implemented	implemented	3.0.1
JPL low-precision ephemerides	implemented	implemented	3.0.1
VSOP2013 ephemerides	implemented (`>=1e-9` feature)	implemented	3.0.1/heyoka 7.10.0

Module	Rust core	Python API	Golden parity
Adaptive integration backends	DOP853 general; Taylor for 11 built-ins	DOP853 model APIs	analytic/C++/heyok 7.10.1
Kepler/CR3BP/BCP dynamics	implemented	implemented	3.0.1/heyoka 7.10.0
ZOH dynamics	implemented	implemented	3.0.1/heyoka 7.10.0
Pontryagin dynamics	implemented	implemented	3.0.1/heyoka 7.10.0
Sims–Flanagan legs	implemented	implemented	3.0.1
Generic ZOH leg	implemented	implemented	3.0.1/heyoka 7.10.0

Module	Rust core	Python API	Golden parity
Ordered parallel batches	shared executor plus named core batches	listed numerical families	same scalar entry points
Python API audit	same native core	complete typed surface	same core entry points

“Implemented” means the public contract is documented, validation is explicit, the committed C++ golden data passes except for documented numerical improvements, independent properties pass, and Rust/Python tests call the same core implementation. It does not imply that later modules exist.

The Python wheel uses the collision-safe `pykep_rust` import, ships a complete stub and `py.typed`, and has no C++ runtime dependency. Clean-wheel CI covers CPython 3.11–3.13 on Linux, macOS, and Windows. The upstream migration matrix records renames, deliberate contract changes, deferrals, and unsupported ecosystem modules.

The example matrix covers the major public modules in Rust and through the installed Python extension. CI compiles the Rust examples and executes the basic Python examples. The gym has separate API, packaging, finite-difference Jacobian and replay checks.

The synchronized 0.1.5 release is published as `pykep-core` on crates.io and `pykep-rust` on PyPI. The internal `pykep-py` implementation crate remains `publish = false` by design. Tag-gated trusted-publishing workflows, clean crate/wheel/source-distribution consumption, docs.rs, and GitHub Pages are part of the release process. Performance regression, Miri, fuzz, Valgrind, dependency, MSRV, cross-platform wheel, and API-documentation checks remain maintained quality gates rather than claims of formal verification.

Version 0.1.4 introduced incremental Taylor coefficient evaluation for all eleven built-in models. Version 0.1.5 adds the trajectory gym, Taylor events and four-lane batches, tangent Taylor sensitivities, analytic DOP853 model Jacobians for the Kepler and CR3BP ZOH models, and Taylor or DOP853 selection for ZOH-leg mismatch, history and Jacobian calls in Rust and Python. Calls without a selector retain their DOP853 behavior. See the performance decision guide for the measured boundary between the ahead-of-time Rust backend and warmed upstream heyoka.

Source map

The port baseline is pykep/kep3 3.0.1 at commit `53b1ca3ce5f8c223f96819b2ea9ba16c3719e63e`. A checked box means the Rust module, C++ golden parity, independent validation, Python binding, and documentation required by the definition of done are complete.

Header-only numerical sources

- [x] `include/kep3/core_astro/constants.hpp` → `constants` (Phase 2)
- [x] `include/kep3/core_astro/convert_julian_dates.hpp` → `time::julian` (Phase 2)
- [x] `include/kep3/core_astro/kepler_equations.hpp` → `math::kepler_equations` (Phase 2)
- [x] `include/kep3/core_astro/special_functions.hpp` → `math::stumpff` (Phase 2)
- [x] `include/kep3/core_astro/convert_anomalies.hpp` → `astro::anomalies` (Phase 3)

Translation units

- [x] `src/linalg.cpp` → `math::linalg` (Phase 2)
- [x] `src/epoch.cpp` → `time::epoch` (Phase 3)
- [x] `src/core_astro/ic2par2ic.cpp` → `astro::elements::classical` (Phase 4)
- [x] `src/core_astro/mee2par2mee.cpp` → `astro::elements::equinoctial` (Phase 4)
- [x] `src/core_astro/ic2mee2ic.cpp` → `astro::elements::equinoctial` (Phase 4)
- [x] `src/core_astro/propagate_lagrangian.cpp` → `astro::propagation::lagrangian` (Phase 5)
- [x] `src/core_astro/stm.cpp` → `astro::propagation::stm` (Phase 5)
- [x] `src/core_astro/basic_transfers.cpp` → `astro::transfers::basic` (Phase 6)
- [x] `src/core_astro/encodings.cpp` → `astro::encodings` (Phase 6)
- [x] `src/core_astro/flyby.cpp` → `astro::flyby` (Phase 6)
- [x] `src/lambert_problem.cpp` → `astro::lambert` (Phase 6)
- [x] `src/core_astro/mima.cpp` → `astro::mima` (Phase 6)
- [x] `src/planet.cpp` → `ephemeris` (Phase 7)
- [x] `src/udpla/keplerian.cpp` → `ephemeris::keplerian` (Phase 7)
- [x] `src/udpla/jpl_lp.cpp` → `ephemeris::jpl_lp` (Phase 8)
- [x] `src/udpla/vsop2013.cpp` → `ephemeris::vsop2013` (Phase 9)

- ☑ heyoka integration requirements → `integration` facade and ADR 0004 (Phase 10; infrastructure rather than a source translation)
- ☑ high-order Taylor recurrence/controller → `integration::Taylor` (independent implementation for the eleven fixed model types; not a symbolic-runtime translation)
- ☑ `src/ta/kep.cpp` → `dynamics::KeplerDynamics` (Phase 11)
- ☑ `src/ta/cr3bp.cpp` → `dynamics::Cr3bpDynamics` (Phase 11)
- ☑ `src/ta/bcp.cpp` → `dynamics::BcpDynamics` (Phase 11)
- ☑ `src/ta/zoh_kep.cpp` → `dynamics::zoh::ZohKeplerDynamics` (Phase 12)
- ☑ `src/ta/zoh_cr3bp.cpp` → `dynamics::zoh::ZohCr3bpDynamics` (Phase 12)
- ☑ `src/ta/zoh_eq.cpp` → `dynamics::zoh::ZohEquinoctialDynamics` (Phase 12)
- ☑ `src/ta/zoh_ss.cpp` → `dynamics::zoh::ZohSolarSailDynamics` (Phase 12)
- ☑ `src/ta/pontryagin_cartesian.cpp` → `dynamics::pontryagin::{CartesianMassOptimal, CartesianTimeOptimal}` (Phase 13)
- ☑ `src/ta/pontryagin_equinoctial.cpp` → `dynamics::pontryagin::{EquinoctialMassOptimal, EquinoctialTimeOptimal}` (Phase 13)
- ☑ `src/leg/sf_checks.cpp` → `leg::sims_flanagan` validation (Phase 14)
- ☑ `src/leg/sims_flanagan.cpp` → `leg::sims_flanagan` (Phase 14)
- ☑ `src/leg/sims_flanagan_alpha.cpp` → `leg::sims_flanagan::SimsFlanaganAlphaLeg` (Phase 14)
- ☑ `src/leg/zoh.cpp` → `leg::zoh` (Phase 15)

The C++-specific visibility, serialization, and type-erasure support headers are reviewed for semantics but are not port targets.

Current gym and benchmark sources

PyKEP 3.1.0 source	Development counterpart
`pykep/trajopt/gym` and fixed/moving ZOH UDPs	`crates/pykep-trajopt`, embedded TOPS data and `pykep_rust.trajopt.gym`
MGA, MGA-1DSM, N-impulse, launcher tables	Reusable Rust formulations and six launcher models
`benchmark/`	Shared C++ exporter, native Rust and C++ runners, Python study scripts
Benchmark notebooks	Symbolic SF validation and separately identified ephemeris interface studies

The gym guide and benchmark report record the exact scope and oracle results.

Explicitly unavailable upstream ecosystem areas

These are not unchecked source-map rows. They are technically outside the native numerical-core product and have stable internal tracking identifiers:

- `PY-EXT-001`: SPICE kernels, TLE parsing, and Python-defined UDPLA providers require external data/runtime and dynamic Python callback contracts that are absent from the C/C++-free core.
- `TA-SYMBOLIC-001`: arbitrary heyoka expression graphs and user-defined Taylor-integrator objects remain out of scope. Native evaluated dynamics support DOP853 and the eleven built-in types additionally support the fixed-system Taylor backend.
- `PY-ECOSYSTEM-001`: general Python trajectory callbacks, upstream plotting methods and other ecosystem integrations outside the development gym.
- `MIMA-HOP-001`: `mima_from_hop` and `mima2_from_hop` depend on an upstream higher-order-propagation object that the native API intentionally does not expose. The Python migration matrix gives user-visible alternatives. A future change must resolve the corresponding tracking item and add a source-map row before claiming support.

Numerical validation

The matched PyKEP 3.1.0 report records the development gym and shared-input benchmarks. The sections below describe the core's existing oracle and independent numerical checks.

The evidence hierarchy for each numerical API is:

1. direct unit cases and invalid-input tests;
2. golden values generated by the pinned C++ implementation;
3. independent mathematical properties or external reference data;
4. Rust/Python cross-interface parity;
5. equivalent release-mode benchmarks.

Golden files use schema-versioned JSON. Floating-point values are encoded as C99 hexadecimal strings, with `NaN`, `+Infinity`, and `-Infinity` used only when recording upstream non-finite behavior. This preserves every finite binary64 value without relying on a JSON parser's decimal conversion.

The initial C++ baseline was built in release mode with GCC 14.3.0. All 29 upstream C++ test executables passed on 2026-07-25. Benchmark executables were built but their results are not presented as Rust comparisons until equivalent Rust workloads exist.

Randomized oracle cases use a named PCG32 seed recorded in each data file. Discovered failures are promoted to fixed regression cases.

Phase-by-phase evidence

Phase 3: epochs and anomalies

Phase 3 adds 9 epoch cases and 134 anomaly cases from the pinned C++ implementation. The anomaly set includes every conversion direction, elliptic and hyperbolic boundary regimes, angles outside one revolution, and 64 deterministic PCG32 solver samples. The Rust layer additionally rejects invalid calendars and true anomalies outside the physical hyperbolic asymptote instead of propagating non-finite values.

Phase 4: elements and Jacobians

Phase 4 adds 206 direct element/Cartesian cases and 16 analytic Jacobians from the pinned implementation. Jacobian files explicitly record row-major output-by-input order. Independent tests cover 2,000 deterministic elliptic and hyperbolic state round trips, both equinoctial pole conventions, finite-difference derivatives, and inverse-Jacobian identities. NumPy batches are compared row-for-row with the scalar Python API.

Phase 5: two-body propagation

Phase 5 adds 28 propagation cases spanning zero and negative duration, circular, elliptic, hyperbolic, near-parabolic, and many-period trajectories. Each case records both Lagrange-coefficient and universal-variable states plus the Lagrangian and Reynolds 6 by 6 STMs. The source file SHA-256 is `63e2340aa8e4f65a4ae831b7e1c8eb3a28930e2e1ed336d9324e0dbf44469e57`.

Independent tests check energy and angular-momentum conservation, time reversal, central finite differences, and STM composition. Scalar propagation uses only fixed-size stack values; the core path performs zero heap allocations. Batch APIs allocate exactly their returned output storage after copying Python-owned input before releasing the GIL.

Phase 6: mission-design utilities

Phase 6 records transfer, encoding, flyby, MIMA, and 13 Lambert solutions across three geometries in `phase6-v1.json` (SHA-256 `d2a81311264ecc94a6caee15854273abb4df5f32f682fe5340ffeb247e3dd3e3`). Lambert ordering is zero revolution followed by left/right pairs. Every returned branch is independently propagated to its requested endpoint; encoding pairs round-trip, and the flyby Jacobian is checked by central differences. MIMA2 is checked against the published upstream reference case.

Phase 7: Keplerian ephemerides

Phase 7 adds six Keplerian states over negative, reference, near-reference, and long-span MJD2000 epochs. `phase7-v1.json` has SHA-256 `7eda8fb03796ab7d70cdb0e94c422773422ed682aff9310cd06512c0f6396a54`. Independent tests cover period recurrence, all supported element representations, ordered batches, explicit unsupported capabilities, and concurrent read-only evaluation through shared ownership.

Phase 8: JPL low-precision ephemerides

Phase 8 adds true-anomaly elements, Cartesian states, and physical metadata for all eight JPL low-precision bodies at five epochs spanning the open 1800–2050 validity interval. `phase8-v1.json` contains 40 cases and has SHA-256 `0a3408893b5c04fdfddb452408057faa92d38a46775290c32eb6d2393683e3da`. Independent tests cover case-insensitive lookup, the exact supported-name set, ordered batches, safe-radius validation, and both excluded interval boundaries. The provider retains the source table's slightly negative fitted Earth inclination near the ends of the interval while keeping the general public classical-element converter's canonical `[0, π]` inclination contract.

Phase 9: VSOP2013 ephemerides

Phase 9 adds 54 VSOP2013 states covering all nine bodies at six epochs around the 1890–2000 fit interval and the J2000/MJD2000 half-day offset. Two additional cases verify coefficient selection at the default `1e-5` and coarse `0.5` thresholds. `phase9-v1.json` has SHA-256 `9d5af01df18acb17fcd1b2356af6f2cc08f1cf75e10864270d0c30732f8b00d8`.

The oracle uses pykep 3.0.1 linked to heyoka 7.10.0. At the embedded high-precision floor of `1e-9`, the maximum observed Rust/C++ difference is 0.185 m in position and `6.6e-8 m/s` in velocity. Independent checks cover case-insensitive names, feature reporting, threshold errors, clone determinism, ordered Python batches, and a build/test run without the optional coefficient feature.

Phase 10: adaptive integration

Phase 10 selects a pure-Rust DOP853 backend through a pykep-owned facade. Decision tests compare Kepler states and the 6 by 6 STM to the independent analytic propagator, verify a parameter-sensitivity column by central differences, bound 100-orbit energy drift, and exercise a CR3BP close approach with Jacobi drift and backward reversal checks. Separate tests cover rejected steps, step exhaustion, bit-for-bit repeated solves, dense interpolation, terminal root location, malformed grids, non-finite values, and physical singularities. The selected final-state path retains no internal trajectory and performs no per-step heap allocation for fixed-size model states.

The candidate harness and matching C++ benchmark use the same six-state Kepler initial condition, final time, and `1e-12` scalar tolerances. Nominal and variational timings, allocation limitations, and the dense-output maximum-step caveat are recorded in ADR 0004. Those tests validate integration machinery independently of the production models added in Phase 11.

Phase 11: evaluated dynamics

Phase 11 adds five sampled states and the final 6 by 6 STM for each of Kepler, CR3BP, and BCP in `phase11-v1.json`. The BCP case uses a nonzero Sun mass and a nonzero initial epoch, while the CR3BP case reproduces the representative upstream trajectory. The file has SHA-256 `1c2b67eb203da62baf921db9f811b0ad0f763cbaa03f0ca80d6319870b0da807`.

The oracle uses pykep 3.0.1 and heyoka 7.10.0 at a requested tolerance of `1e-16`; Rust comparison settings and achieved tolerances are documented in `dynamics.md`. Separate tests evaluate the source equations directly, verify the triangular equilibrium, preserve the Jacobi constant, make zero-Sun BCP reduce to CR3BP, check every state/parameter Jacobian column by central differences, and distinguish body singularities from solver failures.

Phase 12: zero-order-hold dynamics

Phase 12 records the final state and all first-order state/control variations for one upstream regression case from each of the four ZOH systems. `phase12-v1.json` has SHA-256 `294bab93355628cd22991b83563c6f93deefc19dd93911dfc4b654a101764f22`. Single-segment nominal tolerances range from `2e-12` to `3e-10`; variation tolerances range from `2e-7` to `2e-5`, reflecting the fixed-size numerical Jacobians documented in `zero-order-hold.md`. Independent tests cover exact switch ownership, malformed grids, zero-control reductions, manual segment-by-segment equivalence, forward/backward reversal, and activation of per-segment sensitivity columns.

Phase 13: Pontryagin dynamics

Phase 13 records mass- and time-optimal Cartesian and modified-equinoctial states and all first-order variations with respect to the seven initial costates and upstream `lambda0` variational argument. It also records the upstream dimensional 100-day equinoctial case. `phase13-v1.json` has SHA-256 `a1276c4c35c7ad60b481d81d69f8df64b542bad1d5cc6571738db820cb0e2c3d`. Normalized nominal trajectories use a scaled `3e-10` bound, the dimensional case uses `3e-9`, and variations use `2e-4` to account for the generic centered numerical Jacobians.

Independent tests cover minimized-Hamiltonian conservation, propagated central differences, output-by-input Jacobian orientation, explicit zero-primer errors, control normalization, and canonical Hamiltonian agreement across the analytic coordinate/costate transform.

Phase 14: Sims–Flanagan legs

Phase 14 records fixed-duration Sims–Flanagan mismatch and throttle constraints, all three analytic mismatch-Jacobian groups, and throttle Jacobians for one physical five-segment case and normalized four-segment cases at cuts zero, one half, and one. It also records equal and irregular direct durations for the alpha variant. `phase14-v1.json` has SHA-256 `0bd6adddc72d850f1de5e4ab95a436128d9b8181f8de2b3bbbc67300e004d542`. The Rust analytic gradients agree with the pinned C++ values to scaled bounds of `2e-10` to `3e-10` and with independent scale-adjusted central differences.

Separate tests cover one segment, odd/even splits, zero and unit-limit throttle, all cut boundaries, normalized weights, invalid propulsion/gravity/mass values, dimension mismatches, and non-finite inputs.

Phase 15: generic ZOH legs

Phase 15 records generic ZOH-leg mismatches and all four Jacobian groups for Kepler, CR3BP, modified-equinoctial, and ideal solar-sail dynamics. `phase15-v1.json` has SHA-256 `9d8d424c2af10ceee497f74f62acb30c53924f78f7be8a528dbe44bf14767935`. Nominal mismatches agree with the pinned C++/heyoka oracle within a scaled `3e-9` bound; endpoint, chronological-control, and time-grid derivatives agree within `3e-5`, reflecting the fixed-size numerical model Jacobians. Every Kepler derivative column is also checked against independent scale-adjusted central differences.

Separate tests cover cut zero and one, strict time grids, dimensions, finite values, solver options, state histories, ordered batches, and maximum-step exhaustion with direction, segment index, and time interval in the reported failure.

Phase 16: Python API

Phase 16 audits the complete Python surface against the pinned upstream exports and records every equivalent, renamed, intentionally different, deferred, and unsupported area in `python-migration.md`.

Pytest checks every runtime export against the shipped stub, including parameter names/order, defaults, return-annotation presence, and class-member documentation, then exercises strided/read-only arrays, wrong dtype and rank, wrong shape, NaN/infinity, typed exceptions, repeatability, copied constructor inputs, and concurrent reuse. Mypy checks representative code against the packaged stub. A clean wheel matrix installs and imports CPython 3.11–3.13 wheels on Linux, macOS, and Windows; local shared-library inspection verifies that the extension has no C++ runtime dependency.

Phase 17: runnable examples

Phase 17 adds deterministic Rust/Python pairs for epochs and anomalies, elements and propagation, Lambert branches, ephemerides, gravity assists, Sims–Flanagan gradients, and CR3BP/ZOH dynamics, plus a NumPy batch example. Every Rust binary is compiled under workspace test and clippy gates and run locally in release mode. Pytest discovers and runs every Python script against the installed release extension. Ten Rustdoc examples cover every major module landing page and representative Lambert/ZOH types and compile with warnings denied.

Phase 18: release stabilization

Phase 18 adds a protocol-matched 100-sample Rust/C++ performance distribution, coarse CI regression thresholds, batch scaling, and allocation/cache/ vectorization profiling before any optimization. Miri runs suitable structural and parser tests, Valgrind checks the release harness, and bounded libFuzzer/AddressSanitizer campaigns cover epoch parsing, element conversion, Lambert inputs, and Reynolds-STM overflow boundaries. `stabilization.md` records exact results, limits, tool constraints, and the absence of algorithm changes.

Taylor extension

The Taylor extension adds closed-form recurrence tests, coefficient-level comparisons between the optimized incremental evaluators and an independent full-series reference through order 24, matched DOP853 trajectories for all eleven built-in model types, and a separate official heyoka 7.10.1 fixture for Kepler, CR3BP, BCP, and the Kepler STM.

`taylor-heyoka-v1.json` is regenerated by the optional `tools/heyoka-cross-validation/generate.py` harness. The committed one/100/1,000-revolution accuracy and timing protocol is `data/taylor-kepler-benchmark.csv`; analytical Lagrange propagation is its state-error reference. The ZOH and Pontryagin Taylor checks use their existing pinned upstream fixtures plus same-problem DOP853 comparisons and are not mislabelled as direct heyoka runs.

Downstream release rehearsals

Before changing the built-in nominal default to Taylor, two downstream applications were rehearsed against the local crate through a temporary Cargo registry patch:

- `fcmaes-rust/tutorials/gtoc1` passed all five release tests and reproduced its stored score, mismatch, flyby margin, and sampled solar distance exactly. A disposable physical-units check propagated all seven stored Lambert arcs with analytical Lagrange, Taylor, and DOP853 propagation at `rtol = 1e-12`, `atol = 1e-6`. Taylor's maximum normalized state error over the arcs was `1.1e-12`, versus `3.8e-10` for DOP853.
- `pykep-lambert` passed all eleven release tests, including scalar/batch identity and every optimizer mode. Its standard 32,768-transfer workload retained identical checksums and the same feasible optimizer solution. Five warmed runs remained consistent with its published medians; this application uses analytical Keplerian ephemerides and Lambert solves, so the Taylor default does not alter its normal computation path.

Neither downstream manifest was changed during that pre-release rehearsal; both pinned `pykep-core = "=0.1.2"` at the time. They were subsequently moved to published versions and retested: the GTOC1 tutorial now pins 0.1.4, while `pykep-lambert` pins 0.1.3 because its analytical ephemeris/Lambert workload does not exercise the 0.1.4 Taylor and ZOH-leg changes.

Performance methodology

See the shared-input PyKEP 3.1.0 report for the development comparison. The measurements below are historical orientation runs on the published core and earlier implementation milestones.

Benchmarks use release mode and Criterion. They contain no file or console I/O inside the measured loop. Raw Criterion state is build output under `target/` and is not committed.

An orientation run on 2026-07-25 used Rust 1.97.1 on an AMD Ryzen 9 9950X under Linux 6.8.0-136. One Criterion process reported:

Foundation workload	Median estimate
`stumpff_c(1e-12)`	1.709 ns
`stumpff_s(-4)`	9.437 ns
`jd_to_mjd2000`	0.240 ns
three-vector cross product	4.794 ns
elliptic mean → eccentric, `e = 0.999`	89.51 ns
hyperbolic mean → anomaly, `e = 1.5`	118.3 ns
64 elliptic conversions, `e = 0.9`	5.857 µs
classical → Cartesian	40.25 ns
Cartesian → modified equinoctial	29.33 ns
Cartesian → equinoctial Jacobian	151.7 ns
64 classical → Cartesian conversions	2.575 µs

The Phase 5 orientation run used identical scalar inputs in Rust and the pinned C++ oracle, with both built in release mode on the same machine:

Propagation workload	Rust Criterion median	C++ elapsed average
Lagrange, elliptic	146.24 ns	149.263 ns
Lagrange, hyperbolic	567.50 ns	450.166 ns
Universal variables, elliptic	250.05 ns	260.613 ns
Lagrange propagation + STM	216.68 ns	444.289 ns
1,024 Lagrange calls	113.78 µs (9.00 million/s)	—

Mission-design kernels

Phase 6 measured each operation separately in the same Rust orientation run:

Workload	Rust median
Hohmann transfer	6.258 ns
Flyby constraints	10.923 ns
Flyby delta-v	34.567 ns
Zero-revolution Lambert problem	235.76 ns
Seven-solution multi-revolution Lambert problem	1.263 µs

Ephemerides

Scalar and ordered-batch measurements were kept separate:

Phase	Provider and workload	Result
7	Keplerian, one epoch	80.299 ns
7	Keplerian, 256 ordered epochs	22.575 µs
8	JPL low-precision Earth, one epoch	95.198 ns
8	JPL low-precision Earth, 256 ordered epochs	31.611 µs
9	VSOP2013 default-threshold initialization	11.615 µs
9	VSOP2013 default-threshold scalar state	339.98 ns
9	VSOP2013 default-threshold 256-state batch	89.554 µs
9	VSOP2013 `1e-9` scalar state	37.157 µs

The matching C++/heyoka VSOP2013 harness measured first-time JIT costs of 63.96 ms at `1e-5` and 552.53 ms at `1e-9`. Warm scalar calls took 166 ns and 5.908 µs, respectively. Enabling VSOP2013 increased the Rust release benchmark executable from 2.6 MiB to 7.0 MiB. ADR 0003 explains the data and cache decision.

Integration and dynamics

DOP853 selection

The Phase 10 decision harness produced these orientation measurements:

Workload	Selected Rust facade	Warmed C++/heyoka
Representative nominal Kepler solve	11.865 µs	3.722 µs
State plus 6 by 6 STM	85.663 µs	84.440 µs

Cloning the cached C++ nominal integrator once cost 0.467 ms. In a same-profile candidate test, the selected facade took 15.431 µs and `ode_solvers` took 7.369 µs. The latter lacked the required root and sensitivity facilities and allocated per step, so nominal timing alone did not decide the dependency. ADR 0004 records the configuration, ranges, and risks.

Taylor integration

The fixed-system Taylor backend uses a matched-accuracy protocol because coefficient sweeps are not comparable to DOP853 right-hand-side calls. For one eccentric nondimensional revolution, Taylor's speed relative to DOP853 was:

Tolerance	Taylor/DOP853 speed
`1e-9`	0.87×
`1e-12`	1.18×
`1e-14`	1.61×
Machine epsilon	2.16×

Taylor produced smaller final-state errors at every point. This is deliberately a narrow conclusion: Taylor is a high-accuracy option, not a low-accuracy replacement. The committed CSV also contains the 100- and 1,000-revolution rows; see High-accuracy Taylor integration.

All eleven built-in Taylor models now use incremental coefficient evaluators. Across the representative eight-model migration benchmark, replacing repeated full-series evaluation reduced warmed end-to-end propagation time by 6.45× to 28.97×. The linked Taylor guide gives per-model results and validation limits.

Evaluated models and controlled dynamics

Phase	Workload	Rust	Warmed C++/heyoka
11	Kepler RHS	10.828 ns	—
11	CR3BP RHS	32.833 ns	—
11	BCP RHS	63.386 ns	—
11	CR3BP propagation	7.808 µs	2.885 µs
11	CR3BP state plus STM	45.368 µs	95.017 µs
12	ZOH Kepler RHS	8.928 ns	—
12	32-segment alternating-control schedule	36.089 µs	10.268 µs
13	Cartesian mass-optimal RHS	52.384 ns	—
13	Cartesian mass-optimal propagation	142.52 µs	11.716 µs

The Phase 11 comparisons use the same initial state, final time, parameter, and `1e-12` tolerance.

The Phase 12 Rust schedule deliberately starts an independent DOP853 solve at each switch. Its timing includes all 32 restarts and confirms that integration does not perform a segment-count-dependent control search.

The Phase 13 propagation covers 1.2345 normalized time units. Rust uses a `0.01` maximum step to meet the recorded oracle tolerance, whereas the C++ Taylor solve has no equivalent limit. Treat this as an identified performance target, not as a like-for-like algorithm comparison.

Low-thrust legs

Phase	Workload	Rust	Warmed C++/heyoka
14	Five-segment Sims–Flanagan mismatch	1.002 µs	1.037 µs

Phase	Workload	Rust	Warmed C++/heyoka
14	Complete analytic mismatch Jacobian	4.241 µs	12.748 µs
15	20-segment normalized Kepler ZOH mismatch	23.67 µs	6.865 µs
15	Complete endpoint/control/time-grid Jacobian	493.55 µs	182.17 µs

The Phase 14 comparison uses the same endpoints, masses, controls, duration, propulsion parameters, gravity parameter, and `cut = 0.6`. Construction and validation are outside both timings.

The Phase 15 comparison uses the same states, chronological controls, time grid, constants, cut, and `1e-12` tolerance, with no maximum step. Rust integrates every segment independently and uses fixed-size numerical dynamics Jacobians. Both operations therefore remain visible optimization targets.

Python batch throughput

The Phase 16 release-wheel harness used 20,000 items and the median of nine samples:

Workload	Python scalar loop	NumPy batch	Batch improvement
`stumpff_c`	38.9 ns/item	14.6 ns/item	2.67×
Lagrange propagation	0.72 µs/item	0.09 µs/item	7.82×

These measurements include Python input/output conversion; they are not Rust-core timings. They show why throughput-sensitive Python code should use the explicit batch APIs.

Interpreting the results

These are not cross-language speed claims. CPU frequency was not fixed and the run is not a substitute for distributions collected under controlled affinity and power settings. The Julian arithmetic result is small enough that compiler optimization and timer resolution dominate its interpretation. The C++ column is an elapsed average from five million calls rather than a Criterion distribution; it is included as the required same-input orientation baseline, not as a statistically controlled language comparison.

Reproducing the measurements

Run the maintained harness with:

```
cargo bench -p pykep-core --bench foundation
cargo bench -p pykep-core --bench elements
cargo bench -p pykep-core --bench propagation
cargo bench -p pykep-core --bench mission
cargo bench -p pykep-core --bench integration
cargo bench -p pykep-core --bench dynamics
cargo bench -p pykep-core --bench legs
python python/benchmarks/wrapper_overhead.py
cargo run --release -p pykep-lambert-optimization-benchmark
cargo run --release -p pykep-taylor-benchmark
```

Each benchmark group has a distinct scope:

Group	Workloads
Foundation	Arithmetic kernels plus scalar elliptic/hyperbolic anomaly solvers and a 64-value loop
Elements	Scalar classical/equinoctial conversions, analytic Jacobians, and a 64-state loop
Propagation	Elliptic/hyperbolic Lagrange coefficients, universal variables, analytic STMs, and a 1,024-state loop
Mission	Transfers, flyby constraints/delta-v, Lambert branches, and scalar/batch ephemerides
Integration	Nominal six-state DOP853 and Taylor propagation plus STM paths
Dynamics	Evaluated right-hand sides, CR3BP nominal/variational propagation, ZOH schedules, and Pontryagin propagation
Legs	Sims–Flanagan mismatch/analytic gradients and generic ZOH mismatch/sensitivities

The foundation anomaly loop keeps branch-heavy iterative work separate from arithmetic kernels. Scalar propagation and STM APIs use fixed-size arrays and perform no heap allocation. Ephemeris measurements distinguish initialization, scalar, high-precision, and batch paths. The integration final-state callback does not retain internal steps.

Standalone Lambert optimization

The standalone Lambert optimization benchmark ports the fixed `easy.kttsp` leg from `pykep-lambert`. It measures deterministic objective throughput and then applies the native `fcmaes-core` CMA-ES and BiteOpt implementations to wait time and time of flight. Its source revision, physical constants, decision bounds, penalty, optimizer budget, and seed are printed with every run; see `tools/lambert-optimization-benchmark/README.md` for the complete protocol.

Cross-language comparison policy

C++ comparisons are added only when both sides execute identical input data, validation policy, branch families, tolerances, and output work. Initialization and batch throughput are reported separately from warm scalar latency.

Release stabilization evidence

Phase 18 adds a protocol-matched 100-sample Rust/C++ distribution, bootstrap median confidence intervals, CI regression limits, allocation/cache/ vectorization profiles, and five-point Python batch scaling. Full results and environment limitations are in stabilization.md.

Matched PyKEP benchmarks

The 0.1.5 benchmark tree was measured against PyKEP 3.1.0 at `c01e19debad5ea22b7079189f786f473046a79d5`. Warmed heyoka led on nine scalar Taylor fixtures. Rust led on both equinoctial Pontryagin fixtures and on the complete ZOH Jacobian in the three leg pilots. The nominal ZOH mismatch and full Jacobian give different results. Source workloads

Native scalar Taylor

The reference uses heyoka 7.10.0 ABI 42. The host was an Intel Xeon Platinum 8573C under Linux. The Rust binary uses Rust 1.88, release thin LTO and the portable x86-64 target with SIMD compiled in. The native driver uses Clang 20.1.2, libstdc++ 13 and `-O3 -march=x86-64`. Heyoka generates its model kernels at runtime. The SDK used a development-only header adapter to link the Clang build to the packaged heyoka ABI. Each process is pinned to one logical CPU. The study uses ten paired rounds in alternating order. Upstream resets and reuses a warmed integrator. The raw samples include separate cold setup, first call and reset-only results.

Both arms use the same initial states, parameters, times and tight DOP853 endpoint vector. Both requested `1e-12` and passed the `1e-11` scaled endpoint-error gate. The ratio is the median of paired Rust / upstream times. The interval resamples whole pairs.

Model	Rust µs	Warmed heyoka µs	Rust / heyoka time	95% paired interval	Scaled errors Rust / heyoka
kepler	4.6	2.02	2.26×	1.73–3.31	6.7e-16 / 7.3e-15
cr3bp	10.8	4.59	2.57×	1.98–3.24	8.6e-15 / 5.1e-15
bcp	7.01	4.22	1.94×	1.31–2.22	6.9e-16 / 6.0e-15
zoh_kepler	5.11	2.92	1.77×	1.25–2.11	2.8e-15 / 8.1e-15
zoh_cr3bp	11.2	6.54	1.67×	1.50–1.84	3.0e-15 / 1.1e-14

Model	Rust µs	Warmed heyoka µs	Rust / heyoka time	95% paired interval	Scaled errors Rust / heyoka
zoh_equinoctial	15	6.73	2.19×	1.69–3.12	3.0e-15 / 3.0e-15
solar_sail	21.3	9.09	2.49×	2.00–2.85	7.1e-15 / 1.1e-14
cartesian_mass	8.88	4.02	2.28×	1.91–2.54	5.6e-16 / 5.6e-16
cartesian_time	8.24	3.72	2.27×	1.71–2.39	2.3e-15 / 2.3e-15
equinoctial_mass	536	673	0.78×	0.69–0.87	9.2e-14 / 2.8e-13
equinoctial_time	516	623	0.83×	0.75–0.91	9.1e-14 / 2.8e-13

Seven upstream benchmark programs

The full-size shared-input matrix uses the upstream case counts. The runners check finiteness and aggregate output counts for every case and retain complete sampled vectors. The numbers are orientation results from one ABBA block. Small differences are sensitive to host load.

Workload	Cases per row	Upstream / Rust time by case band	Max sampled scaled difference
Anomaly conversion	1,000,000	1.21× / 1.25× / 0.69×	1.1e-15
Anomaly round trip	100,000	0.48× / 1.15× / 1.47×	9.4e-16
Lagrange state	1,000,000	1.31× / 2.94× / 1.15× / 2.28×	2.4e-15
Lagrange round trip	100,000	1.54× / 1.23× / 1.57×	5.1e-15
State and 6 × 6 STM	100,000	2.47× / 2.49× / 1.58×	3.5e-14
Lambert 0 / ≤20 revolutions	50,000	0.84× / 0.79×	3.3e-16

Workload	Cases per row	Upstream / Rust time by case band	Max sampled scaled difference
SF simple mismatch	1	1.33×	1.9e-16
SF complete Jacobian	1,000	4.05× / 4.47× / 3.29× / 2.53×	1.8e-15
SF numerical UDP gradient	1,000	1.05× / 1.14× / 1.12× / 1.17×	1.1e-07

The initial sweep found convergence exhaustion in the previous Lagrange anomaly solve. With safeguarded Newton iteration, the repeat passed the failure-count, output-count and sampled-value gates. The measurement archive preserves the initial failed cases and the repeat.

Complete ZOH legs

Each segment count uses 200 shared cases, seed 424242, cached legs and five passes per arm. The complete Jacobian includes both endpoint-state blocks, every segment control and all grid-node derivatives. The table uses `1e-13` for both variational tolerances and `1e-14` for nominal integration. The measurement archive also records the source `1e-8` derivative setting and a separate Rust DOP853 arm. That arm predates the analytic DOP853 model Jacobians for the Kepler and CR3BP ZOH models in 0.1.5; the Taylor rows are unaffected.

Segments	Operation	Rust µs	Upstream µs	Upstream / Rust time	Sampled scaled difference
20	mismatch	36.9	23.9	0.65×	4.4e-16
20	jacobian	272	556	2.05×	1.9e-15
40	mismatch	67.2	51.2	0.76×	2.2e-16
40	jacobian	525	1.16e+03	2.22×	1.0e-15
80	mismatch	126	102	0.81×	2.2e-16
80	jacobian	1.1e+03	2.13e+03	1.93×	1.6e-15

Python gym evaluation

Four TOPS cases use 16 segments, uniform time and the same first vector in each Python implementation. The source variational tolerance is `1e-13`. Fitness and derivative agreement was checked on four vectors for each case. Each cell is the median of seven inner samples in

one process. The measurement archive has the softmax results, two classical examples, construction, ordered batch, fitness plus gradient, and history timings.

Problem	Rust fitness µs	PyKEP fitness µs	Rust gradient µs	PyKEP gradient µs
tops.twobody.P4	27.2	61.2	301	670
tops.mee.P8	5.03e+03	2.9e+03	3.8e+04	8.28e+04
tops.cr3bp.P0	434	817	5.49e+03	1.21e+04
tops.sail.P4	1.54e+03	1.01e+03	6.91e+03	1.1e+04
gym.eve_mga1dsm	6.11	66.7	—	—
gym.juice_mo	22.6	530	—	—

Gym validation

The catalog exposes 82 stable IDs. The primary comparison covers 14 classical configurations and both time encodings for each of the 68 TOPS IDs. It uses 16 segments and two vectors per configuration. A five-segment DOP853 arm exercises an alternative cut and direction mode.

Source catalog	Configurations	Direct pass	Rust extension	Discrepancy	Nonfinite/
Classical	14	14	0	0	0
Fixed TOPS	68	68	0	0	0
Moving TOPS	68	34	14	12	8

The moving discrepancies are ten moving-MEE rows and two moving-CR3BP stress vectors whose trajectories pass close to the Moon. The CR3BP pair differs from the source gradient by a scaled `1.07e-6` and `4.23e-6`, inside the `2.4e-5` spread of the Rust Taylor gradient across tolerances on the same vectors; see TOPS known limitations. Moving CR3BP softmax has a separate Rust formulation ID and an independent gradient check. The source moving CR3BP constructor selects uniform time. Moving MEE retains the original source discrepancies and a separate comparison using the source symbolic MEE conversion and finite differences. The P5 and P8 fixed-time anchors have off-anchor full-column checks. TOPS guide

Optimizer comparisons

The same native fcmaes DE-to-CMA adapter ran the two Rust backends on four fixed-boundary TOPS cases with 20 held-out seeds per case and budget. The fixed-work arm requests 128 evaluations. The wall arm uses an 80 ms budget checked between populations. The four feasible tutorials use a separate least-squares run and a tighter independent forward replay.

Case	Transcription	128 calls, ms Taylor / DOP853	80 ms, calls Taylor / DOP853	Runs with feasible candidate
tops.twobody.P4	16 uniform	9.24 / 14.2	1616 / 896	0/80
tops.mee.P8	24 softmax	350 / 141	32 / 96	0/80
tops.cr3bp.P0	30 uniform	85.2 / 72.8	128 / 192	0/80
tops.sail.P4	24 softmax	103 / 51.2	128 / 224	0/80

For Sims–Flanagan, both Python evaluator arms use the same PyGMO NLopt SLSQP, starting points, `1e-8` raw feasibility threshold and maximum 1,000 solver evaluations. Each segment count has 100 starts. The raw counts include the calls made by numerical differentiation.

Segments	Rust analytic	PyKEP analytic	Rust numerical	PyKEP numerical
5	84/100	84/100	84/100	84/100
10	83/100	83/100	83/100	83/100
15	84/100	84/100	85/100	85/100

Benchmark notebooks

The five-segment SF notebook checks all Jacobian groups against both the C++ leg and symbolic heyoka. Its legacy Isp assignment uses the current exhaust-velocity constructor in the reproduction. Complete derivative differences were below `2.4e-13` in both checks.

The provider study ran TLE and the available SPICE notebook cells as reference-only rows. The SPICE wrapper batch cell fails on its list-to-time multiplication. The separately labelled provider comparison covers Keplerian, JPL low-precision and VSOP2013 scalar and batch paths.

The measurement archive attached to the 0.1.5 release contains the paired samples, per-case gym results, optimizer records, fixture hashes and SDK provenance, including the development-only heyoka header adapter. Run the comparisons

Stabilization and release-candidate evidence

This document is the historical Phase 18, pre-0.1.0 evidence snapshot that complements numerical parity and invariant testing. Its artifact versions and then-open blockers are intentionally preserved as the record of that release candidate; they do not describe the current release state. The synchronized 0.1.4 artifacts were subsequently published, followed by 0.1.5. See Implementation status and the release procedure for the current state and procedure.

Matched Rust/C++ distribution

The representative five-segment Sims–Flanagan case uses byte-for-byte equivalent inputs and output work in the Rust release tool and an external C++ oracle harness. Both ran sequentially on CPU 0 with one thread, a three-second warmup per workload, 100 samples, 10,000 mismatch calls/sample, and 1,000 gradient calls/sample. Rust used 1.97.1 with the workspace release profile (`opt-level=3`, thin LTO, one codegen unit); C++ used GCC 14.3.0 and CMake Release. The host is an AMD Ryzen 9 9950X under Linux 6.8.0-136.

The host governor reported `powersave`; CPU frequency was not fixed. Hardware performance counters were unavailable because `perf_event_paranoid=4`. Accordingly, these are same-session orientation distributions, not universal language-speed claims. The deterministic analysis script reports a percentile-bootstrap 95% interval for the median:

Implementation	Workload	N	Mean	Median	P05–P95
Rust	Sims–Flanagan mismatch	100	1,014.60 ns	1,013.85 ns	1,012.53–1,019.03 ns
Rust	Sims–Flanagan gradient	100	4,287.93 ns	4,253.52 ns	4,234.28–4,334.24 ns
C++	Sims–Flanagan mismatch	100	1,051.25 ns	1,046.24 ns	1,044.23–1,082.56 ns
C++	Sims–Flanagan	100	13,720.82 ns	12,831.45 ns	12,767.07–20,305.14

Implementation	Workload	N	Mean	Median	P05–P95
	gradient				ns

Reproduce the Rust samples and table with:

```
cargo run --release -p pykep-release-benchmark > rust.csv
python tools/analyze_benchmark.py Rust=rust.csv C++=cpp.csv
```

The C++ oracle remains outside the public repository by policy.

Regression policy

CI runs an 11-sample/100-ms-warmup smoke protocol. Median limits are 10 µs for mismatch and 45 µs for the gradient, about 9.9× and 10.6× the local baselines. They intentionally catch order-of-magnitude regressions while leaving room for shared-runner scheduling and CPU differences. They are not used for cross-language assertions or small optimization claims.

Allocation, cache, vectorization, and scaling

Valgrind 3.22.0 found zero memory errors and zero definitely/indirectly/ possibly lost bytes; one 544-byte runtime block remained reachable at exit. DHAT quick-protocol aggregates were:

Workload	Allocated bytes	Blocks	Peak live
mismatch only	547,760	13,609	1,938 bytes / 9 blocks
gradient only	24,615,408	116,945	6,898 bytes / 33 blocks

The gradient returns dynamic output matrices and is the clear allocation target. Cachegrind over both quick workloads recorded 335,788,370 instructions, 133,876,521 data references, 3,775 L1 data misses, 5,590 last-level misses, and a 2.2% branch-mispredict rate. LLVM loop-vectorization remarks and emitted IR showed no packed-double vector loop in this workload. No algorithm was changed: the active parity and invariant suite is green, and the profile points to API output storage rather than numerical formulas.

Release-wheel Lagrange batch scaling (median of 11) was:

Batch	ns/item	items/s
1	590.00	1.69 million

Batch	ns/item	items/s
16	138.12	7.24 million
256	97.00	10.31 million
4,096	99.45	10.06 million
65,536	93.51	10.69 million

Dynamic analysis and fuzzing

- Miri nightly passed selected fixed-layout, derivative-consistency, error-formatting, and epoch-parser tests. Native tests remain authoritative for exact floating-point references because Miri software floating point can differ by a few ULPs.
- Valgrind Memcheck passed the release benchmark with zero errors.
- Ten-second libFuzzer/AddressSanitizer campaigns completed without a crash: 13,016,052 epoch-parser inputs, 10,115,059 element-conversion inputs, and 4,588,894 Lambert inputs. The 2026-07-26 review follow-up added a Reynolds-STM overflow target, which completed 7,197,911 inputs without a crash.
- RustSec and cargo-deny pass. `paste 1.0.15` remains an allowed unmaintained transitive dependency under `RUSTSEC-2024-0436`, documented in ADR 0004.

The fuzz harness is excluded from released artifacts. Its native compiler and sanitizer are development tools, not core build/runtime dependencies.

Public API documentation

The Rust core denies missing documentation for public items, and rustdoc builds with warnings denied. The Python audit inventories all 145 exported names; each is represented in the checked type stub and directly exercised by the integration suite. Public module, object, method, and descriptor docstrings are checked alongside that inventory. The stub audit compares runtime parameter names, order, and defaults and requires return annotations.

The 2026-07-26 review follow-up measured 376/376 documented Rust items (100.0%) and ten compiled examples, up from one. Rustdoc's per-item example metric is 5.5% because examples are intentionally placed on major module and type landing pages rather than repeated on every field and accessor. Core coverage is 91.01% regions, 92.26% functions, and 90.26% lines. The formerly weak Lambert module rose from 74.86% to 88.07% line coverage; `ephemeris/mod.rs` is 97.48% and `error.rs` is 100%.

The migration matrix records every unavailable upstream name with a technical reason and tracking identifier. External review is still required before declaring these public names frozen.

Local artifact consumption

`cargo package` produced and verified an 80-file, 2.3 MiB compressed `pykep-core-0.1.0.crate`. Its normal/build graph contains only Rust crates and no `cc`/CMake/native-library dependency. The extracted archive—not the workspace source—compiled and ran from a fresh Cargo binary project.

Maturin produced a 2.9 MiB CPython 3.12 manylinux wheel and a 2.4 MiB sdist. The wheel and the wheel built from the sdist both imported from separate fresh virtual environments. The packaged stub and `py.typed` marker were present. `ldd` reported only `libgcc_s`, `libm`, `libc`, and the ELF loader; there was no C++/kep3/Boost/heyoka runtime. This is the local Linux result. The hosted matrix is configured to build and consume wheels for CPython 3.11–3.13 on Linux, macOS, and Windows, but is not represented here as an executed remote run.

Historical external blockers

At the time of this snapshot, public names were not declared permanently frozen and external API review was still required. The repository URL and owner had been recorded, while the private security contact, registry-side trusted publishers, registry uploads, download checks, and release tag still required release-owner authority. Those publication blockers were later resolved; the paragraph remains to explain what the Phase 18 evidence did and did not establish.

Development

See the reference SDK and benchmark instructions for the pinned comparative studies. Normal Cargo and Maturin builds use the Rust evaluator and bundled gym data.

The workspace MSRV is Rust 1.88.0. The normal local quality gate is:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace --all-features --locked
RUSTDOCFLAGS="-D warnings" cargo doc --workspace --all-features --no-deps
cargo +1.88.0 check --workspace --locked
```

Build the GitHub Pages documentation with the same mdBook 0.5.4 release used by `.github/workflows/docs.yml`:

```
python tools/check_markdown_links.py
mdbook build
```

Build and verify the complete PDF edition after the HTML book is current:

```
python scripts/build_book_pdf.py
```

The PDF builder needs Chrome or Chromium plus `mutool`. It creates a navigable bookmark tree, keeps chapter links inside the book, converts links to repository artifacts into portable GitHub URLs, and writes `docs/pykep-rust-book.pdf`.

Run benchmarks separately so timing work is never hidden in the test suite:

```
cargo bench -p pykep-core
cargo run --release -p pykep-release-benchmark -- --quick --check
python python/benchmarks/wrapper_overhead.py --scaling
```

The standalone Phase 10 candidate comparison is reproducible with:

```
cargo run --release --manifest-path tools/phase10-candidates/Cargo.toml
```

For Python integration:

```
python -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install "maturin[patchelf]>=1.7,<2" pytest
env -u CONDA_PREFIX VIRTUAL_ENV="$PWD/.venv" \
  PATH="$PWD/.venv/bin:$PATH" \
  .venv/bin/maturin develop --release \
  --manifest-path crates/pykep-py/Cargo.toml
.venv/bin/python -m pytest
```

Coverage:

```
cargo llvm-cov -p pykep-core --all-features --all-targets --summary-only
cargo llvm-cov --workspace --all-targets --summary-only
```

Release-candidate dynamic checks:

```
cargo +nightly miri test -p pykep-core --lib --no-default-features
fixed_matrix_operations
cargo +nightly fuzz run epoch_parser -- -max_total_time=30
cargo +nightly fuzz run element_conversions -- -max_total_time=30
cargo +nightly fuzz run lambert_inputs -- -max_total_time=30
cargo +nightly fuzz run reynolds_stm -- -max_total_time=30
valgrind --error-exitcode=1 --leak-check=full \
  target/release/pykep-release-benchmark --quick
```

The full protocol, profiler commands/results, and Miri floating-point scope are documented in stabilization.md. Packaging and clean-artifact consumption are documented in `RELEASE.md`.

Build state belongs in `target/` or `.venv/` and is ignored. Development-only C++ oracle tools and internal planning notes are not part of this standalone repository.

For the model contract and the complete definition of done for a new dynamics family, see Adding an ODE system.

Adding an ODE system

This guide describes the complete path for adding a first-order ordinary differential equation to `pykep-core`, including the evaluated Rust model, sensitivities, the optional fixed-system Taylor backend, batches, Python bindings, tests, benchmarks, and documentation.

The central contract is

```
dy/dt = f(t, y, p)
```

where the state has a compile-time dimension `N` and the constant parameter vector has a compile-time dimension `P`.

Decide the scope first

There are two materially different extension levels.

External or DOP853-only model

Any downstream crate can implement `DynamicsModel<N, P>` and pass the model to `Dop853`. This is the stable public extension point. It supports nominal propagation, dense output, events, and, after implementing `DifferentiableDynamicsModel`, direct variational sensitivities.

An external model cannot implement Taylor support. `TaylorCoefficientModel` is private and `TaylorDynamicsModel` is deliberately sealed. This prevents the crate from promising a symbolic/coefficient API before it has a stable third-party contract.

Built-in pykep model

A model shipped by this repository normally needs the complete product surface:

- evaluated Rust right-hand side;
- domain validation and typed errors;
- state and parameter Jacobians when sensitivities are meaningful;
- Taylor coefficients and the `TaylorDynamicsModel` marker;
- convenience propagation methods;
- ordered scalar and parallel-batch APIs where users will evaluate many independent cases;

- Python scalar and batch bindings if the model is part of the Python product;
- independent validation, performance evidence, Rustdoc, mdBook documentation, source-map/status updates, and a changelog entry.

Do not add a model to the default `AdaptiveIntegrator` surface until its Taylor implementation exists and has been validated. A deliberately DOP853-only built-in model must use `Dop853` directly and document that limitation.

1. Write down the numerical contract

Before coding, record:

1. The model name and source equation or upstream reference.
2. `N`, the state dimension, and the exact order of every state component.
3. `P`, the parameter dimension, and the exact order of every parameter.
4. Units, normalization, coordinate frame, epoch/time convention, and sign conventions.
5. The valid domain of every state and parameter.
6. Singular geometries and non-analytic switching surfaces.
7. Whether the equations depend explicitly on time.
8. Conserved quantities, symmetries, closed-form cases, equilibria, or reversible cases that can serve as independent checks.
9. Whether analytic state and parameter Jacobians are available.
10. Whether users need dense output, events, sensitivities, ZOH control, batches, or Python access.

Treat state and parameter ordering as API. Changing it later is a breaking change even if the Rust type remains `[f64; N]`.

2. Add the evaluated Rust model

Place a substantial new family in `crates/pykep-core/src/dynamics/<system>.rs` and export the module from `crates/pykep-core/src/dynamics.rs`. A small closely related model can live beside its family.

Prefer a zero-sized, copyable model:

```rust
use pykep_core::integration::{DynamicsModel, Dop853, InitialValueProblem};
use pykep_core::integration::IntegratorOptions;
use pykep_core::{PykepError, Result};

#[derive(Clone, Copy, Debug, Default, PartialEq, Eq)]
pub struct OscillatorDynamics;

impl DynamicsModel<2, 1> for OscillatorDynamics {
    const NAME: &'static str = "harmonic oscillator";

    fn validate(
        &self,
        time: f64,
        state: &[f64; 2],
        parameters: &[f64; 1],
    ) -> Result<()> {
        if !time.is_finite() {
            return Err(PykepError::NonFiniteInput { parameter: "time" });
        }
        if state.iter().any(|value| !value.is_finite()) {
            return Err(PykepError::NonFiniteInput { parameter: "state" });
        }
        if !parameters[0].is_finite() {
            return Err(PykepError::NonFiniteInput {
                parameter: "angular_frequency",
            });
        }
        if parameters[0] <= 0.0 {
            return Err(PykepError::InvalidInput {
                parameter: "angular_frequency",
                reason: "must be greater than zero".into(),
            });
        }
        Ok(())
    }

    fn rhs(
        &self,
        time: f64,
        state: &[f64; 2],
        parameters: &[f64; 1],
        derivative: &mut [f64; 2],
    ) -> Result<()> {
        self.validate(time, state, parameters)?;
        let omega = parameters[0];
        *derivative = [state[1], -omega * omega * state[0]];
        if derivative.iter().all(|value| value.is_finite()) {
            Ok(())
        } else {
            Err(PykepError::NumericalOverflow {
                operation: Self::NAME,
            })
        }
```

```
    }
}
```

Inside `pykep-core`, reuse the crate-private finite-input and finite-output helpers instead of duplicating them. External crates should construct the public `PykepError` variants as above.

The implementation rules are:

- `rhs` writes every derivative component into caller-owned storage.
- `rhs` must not allocate.
- Validate before divisions, roots, logarithms, normalizations, or other domain-sensitive operations.
- Do not silently clamp, normalize, or cross a physical singularity.
- Use `InvalidInput` for finite values outside the declared domain, `NonFiniteInput` for NaN/infinity, `SingularGeometry` for mathematically undefined geometry, and `NumericalOverflow` for a non-finite result from otherwise finite inputs.
- Keep model-domain errors distinct from `IntegrationFailure`.
- Return a stable, descriptive `NAME`; integration errors include it.
- Do not keep mutable propagation state in the model object. Constant model configuration belongs in the parameter array or in an immutable model value.

An external model is immediately usable with DOP853:

```
let result = Dop853.propagate(
    &OscillatorDynamics,
    InitialValueProblem::new(0.0, [1.0, 0.0], 10.0, [2.0]),
    IntegratorOptions::default(),
)?;
```

Use DOP853 for a general event callback. Built-in Cartesian models can also opt in to Taylor's analytic plane, sphere, and radial events.

3. Add Jacobians and sensitivity support

Implement `DifferentiableDynamicsModel<N, P>` when state-transition matrices, parameter sensitivities, or gradients are part of the model's use case:

```rust
use pykep_core::integration::DifferentiableDynamicsModel;

impl DifferentiableDynamicsModel<2, 1> for OscillatorDynamics {
    fn jacobians(
        &self,
        time: f64,
        state: &[f64; 2],
        parameters: &[f64; 1],
        state_jacobian: &mut [[f64; 2]; 2],
        parameter_jacobian: &mut [[f64; 1]; 2],
    ) -> Result<()> {
        self.validate(time, state, parameters)?;
        let omega = parameters[0];
        *state_jacobian = [[0.0, 1.0], [-omega * omega, 0.0]];
        *parameter_jacobian = [[0.0], [-2.0 * omega * state[0]]];
        Ok(())
    }
}
```

Matrix layout is output-by-input:

```
state_jacobian[i][j]     = d f_i / d state_j
parameter_jacobian[i][k] = d f_i / d parameter_k
```

Always overwrite both complete output matrices. Never assume the caller provided zeros.

Prefer analytic Jacobians. If a family uses numerical Jacobians, reuse the existing scale-aware helper and list every strictly positive state or parameter index. Numerical differentiation must not step across a mass, radius, barrier, or other one-sided domain.

DOP853 integrates these Jacobians directly. The Taylor path differentiates the coefficient recurrence using seeded tangent scalars and uses the tangent coefficients in its step controller.

4. Add convenience APIs

For a public built-in model, mirror the shape of the existing dynamics APIs:

- `evaluate`;
- `propagate`;
- `propagate_with_method`;
- `propagate_with_stm` or a general sensitivity method;
- `propagate_with_stm_method`;
- model-specific invariants or controls, if they are already part of the underlying physics.

The no-suffix `propagate` method may use `AdaptiveIntegrator::default()` only after Taylor support is complete. The method-selecting variant should accept `IntegrationMethod`, and documentation must say which method is the default.

Avoid inventing derived physics merely to make an API appear symmetrical. Expose only quantities actually defined by the implemented system.

5. Add the fixed-system Taylor evaluator

This section applies only to models shipped inside `pykep-core`.

Add the model to `crates/pykep-core/src/integration/taylor/systems.rs` by implementing the private coefficient contract and the public sealed marker:

```
impl TaylorCoefficientModel<2, 1> for OscillatorDynamics {
    fn generated_coefficients<T: Coefficient>(
        &self,
        time: T,
        state: &[T; 2],
        parameters: &[T; 1],
        order: usize,
        jet: &mut [[T; MAX_ORDER + 1]; 2],
    ) {
        generated::oscillator(time, state, parameters, order, jet);
    }

    fn coefficients(
        &self,
        time: f64,
        state: &[f64; 2],
        parameters: &[f64; 1],
        order: usize,
        jet: &mut [[f64; MAX_ORDER + 1]; 2],
    ) -> Result<()> {
        self.validate(time, state, parameters)?;
        self.generated_coefficients(time, state, parameters, order, jet);
        Ok(())
    }
}

impl TaylorDynamicsModel<2, 1> for OscillatorDynamics {}
```

Register the model graph and dimensions in the ignored `export_taylor_graphs` developer task. Run `python tools/taylor-codegen/regenerate.py` and validate the generated jet against the tape and an independent reference. A separately measured handwritten recurrence can still serve the nominal scalar path.

Specialized recurrence

Use a hand-written recurrence for a small equation where doing so clearly reduces work. The Kepler and ZOH Kepler implementations are the templates.

The recurrence must:

1. Clear `jet`.
2. Copy the initial state into coefficient zero.
3. Advance only coefficient `n + 1` from already available coefficients `0..=n`.
4. Divide the right-hand-side coefficient by `n + 1`.
5. Stop at the requested `order`, never beyond `MAX_ORDER`.

Provide an independent coefficient reference in tests. Hand-written recurrences without coefficient-level tests are not acceptable.

Cached expression tape

Use `TapeBuilder` for a larger fixed expression:

```
use std::sync::OnceLock;

fn oscillator_tape() -> &'static IncrementalTape<2> {
    static TAPE: OnceLock<IncrementalTape<2>> = OnceLock::new();
    TAPE.get_or_init(build_oscillator_tape)
}

fn build_oscillator_tape() -> IncrementalTape<2> {
    let builder = TapeBuilder::new();
    let outputs = {
        let position = builder.state(0);
        let velocity = builder.state(1);
        let omega = builder.parameter(0);
        [velocity, -omega * omega * position].map(Expression::index)
    };
    builder.finish(outputs)
}
```

The lexical block is important: `finish` consumes the builder after all borrowed expressions have been reduced to output indices.

The tape currently supports:

- constants, time, state components, and parameters;
- addition, subtraction, multiplication, division, and negation;
- real powers and square roots;

- exponentials;
- sine, cosine, and paired `sin_cos`;
- `stop_gradient`;
- reverse symbolic gradients.

Use `builder.time()` for explicit time dependence. Treating the initial time as a constant produces incorrect higher-order coefficients.

`TapeBuilder` performs structural common-subexpression elimination. Reuse the same algebraic form where practical, but do not obscure the equations solely to reduce the operation count.

If the system needs a new elementary operation, adding only an evaluated operation is insufficient. Extend all of these together:

1. the `Operation` enum;
2. the builder and `Expression` API;
3. constant folding;
4. the incremental Taylor recurrence;
5. reverse differentiation, when gradients can traverse the node;
6. companion/workspace metadata, if required;
7. direct elementary-function and derivative tests in `tape.rs`.

Piecewise expressions and optimal control

A Taylor series is valid only on an analytic branch. For algebraically equivalent stable formulas, keep one `OnceLock` tape per branch and select the branch from coefficient-zero state/parameter values at the start of each step. Test every branch and its boundary policy.

For minimized Pontryagin Hamiltonians, controls may evolve as Taylor series while remaining excluded from the Hamiltonian partial derivative. Wrap those direction or throttle expressions with `stop_gradient`, matching the envelope convention. Omitting this changes the costate equations.

Do not smooth a real discontinuity merely to make Taylor integration possible. Document the boundary and use segmented propagation or DOP853 when the physical model requires it.

6. Add ZOH support when applicable

If controls are constant over user-supplied segments, implement the internal `ZeroOrderHoldModel<N, C, K, P>` mapping:

```
parameters = parameters(control[C], constants[K])
```

Also map control and constant sensitivity seeds into parameter seeds. Define which side owns an exact switch time, validate the complete schedule once, and test forward and backward segment traversal.

Add the model to every relevant ZOH dispatch point:

- scalar segment propagation;
- dense/history output;
- sensitivities;
- leg support, if the state/control semantics match the leg;
- Python `ZohModel` selection, if exposed there.

Do not hide a control lookup inside `rhs`; a validated schedule should split the integration into constant-parameter segments.

7. Add ordered batch APIs

Provide a batch when callers will naturally evaluate many independent states, controls, epochs, or final times. A batch is an explicit API extension, not an implicit change to scalar behavior.

Use `pykep_core::batch::try_map` so that:

- `workers = 0` uses Rayon's global pool;
- `workers = 1` is deterministic serial execution;
- larger values use a cached fixed-size pool;
- outputs preserve input order;
- if multiple items fail, the earliest failing input is reported.

Test empty input, one item, multiple worker counts, mismatched input lengths, deterministic output order, and deterministic error order. Every batch result must match the scalar API for the same item.

Avoid nested parallelism. If the objective is parallelized at a higher level, call the scalar or `workers = 1` model API inside each objective evaluation.

8. Add Python bindings

Python exposure is optional for a low-level internal model and expected for a user-facing pykep model.

Implement bindings in `crates/pykep-py/src/dynamics.rs`:

1. Parse dynamically sized Python inputs into fixed Rust arrays before releasing the GIL.
2. Call the same `pykep-core` scalar implementation; do not duplicate the equations in the binding.
3. Convert `PykepError` with the shared `to_python` mapping.
4. Release the GIL with `Python::detach` for propagation and batch work.
5. Use NumPy `N × state_dimension` arrays for batch states.
6. Accept `workers` on parallel batches and preserve input order.
7. Register every function in `dynamics::register`.

Then update:

- `python/pykep_rust/__init__.py` imports and `__all__`;
- `python/pykep_rust/_pykep_rust.pyi` signatures and docstrings;
- `python/tests/test_smoke.py` for scalar values and error mapping;
- `python/tests/test_parallel_batch.py` for scalar/batch parity, shapes, worker counts, and deterministic failures;
- a Python example when the usage is not obvious.

Use consistent names:

```
<system>_rhs
<system>_rhs_batch
propagate_<system>
propagate_<system>_batch
propagate_<system>_with_stm
propagate_<system>_with_stm_batch
```

If the Rust model supports both DOP853 and Taylor but Python intentionally exposes only the default, state that explicitly. Do not expose internal tape objects.

9. Build the test pyramid

No single reference is sufficient. Add the following layers in order.

Evaluated right-hand side

- One or more hand-computed or authoritative reference states.
- Exact zero/control-free/equilibrium reductions where available.
- Explicit-time cases at more than one time.
- Forward-frame and sign-convention checks.
- Every invalid parameter range.
- Every singular geometry.
- NaN and infinity for time, state, and parameters.
- Finite inputs that would overflow the result.

Put small local tests beside the implementation. Put source-parity and cross-family tests in a dedicated `crates/pykep-core/tests/phase*_*.rs` integration test.

Jacobians

Compare every state and parameter column with independent, scale-adjusted central differences over several non-singular states. Use perturbations based on each variable's scale, not one absolute epsilon for all columns.

Also test:

- the documented output-by-input layout;
- analytically zero columns;
- one-sided positive domains;
- consistency between the Jacobian and propagated sensitivities.

Propagation

Use at least two independent checks:

- a closed-form solution, invariant, upstream trajectory, or independently generated fixture;
- a same-problem DOP853 comparison at a tighter reference tolerance.

Cover forward and backward time, zero duration, dense sampling, step limits, rejected steps when relevant, and a duration long enough to expose drift or branch errors. Validation tolerances must be justified by the state scale and reference accuracy; they are not general user guarantees.

Taylor coefficients

Keep a test-only full-series form of the right-hand side and compare its jet with the optimized recurrence/tape at representative orders such as `[8, 15, MAX_ORDER]`.

The test should verify:

- every state component and every coefficient;
- explicit-time coefficients;
- trigonometric/exponential/power paths used by the model;
- every stable algebraic branch;
- an operation-count ceiling for expression tapes.

Choose a scaled relative comparison:

```
abs(incremental - reference) <= tolerance * max(abs(reference), 1)
```

High-order coefficients of poorly scaled systems can be ill-conditioned. Use a realistic state, record why a looser coefficient tolerance is needed, and retain the independent propagated-state comparison.

Cross-backend and regression tests

Add the model to the central Taylor-versus-DOP853 tests. If an upstream heyoka/C++ implementation exists, generate committed fixtures with a pinned upstream version and keep the generator outside the normal runtime dependency graph.

Compare physical outputs, not only internal work counters. Taylor coefficient sweeps and DOP853 RHS evaluations are not equivalent units.

Batch and Python tests

For every scalar Python function, test the corresponding batch when one exists. Verify dtype/shape, empty batches, mismatched lengths, exception classes, worker behavior, and equality to scalar calls.

10. Add benchmarks and apply a performance gate

Add Criterion entries in `crates/pykep-core/benches/dynamics.rs` for:

- one RHS evaluation;
- one representative DOP853 propagation;
- one Taylor propagation for a Taylor-capable built-in;
- one state/parameter sensitivity propagation when it is important;
- representative scalar and batch throughput.

Benchmark release builds. Warm lazy `OnceLock` tapes before recording steady state, but report first-call setup separately if it is material.

For an optimization, declare the benchmark, correctness tolerance, and minimum meaningful gain before changing the implementation. Pin:

- initial time, state, parameters, and final time;
- integration tolerances and maximum step;
- release profile and dependency versions;
- iteration/sample counts;
- CPU affinity when comparing small kernels;
- final-state checksum or independent accuracy measure.

Keep an optimization only when the gain is larger than run-to-run noise and all accuracy checks remain satisfied. Record development-host timings as evidence, not portable latency promises.

11. Document the model

Every public type and method needs Rustdoc describing:

- state and parameter order;
- units and reference frame;
- default integration backend;
- singularities and domain restrictions;
- output layout;
- `# Errors`;
- a short runnable example for a non-obvious API.

Update the relevant mdBook pages:

- `docs/dynamics.md`, `docs/zero-order-hold.md`, or `docs/pontryagin.md`;
- `docs/taylor-integration.md` when Taylor support changes;
- `docs/python-api.md` and `docs/python-migration.md` for Python;
- `docs/batch-processing.md` for a new batch family;
- `docs/validation.md` with the independent evidence and tolerances;
- `docs/performance.md` with the benchmark protocol and interpretation;

- `docs/status.md` and `docs/source-map.md`;
- `docs/SUMMARY.md` for any new page;
- `CHANGELOG.md` under `Unreleased`.

Update model counts such as “eleven built-in models” wherever the new model changes them. Search for the old count rather than fixing only one page:

```
rg -n "eleven|11 built-in|supported models|TaylorDynamicsModel" \
  README.md CHANGELOG.md docs crates
```

If the model ports an upstream equation, add the exact source file, upstream version/commit, and Rust destination to `docs/source-map.md`.

12. Run the complete quality gate

From the `public` repository:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace --all-features --locked
cargo test --workspace --all-features --locked --release
cargo test -p pykep-core --no-default-features --locked
RUSTDOCFLAGS="-D warnings" cargo doc --workspace --all-features --no-deps
cargo +1.88.0 check --workspace --locked
python tools/check_markdown_links.py
mdbook build
```

For Python changes, build the extension in an isolated environment and run the Python suite:

```
python -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install "maturin[patchelf]>=1.7,<2" pytest
env -u CONDA_PREFIX VIRTUAL_ENV="$PWD/.venv" \
  PATH="$PWD/.venv/bin:$PATH" \
  .venv/bin/maturin develop --release \
  --manifest-path crates/pykep-py/Cargo.toml
.venv/bin/python -m pytest
```

Also run the relevant Criterion benchmark and coverage report:

```
cargo bench -p pykep-core --bench dynamics
cargo llvm-cov -p pykep-core --all-features --all-targets --summary-only
```

Use Miri for new unsafe-sensitive structure or recurrence code if applicable. The crate forbids unsafe code, but Miri can still catch invalid assumptions in dependency-free state manipulation. Fuzz new parsers or dynamically shaped input boundaries; do not fuzz a fixed smooth RHS merely to increase a metric.

Definition of done

A built-in ODE system is complete only when:

- [ ] State, parameter, unit, frame, and domain contracts are written down.
- [ ] `DynamicsModel` is allocation-free, validated, and fully documented.
- [ ] Errors use the stable public taxonomy.
- [ ] Jacobians are implemented and independently checked, or their absence is explicitly justified.
- [ ] Taylor support is implemented and coefficient-tested, or the model is explicitly documented as DOP853-only.
- [ ] Propagation matches at least two independent references or invariants.
- [ ] Forward, backward, zero-duration, singular, and non-finite cases pass.
- [ ] Every analytic branch and control switch policy is tested.
- [ ] Scalar and batch APIs agree for all worker modes.
- [ ] Python functions, stubs, exports, error mapping, and tests are complete when Python exposure is in scope.
- [ ] RHS, propagation, sensitivity, and batch benchmarks are recorded where relevant.
- [ ] Rustdoc, mdBook, status, source map, validation, performance, and changelog are updated.
- [ ] Formatting, Clippy, debug/release/no-default tests, Rustdoc, MSRV, Markdown links, mdBook, Python tests, and relevant coverage checks pass.
- [ ] The `public` worktree contains only intended changes and the work is committed.

ADR 0001: Fixed numerical types

- Status: accepted
- Date: 2026-07-25

Context

The common pykep values are three-vectors, six-element Cartesian states or orbital elements, and small fixed matrices. Dynamic allocation and a general-purpose public matrix type would add cost and couple the API to a dependency without improving these fixed-shape contracts.

Decision

Parity work starts with documented aliases over stack-allocated Rust arrays:

- `Vector3 = [f64; 3]`;
- `CartesianState = [f64; 6]`;
- `Elements6 = [f64; 6]`;
- `Matrix3 = [[f64; 3]; 3]`;
- `Matrix6 = [[f64; 6]; 6]`.

Small matrix helpers use const generics internally and in deliberately generic public operations. Variable-sized solution families, controls, grids, and batches use owned vectors. No external matrix type appears in the public API.

Consequences

The common hot path is allocation-free and Python conversion remains straightforward. Aliases do not prevent semantic mix-ups, so public functions must use meaningful parameter names and named result structs where multiple arrays would be ambiguous. Newtypes remain an option before API stabilization if later phases demonstrate enough safety benefit.

Phase 4 exercised that option for semantically distinct element sets: `ClassicalElements` and `ModifiedEquinoctialElements` provide named fields and lossless `[f64; 6]` conversions.

Cartesian states retain the fixed alias because their ordering is unambiguous in every consuming API.

ADR 0002: Numerical error taxonomy

- Status: accepted
- Date: 2026-07-25

Context

The C++ implementation mixes exceptions, NaN sentinels, and unchecked floating-point behavior. A mission-analysis library must not turn failed validation or convergence into a plausible output.

Decision

All fallible public numerical operations return `pykep_core::Result<T>` with a non-exhaustive `PykepError`. Stable categories distinguish:

- invalid values and non-finite values;
- singular geometry;
- convergence failure;
- dimension mismatch;
- unsupported capabilities;
- floating-point overflow;
- integration failure.

Input validation happens at public boundaries. Private, already-validated helpers may avoid repeated checks in iterative hot paths. Panics are reserved for violated internal invariants, not caller input.

The Python layer maps invalid values and dimensions to `ValueError`, numerical overflow to `OverflowError`, and exposes package exceptions for convergence, singular geometry, unsupported capabilities, and integration failures.

Consequences

This intentionally differs from upstream functions that return NaN for an invalid anomaly domain or accidentally return finite limit values for NaN inputs. Error strings retain useful context, but callers should match the category rather than punctuation.

ADR 0003: embedded thresholded VSOP2013 evaluator

- Status: accepted
- Date: 2026-07-25

Context

The upstream pykep provider asks heyoka to construct VSOP2013 expressions and JIT-compile a function for each `(body, threshold)` pair. That is not suitable for a C/C++-free Rust crate, reproducible offline builds, or bounded global state.

The authoritative IMCCE solution contains 2,607,947 terms. In heyoka's packed layout, retaining every term would require about 100 MiB before compression. The pykep tests evaluate at thresholds down to `1e-9`; 112,270 terms survive that floor and occupy 4.3 MiB in the committed encoding. The ordinary pykep default is `1e-5`, for which only 1,858 terms survive across all bodies.

The coefficient source in heyoka 7.10.0 is MPL-2.0-covered code generated from the IMCCE VSOP2013 release. The public data notice pins the exact heyoka commit, generator, retained threshold, and binary hash.

Decision

The default `vsop2013` Cargo feature embeds the original integer multipliers and binary64 coefficients down to `1e-9`. A provider validates and decodes only its selected planet, applies the requested threshold once, and owns the result through `Arc`. Evaluation is a direct Rust series evaluator followed by the published equinoctial-to-Cartesian transformation and ICRF rotation.

Thresholds below `1e-9` return an explicit validation error. Disabling default features removes the coefficient asset; construction then returns an explicit unsupported-capability error. Availability and the threshold floor are queryable in Rust and Python.

There is no process-global provider cache. Default-threshold initialization measured about 11.6 µs, so a cache would add synchronization and lifetime complexity without justifying itself.

Cloning an existing provider shares its immutable decoded data.

No source checkout, generator, network access, C++, LLVM, or heyoka is needed to build or run the public crate.

Measurements

An orientation run on an AMD Ryzen 9 9950X measured:

Workload	Rust	C++/heyoka JIT
default `1e-5` initialization	11.6 µs	64.0 ms
default scalar state	340 ns	166 ns
Rust default 256-state batch	89.6 µs	no batch API
`1e-9` initialization	not separately isolated	553 ms
`1e-9` scalar state	37.2 µs	5.91 µs
release benchmark executable size increase	about 4.4 MiB	requires shared heyoka/LLVM runtime

The Rust and C++ timing harnesses use the same Earth–Moon provider and epoch. These are orientation results without fixed CPU frequency, not release performance guarantees.

Across 54 golden states at `1e-9`, the largest absolute difference from the JIT-compiled upstream path was 0.185 m in position and `6.6e-8 m/s` in velocity. The difference is consistent with floating-point summation and trigonometric evaluation order.

Consequences

- Startup, builds, and runtime dependencies are bounded and reproducible.
- Keplerian and JPL low-precision providers remain available without the feature.
- The full sub-`1e-9` theory is intentionally not embedded because its data and evaluation cost are disproportionate to the upstream-tested public use.
- The direct evaluator is slower than optimized JIT code at `1e-9`; Phase 18 may profile argument precomputation, SIMD-friendly term grouping, and batch evaluation while golden tests remain active.

ADR 0004: adaptive integration backend

- Status: accepted
- Date: 2026-07-25

Context

The upstream `ta` modules return heyoka expression graphs and cached Taylor-adaptive integrators. Rust cannot preserve that implementation API without a C++/LLVM runtime. The remaining physical models and legs instead need an evaluated-model interface with:

- six-, seven-, fourteen-, and augmented state dimensions up to 126;
- constant parameters that can change exactly between ZOH segments;
- scalar relative and absolute tolerances down to the upstream test regimes;
- initial/maximum step and accepted/rejected-step limits;
- forward and backward propagation;
- deterministic exact-final-time stops;
- dense evaluation and terminal zero-crossing events;
- state and parameter Jacobians plus arbitrary first-order sensitivity seeds;
- no heap allocation per accepted step on nominal fixed-size hot paths.

Kepler and CR3BP were used as representative smooth and close-approach systems. ZOH switching is treated as a known discontinuity: a leg ends one solve exactly at a switch and starts the next with new parameters. It does not ask a continuous interpolant to cross a discontinuity.

Candidates

The research snapshot was taken on 2026-07-25.

`differential-equations` 0.6.1 provides pure-Rust DOP853, seventh-order dense interpolation, backward solves, terminal root finding, scalar/component tolerances, and explicit step and rejection limits. It accepts stack-allocated arrays and user-defined state types. Its Apache-2.0 license is compatible, and a no-default-feature build passes Rust 1.88 even though the crate does not declare an MSRV.

Its `simba` dependency currently brings `paste` 1.0.15. RustSec advisory RUSTSEC-2024-0436 marks that macro crate unmaintained but reports no vulnerability and no safe upgrade. The

exact advisory is narrowly allowlisted in `deny.toml` and remains visible in `cargo audit` output.

`ode_solvers` 0.6.2 provides pure-Rust DOP853 and dense output under Apache-2.0. The same-input spike was faster, but the implementation allocates vectors and intermediate states inside each solve/step. It has a post-step stop callback rather than interpolated root location and no first-order sensitivity interface.

`diffsol` 0.16.1 has the richest event, reset, dense-output, forward, and adjoint sensitivity system. It was screened out before the timing final because its implicit-solver and matrix abstractions are disproportionate for these small non-stiff explicit systems. Its optional DSL/JIT path also solves a problem this port explicitly avoids.

A local DOP853 implementation would give maximum workspace control but would make pykep responsible for a second adaptive-solver implementation, dense coefficients, root location, step controller, and long-term maintenance. C++ FFI was excluded by the release requirements.

Decision

Use `differential-equations` 0.6.1 DOP853 behind the pykep-owned `integration` facade. No backend type appears in a public signature.

`DynamicsModel<N, P>` evaluates into caller-owned arrays and validates the physical domain. `DifferentiableDynamicsModel<N, P>` adds row-major state and parameter Jacobians. `InitialValueProblem` owns time, state, and constant parameters. `SensitivityProblem` supplies arbitrary `dstate/dseed` and `dparameters/dseed` matrices, integrating

```
dS/dt = (df/dstate) S + (df/dparameters) (dparameters/dseed).
```

This directly represents STMs, parameter variations, and the state/control columns needed by ZOH legs. The facade provides final-state, dense-sampling, terminal-event, and sensitivity propagations with stable pykep errors and work counters.

The ordinary and variational final-state paths use fixed-size state storage and an output callback that does not retain internal steps. Dense and event APIs allocate returned samples intentionally; the event adapter currently retains accepted steps because the selected crate's event wrapper owns its output strategy. Events are not on the remaining leg hot path.

Measurements

All values are orientation measurements on the same AMD Ryzen 9 9950X host. CPU frequency was not pinned.

The Git-tracked candidate tool used 30 samples of 200 complete Kepler propagations with `rtol = atol = 1e-12`:

Candidate	Mean	Median	Range
pykep facade / `differential-equations`	15.431 µs	15.228 µs	14.882–18.109 µs
`ode_solvers`	7.369 µs	7.333 µs	7.299–7.901 µs

The maintained workspace Criterion profile measured the selected nominal path at 11.865 µs (95% estimate 11.849–11.884 µs) and the state plus 6 × 6 STM path at 85.663 µs (85.260–86.102 µs).

The matching C++/heyoka harness measured 3.722 µs mean nominal steady-state propagation and 84.440 µs mean for the STM. Cloning the cached C++ nominal integrator once cost 0.467 ms; the steady-state result excludes that clone.

Validation and risks

- A 100-orbit eccentric Kepler solve kept absolute specific-energy drift below `2e-10` at `1e-13` tolerances.
- Kepler final states and the STM match the independent analytic propagator; parameter columns match central finite differences.
- A CR3BP close approach preserves its Jacobi constant and reverses within scale-aware tolerances.
- Rejected steps, exhausted step limits, backward integration, dense samples, terminal roots, non-finite values, and bit-for-bit repeatability are tested.
- DOP853 is not a Taylor method. Equal numeric tolerance values do not imply equal internal steps or bitwise parity with heyoka.
- Nominal integration is about three times slower than warmed C++/heyoka in this case. The variational path is approximately equal. Phase 18 will profile before changing the solver.
- Dense interpolation error depends on internal step length as well as local solve tolerance. High-accuracy dense/event callers should set `maximum_step`; ZOH switches always use exact final-time solves.

- Close singularities may exhaust rejections or step size. Errors retain the model and integration context instead of returning a plausible state.

Consequences

The backend supports every remaining `ta` and generic ZOH-leg requirement without C++, LLVM, a JIT, global mutable caches, or a hidden thread pool. Physical Kepler, CR3BP, BCP, ZOH, and Pontryagin models remain Phase 11–13 work; completing this gate does not claim those APIs are implemented.

The dependency is deliberately hidden so it can be upgraded or replaced without changing model, problem, result, or Python contracts. The transitive `paste` maintenance advisory must be reconsidered on every backend upgrade; the allowlist must not be broadened to a vulnerability.

ADR 0005: Add Taylor as the built-in nominal backend

- Status: accepted
- Date: 2026-07-28
- Supersedes: ADR 0004's default for built-in nominal dynamics; DOP853 remains the general backend

Context

ADR 0004 selected DOP853 because it supplied final state, dense output, events, and direct variational equations behind a small pure-Rust dependency. The same evidence showed a nominal high-accuracy performance gap relative to the upstream warmed heyoka implementation.

The eleven pykep dynamics types come from only nine fixed equation families. They do not require arbitrary symbolic expressions, LLVM, events in Taylor mode, heyoka batch mode, or arbitrary precision. A disposable eccentric Kepler prototype passed a predeclared matched-error gate at tight tolerances.

Decision

Add a private fixed-capacity Taylor-series engine inside `pykep-core`. Implement coefficient kernels only for the built-in models. Publish `Taylor`, `IntegrationMethod`, and `AdaptiveIntegrator`, but keep the coefficient trait private until a real third-party system requires it.

Make Taylor the default of `AdaptiveIntegrator` and of the nominal Kepler/CR3BP/BCP convenience methods. This follows upstream pykep, where these TA-derived equation families are exposed as heyoka Taylor-adaptive integrators. Keep direct `Dop853` calls, generic user dynamics, event location, and the existing direct-variational sensitivity convenience methods on DOP853. Add explicit `*_with_method` variants for built-in dynamics and ZOH schedules.

Use tolerance-selected order 8–24 and component-scaled step selection from the final two coefficients. Preserve an optimized Kepler recurrence where full-series generic evaluation would erase the measured crossover.

For the first release, dense grids clip Taylor steps at sample times and seeded sensitivities use centered complete propagations. Document their cost instead of claiming the not-yet-implemented polynomial interpolation and direct Taylor variational optimizations.

The later development backend implements both polynomial interpolation and direct tangent recurrences. The original release decision is recorded here. Current Taylor guide · Current measurements

Consequences

- No C/C++ runtime, LLVM, JIT latency, or new runtime dependency is added.
- At `1e-14` and machine epsilon the representative Kepler solve is faster and more accurate than DOP853; at `1e-9` it is slower.
- Built-in nominal dynamics default to the algorithm family used by upstream pykep; callers can still request DOP853 explicitly.
- User-defined `DynamicsModel` implementations continue to use DOP853.
- Wide STMs and event-driven problems should continue to use DOP853.
- Equation duplication between evaluated and coefficient kernels is accepted as localized technical debt, protected by upstream, DOP853, and official-heyoka cross-validation.

The raw measurements and validation boundaries are recorded in High-accuracy Taylor integration.